

Algorithms

Lecture 7

Graph Traversal and Applications

Prantik Paul [PNP]

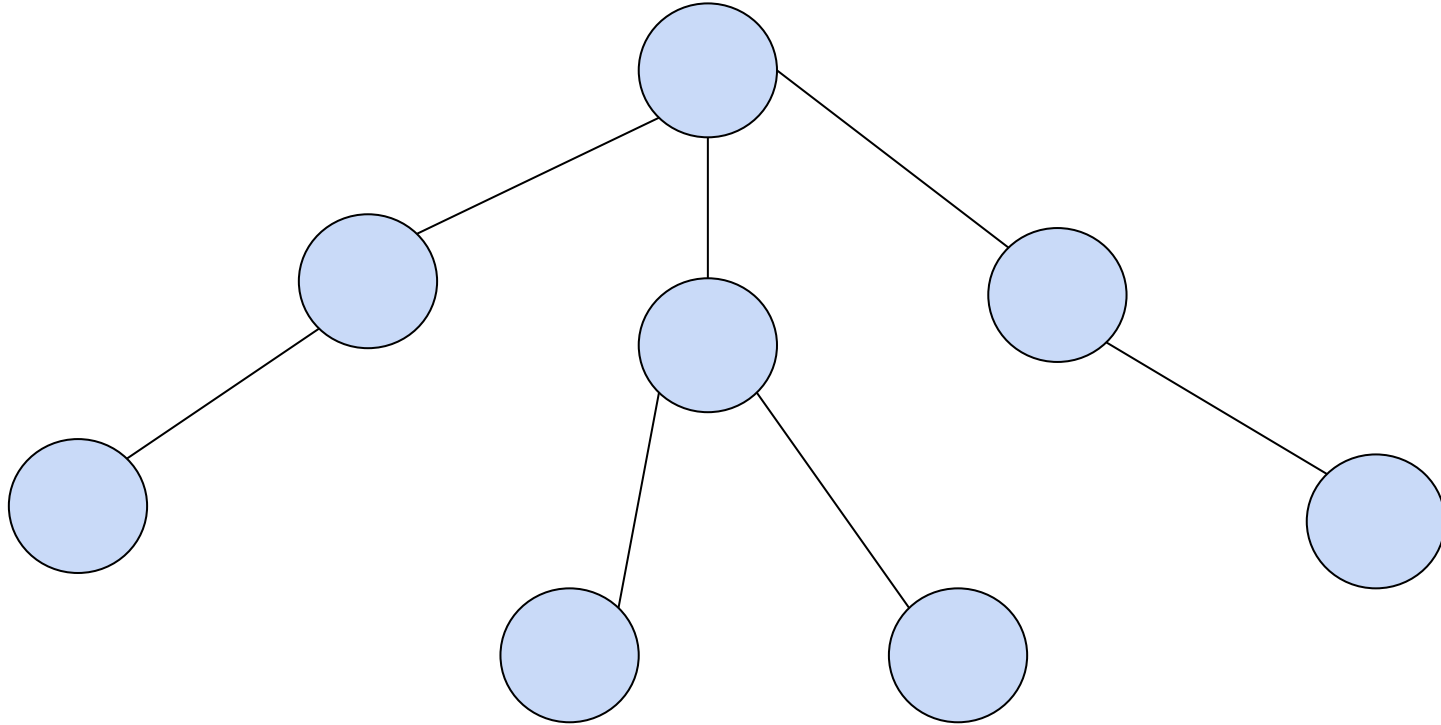
Lecturer

Department of Computer Science and Engineering

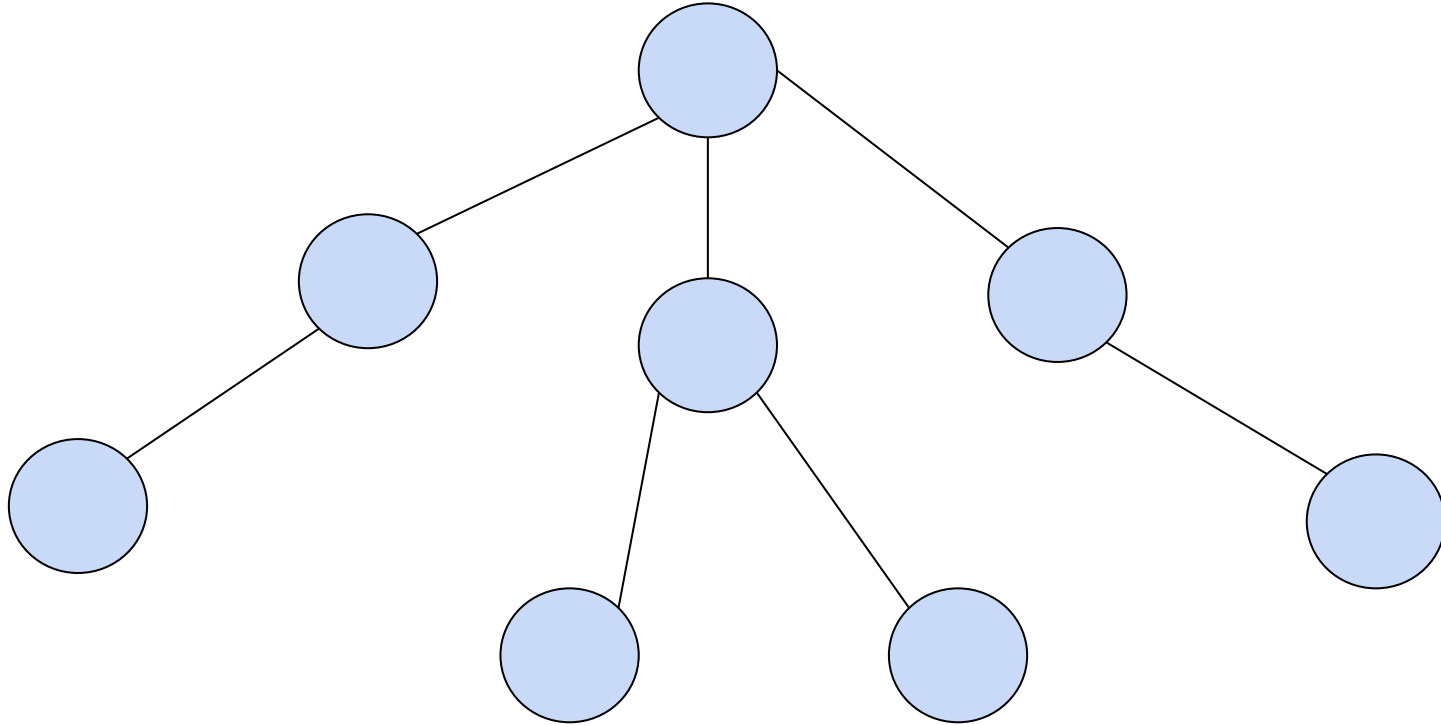
BRAC University

Adapted From: Slides by Dr. Md Abul Kashem Mia, CSE, BUET

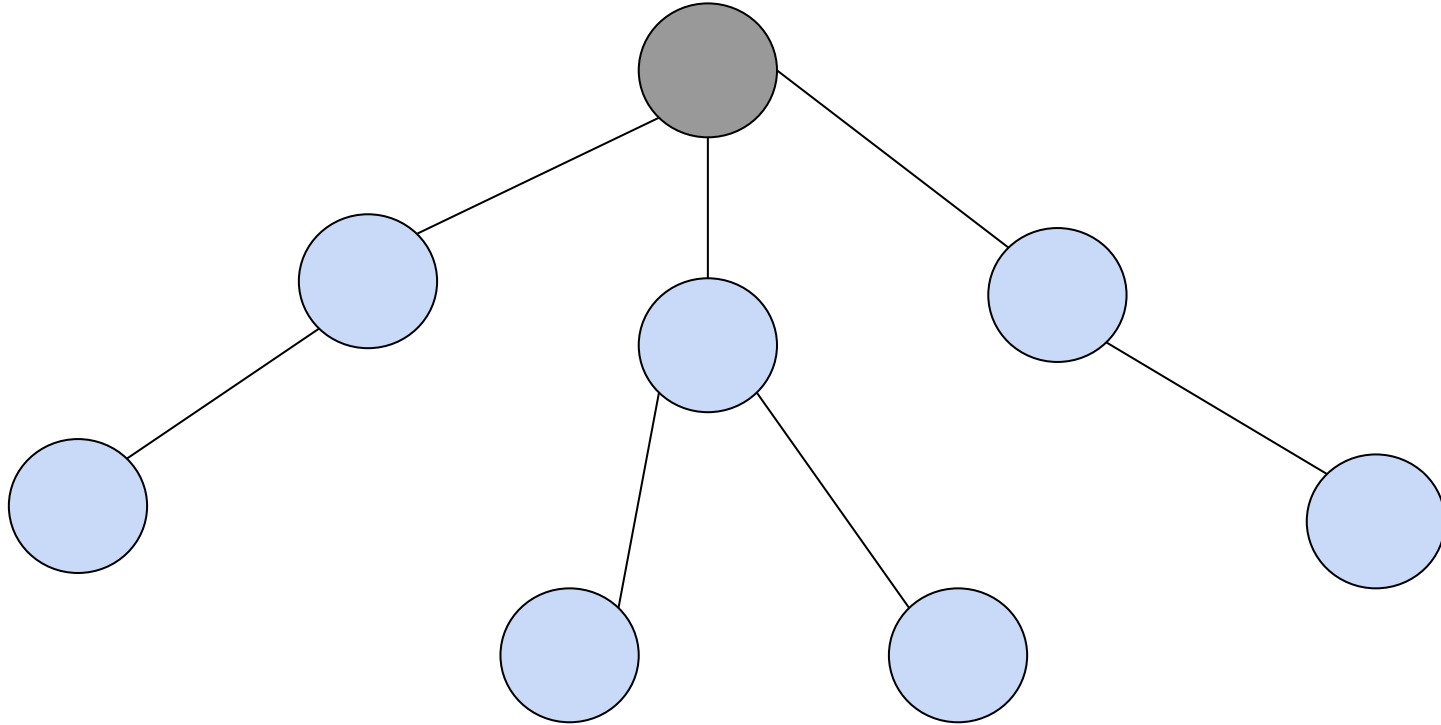
Graph Traversal: We want to visit all nodes



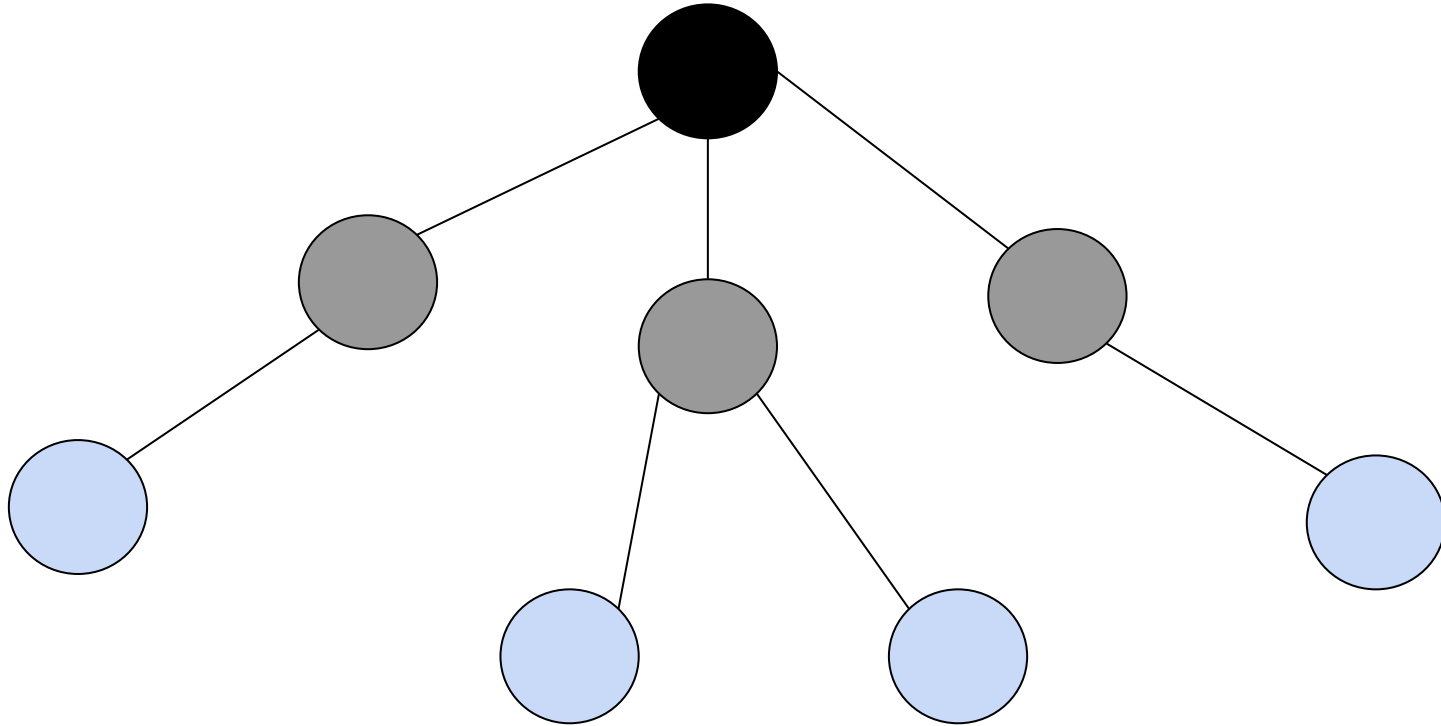
Breadth First Search



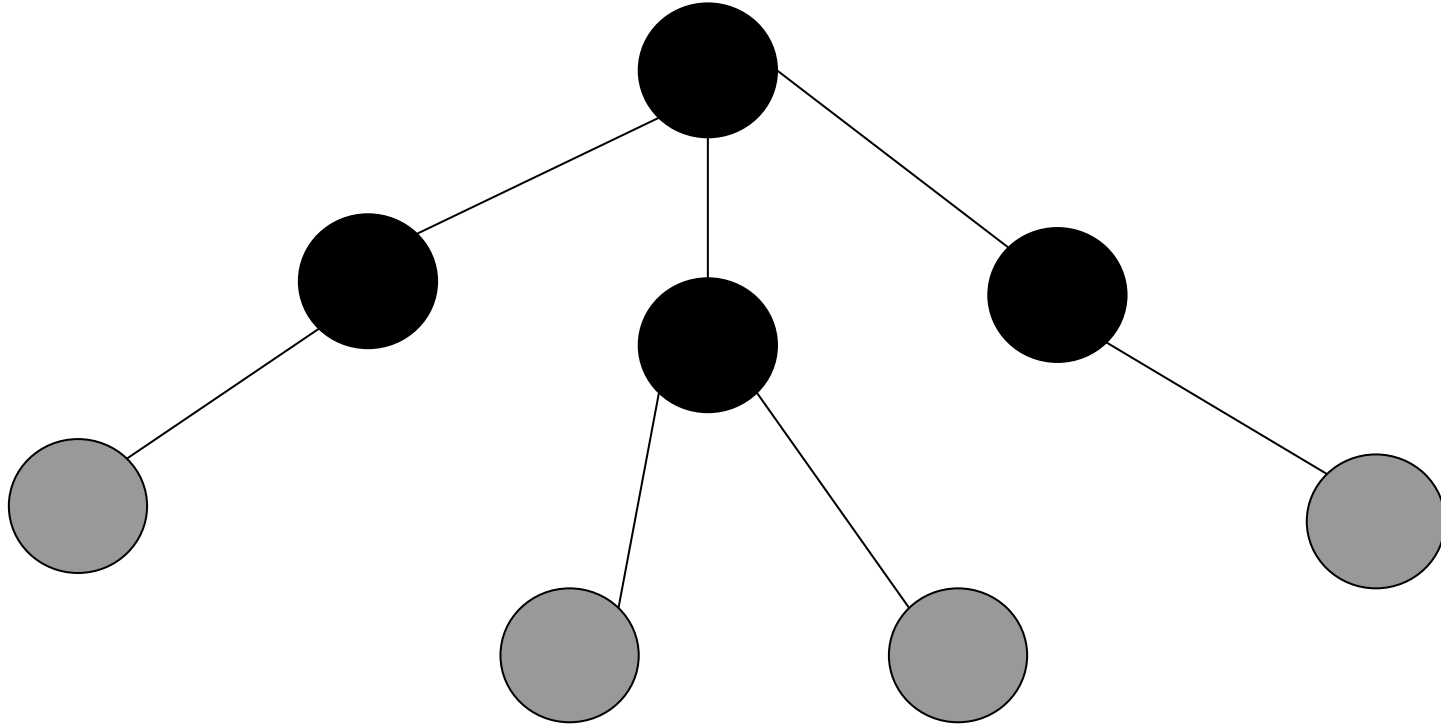
Breadth First Search



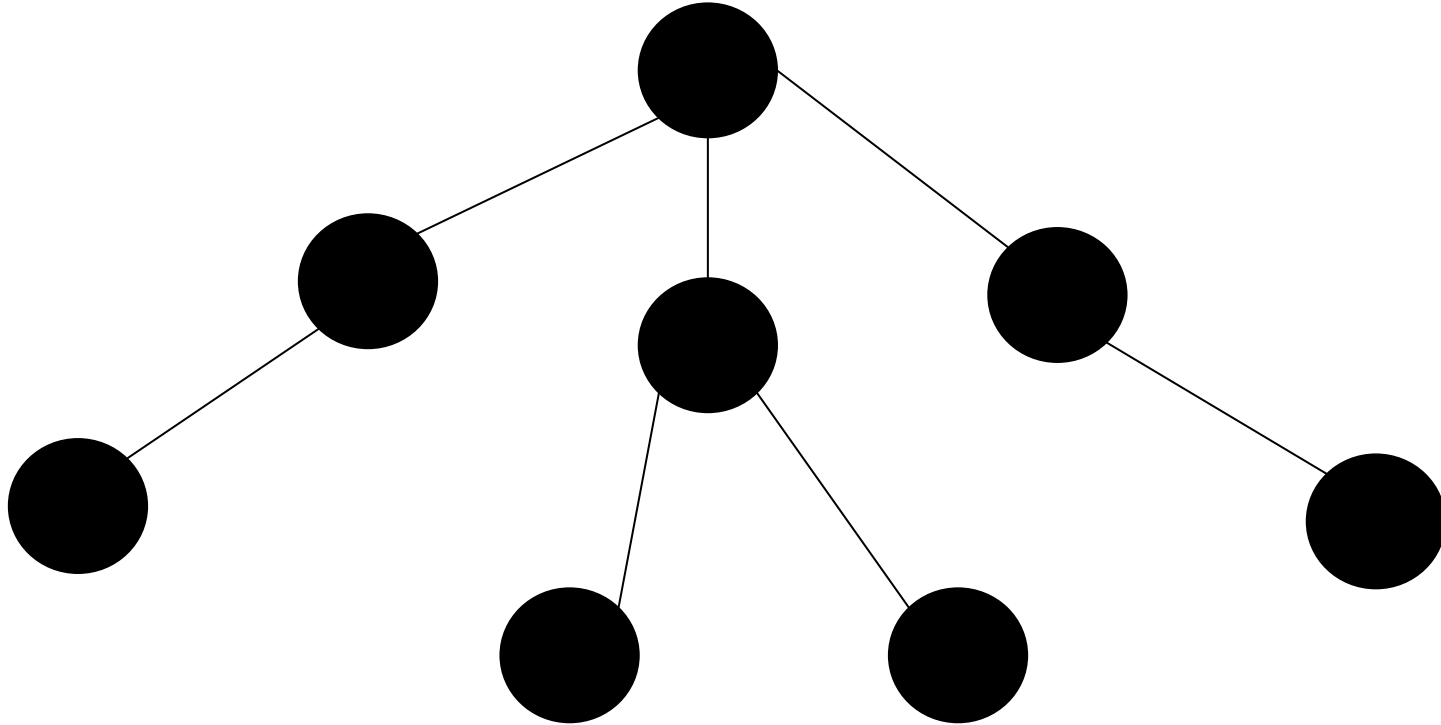
Breadth First Search



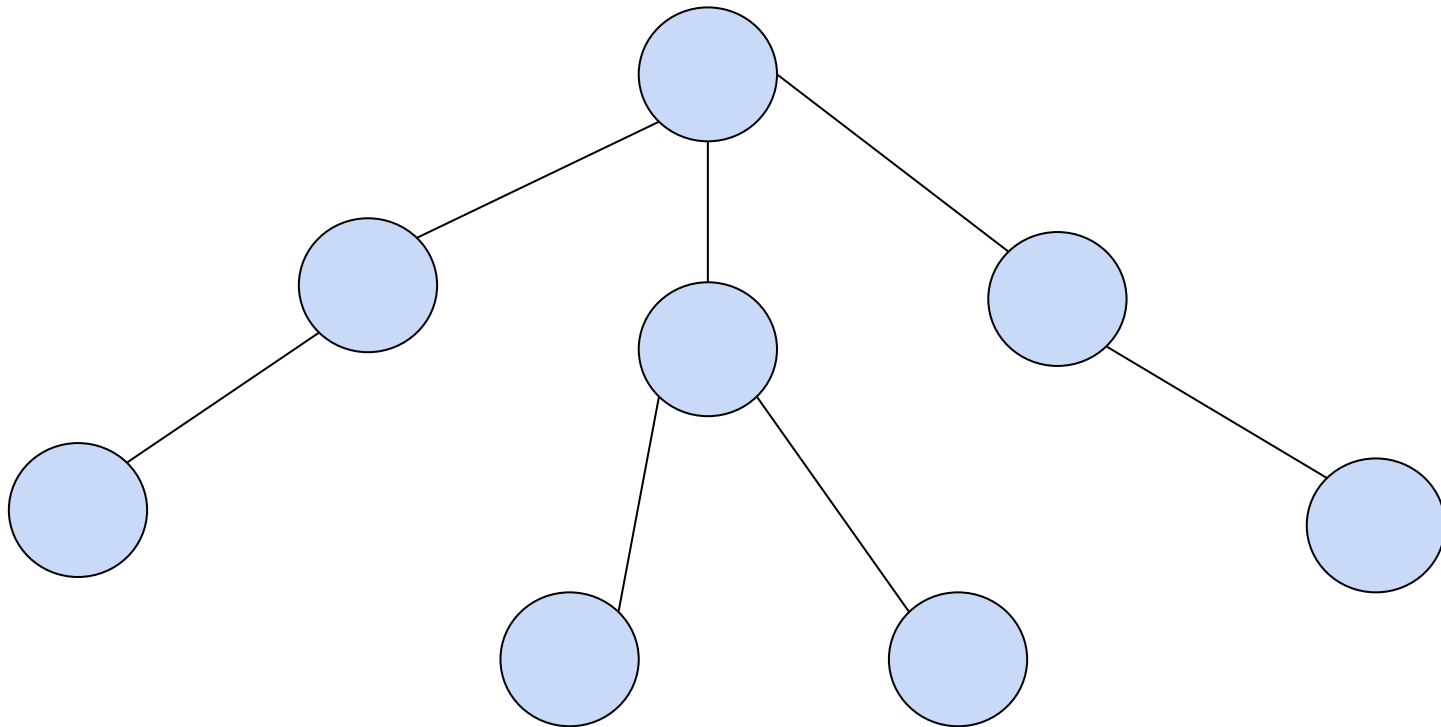
Breadth First Search



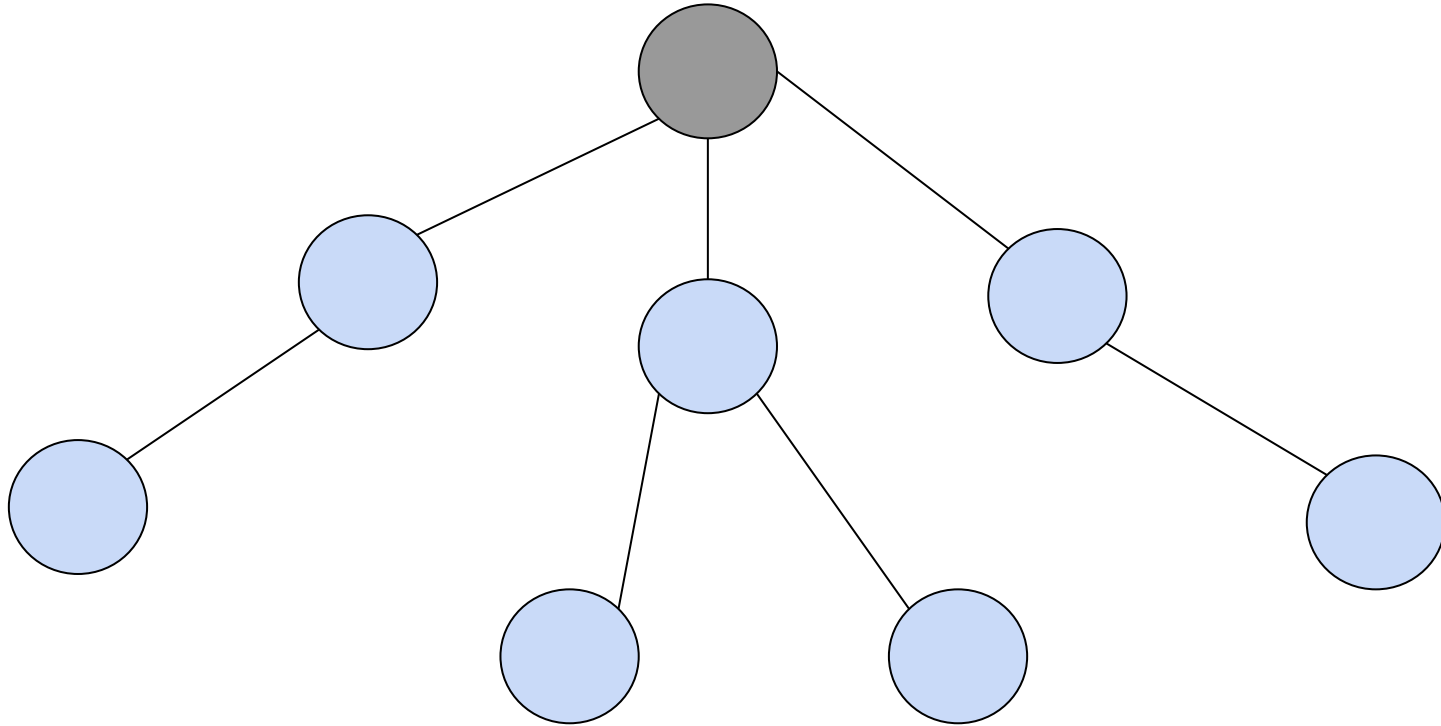
Breadth First Search



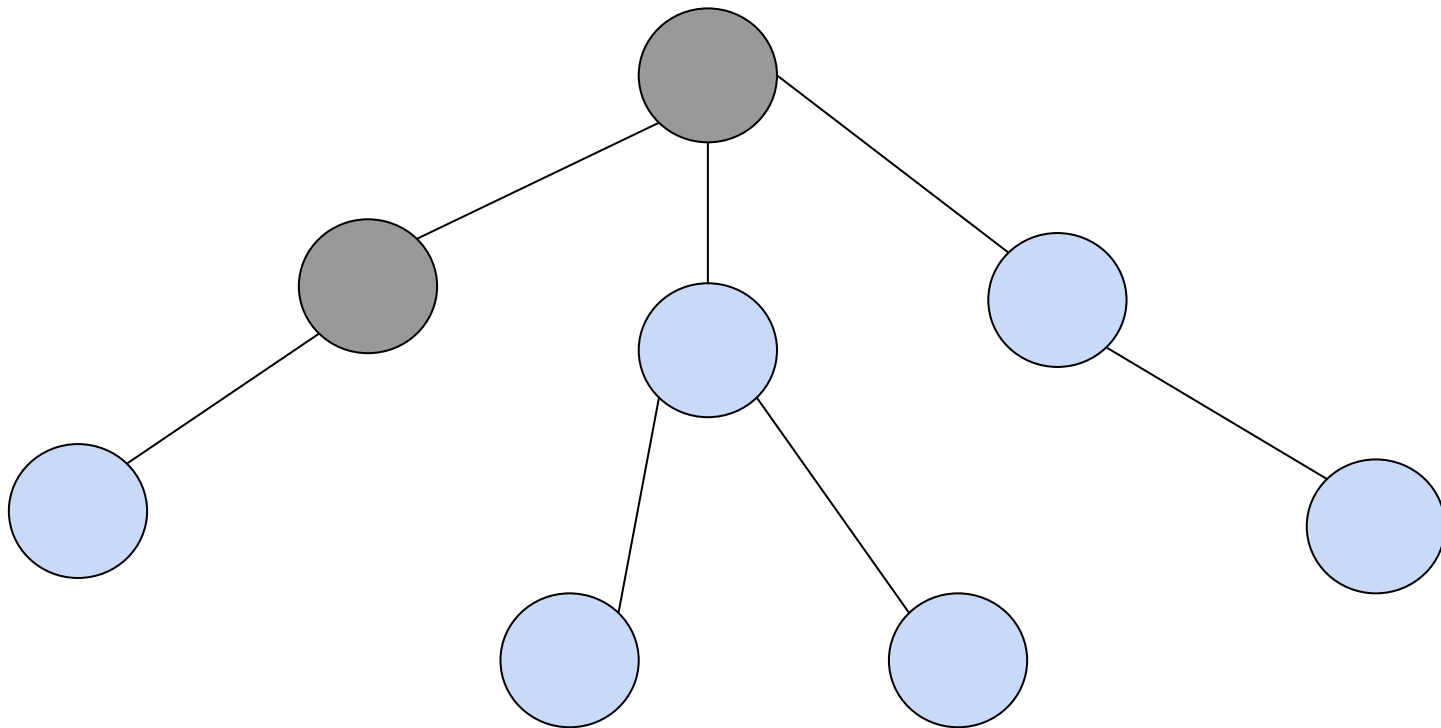
Depth First Search



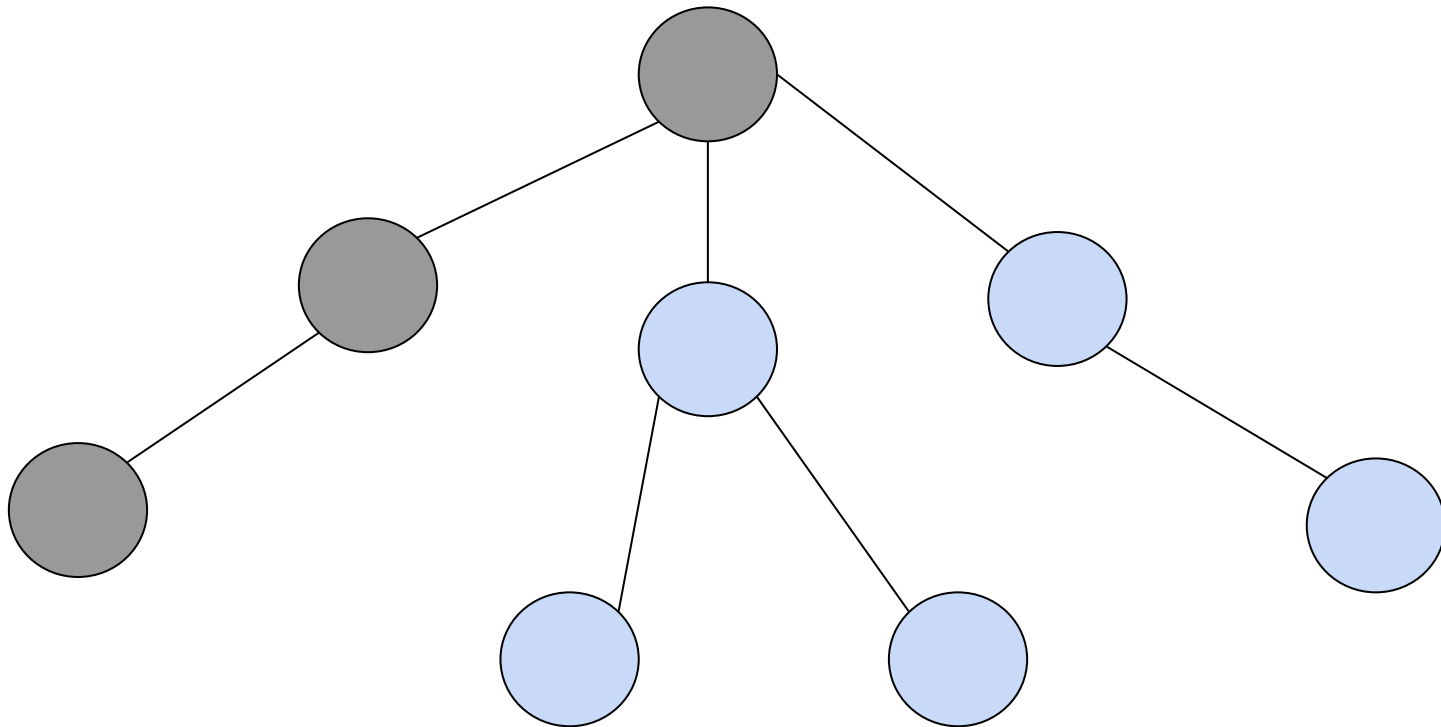
Depth First Search



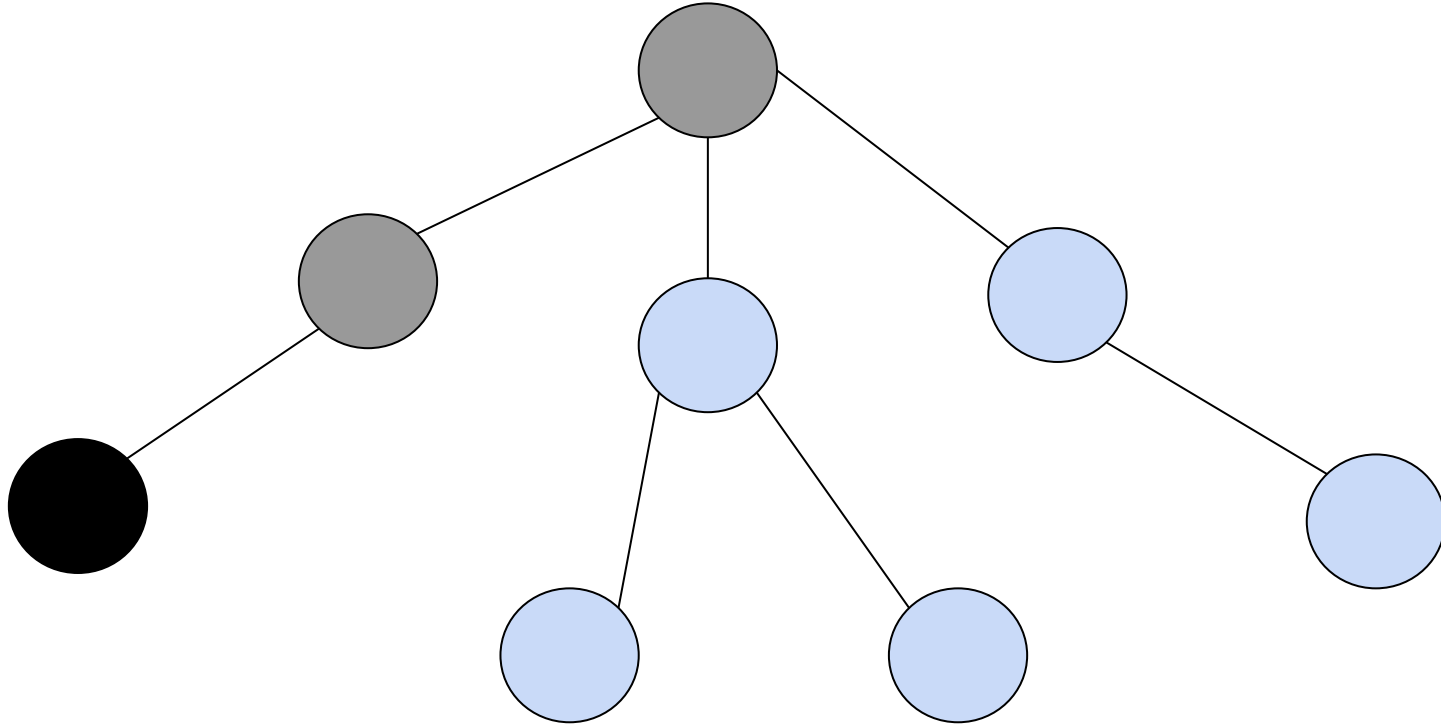
Depth First Search



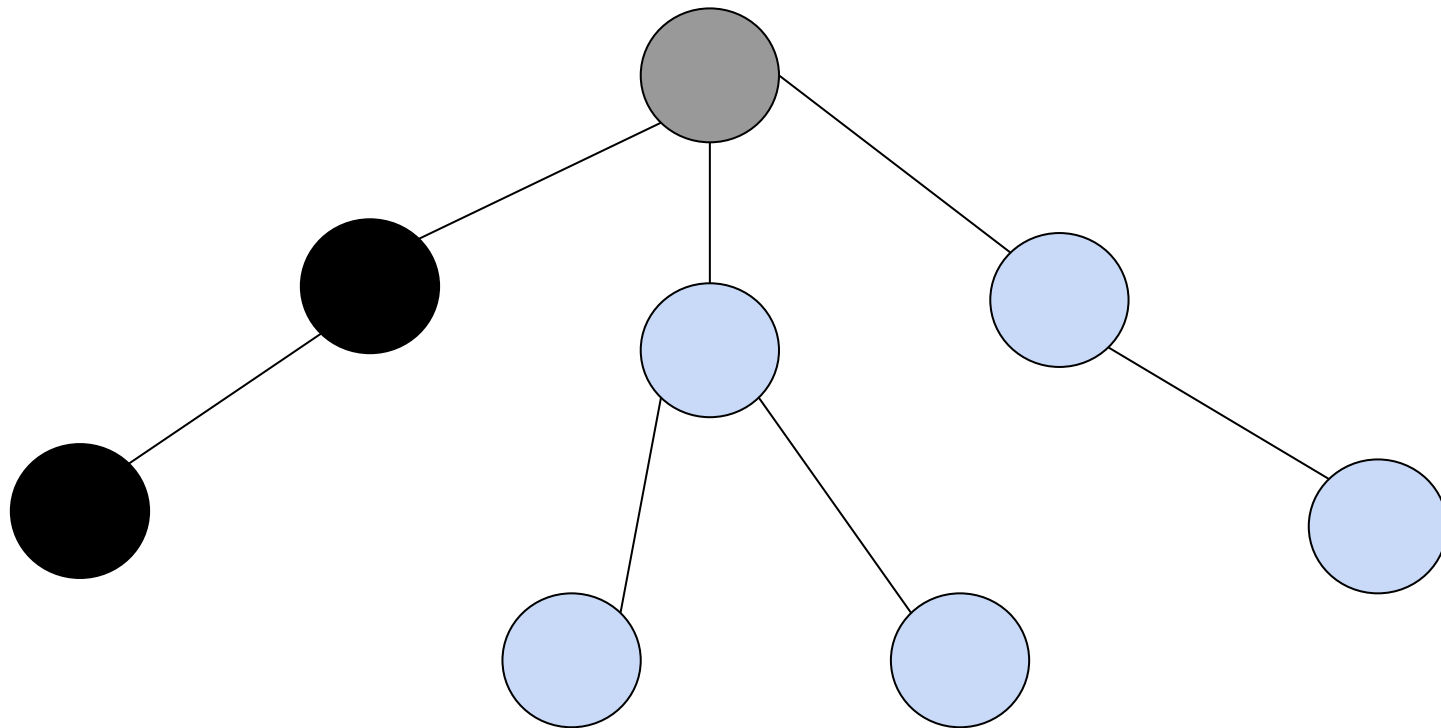
Depth First Search



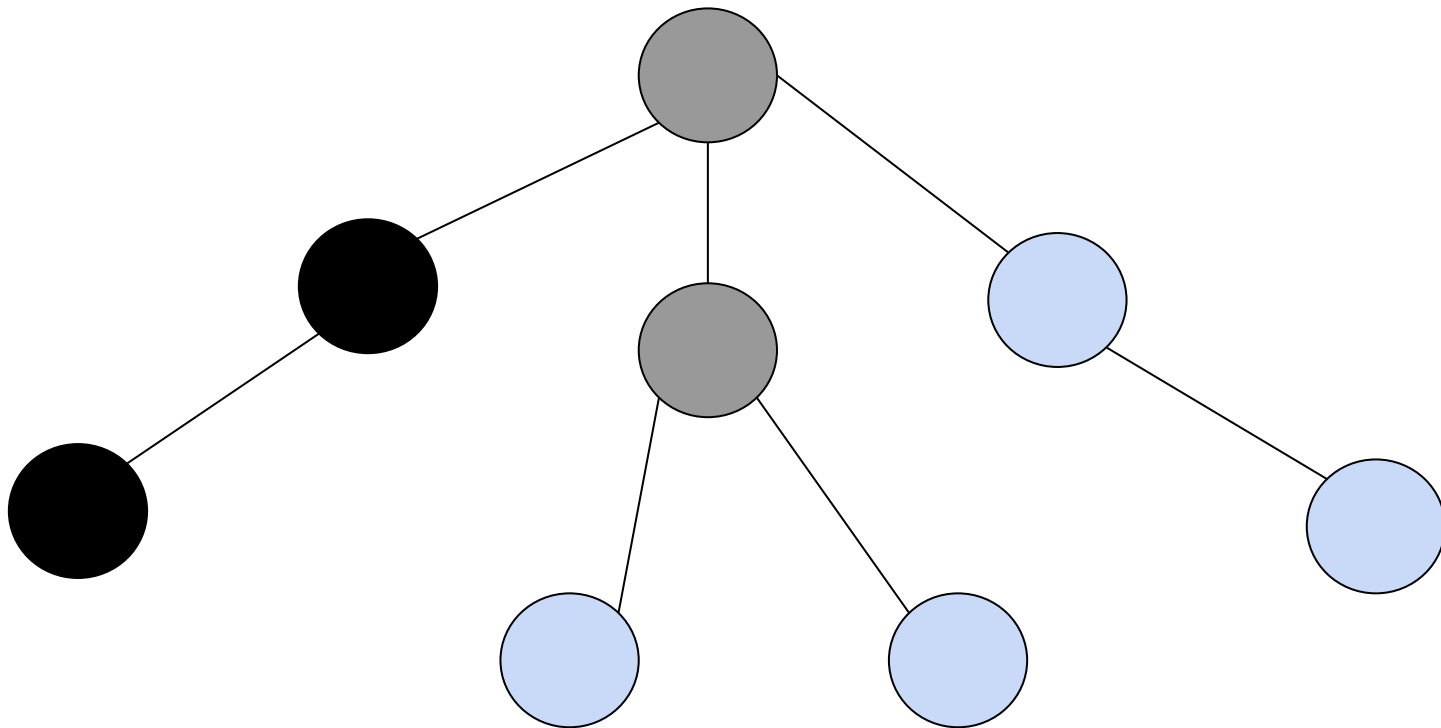
Depth First Search



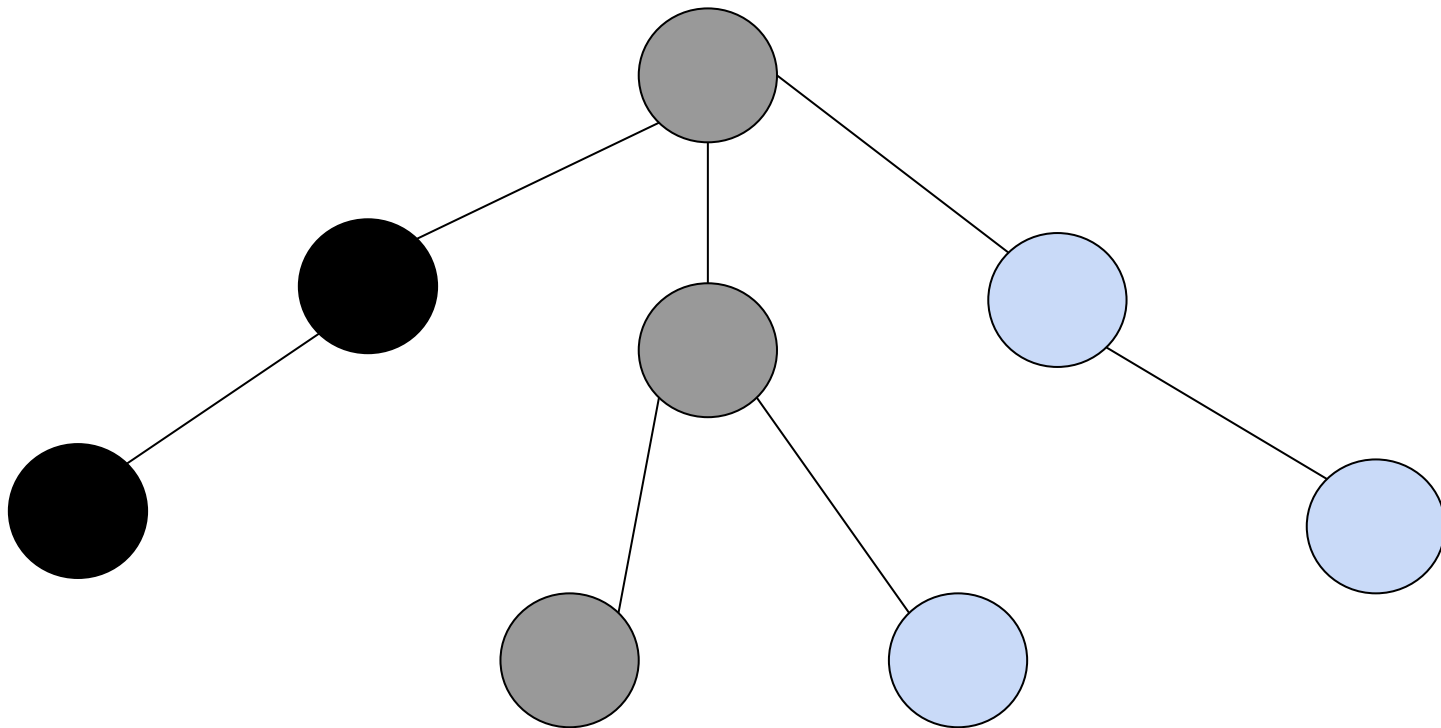
Depth First Search



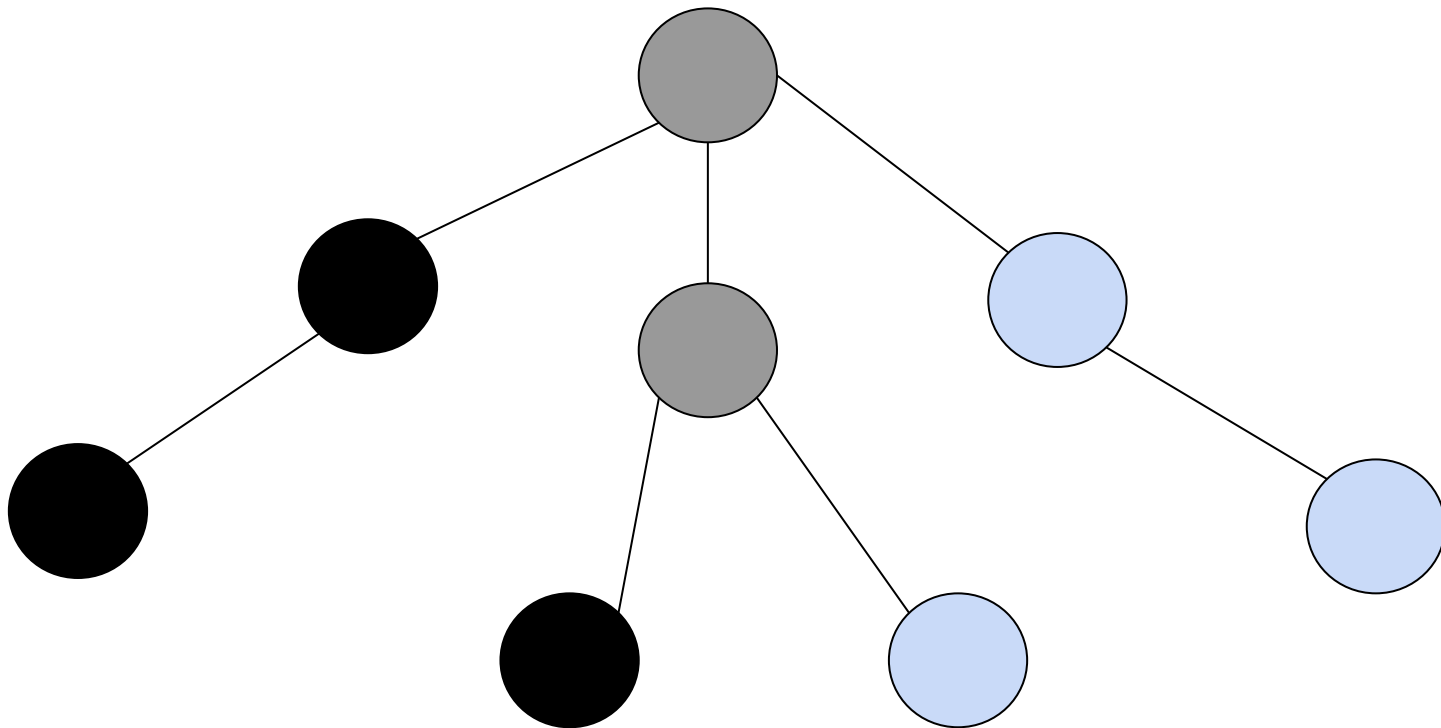
Depth First Search



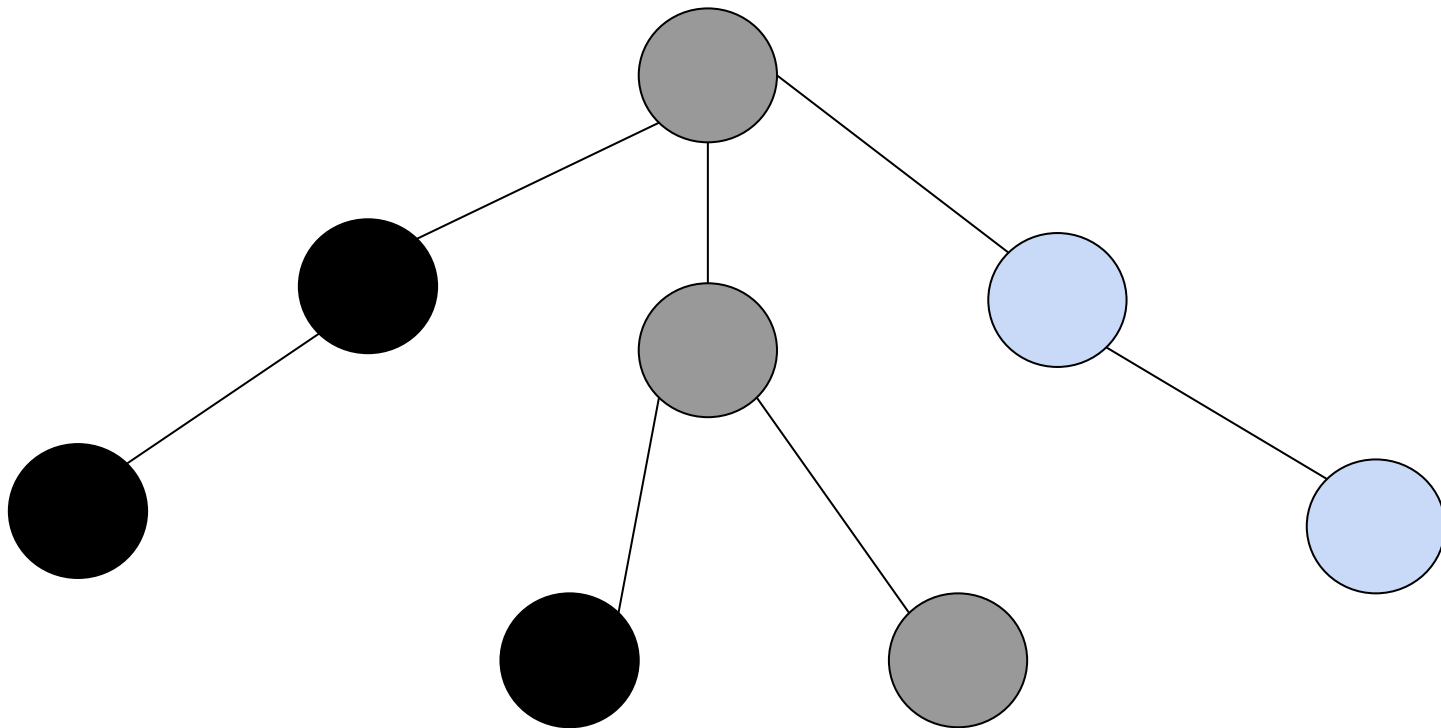
Depth First Search



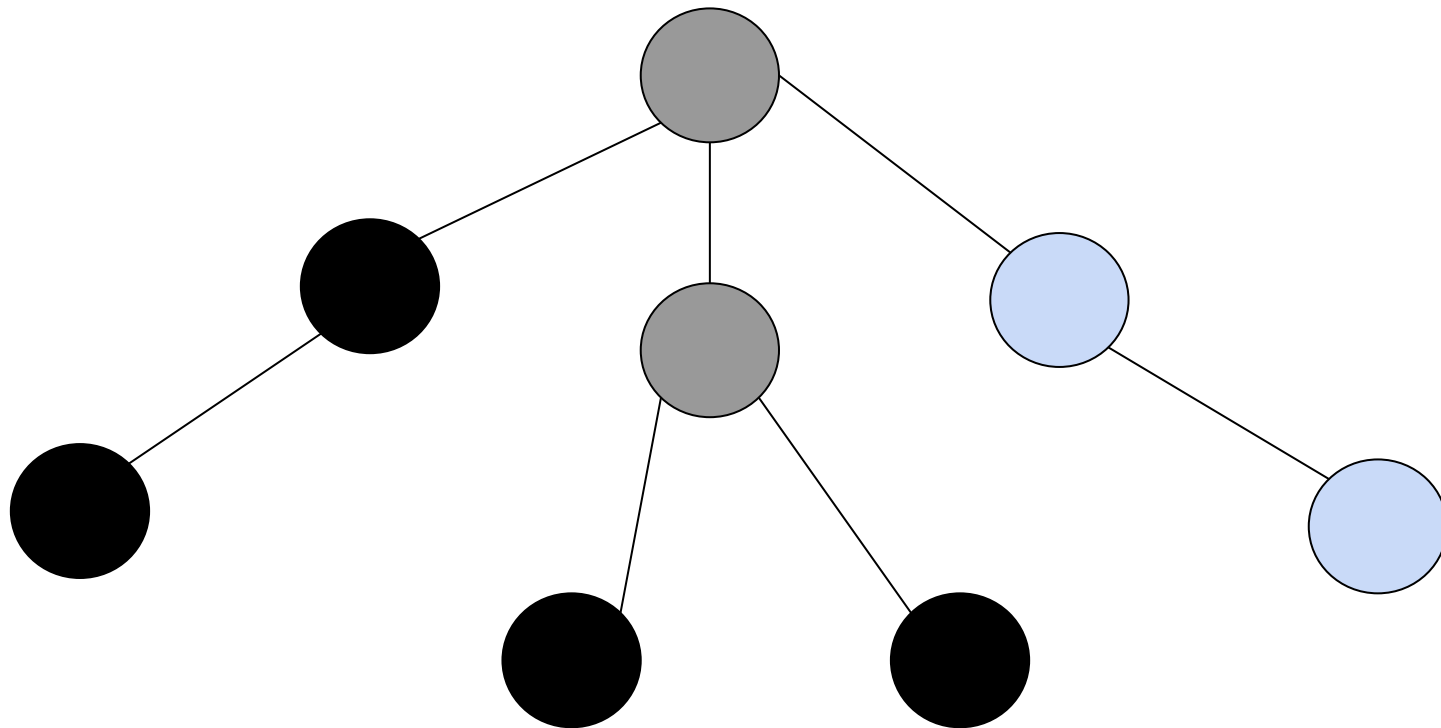
Depth First Search



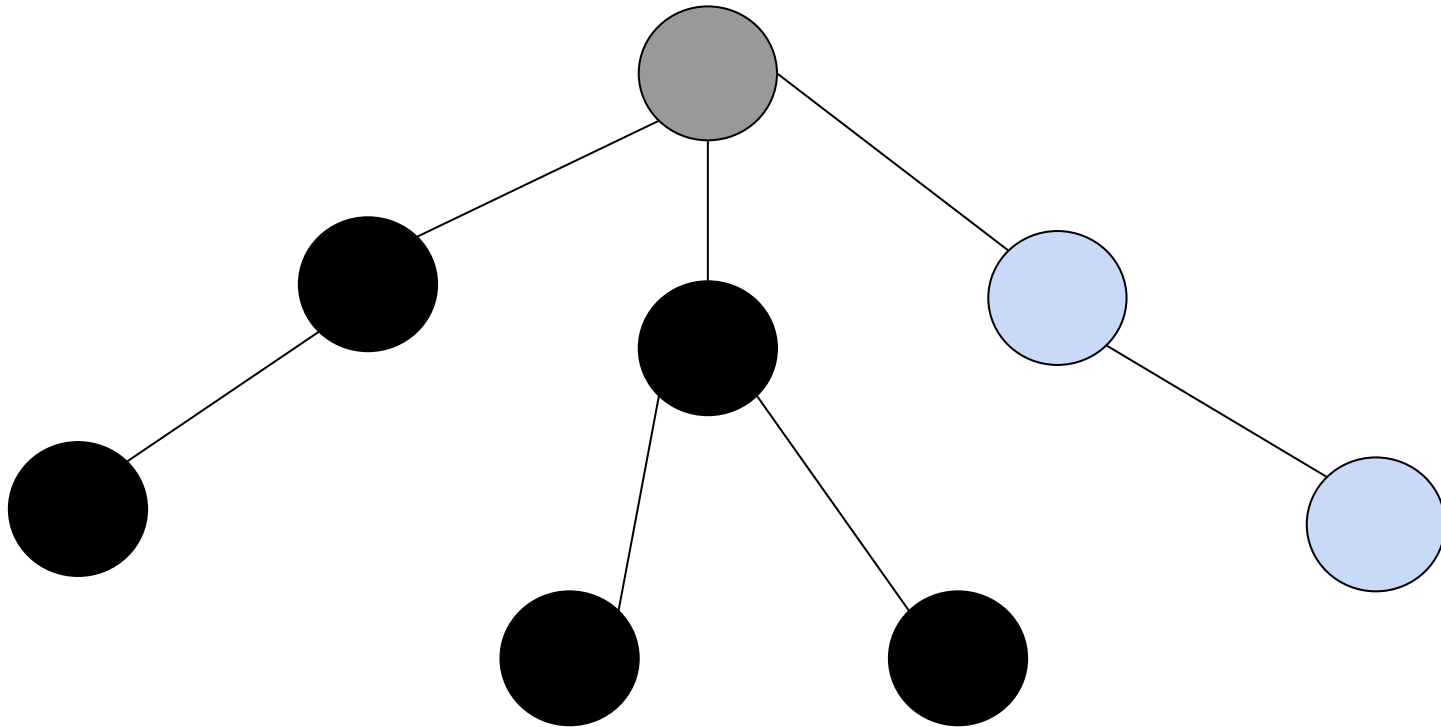
Depth First Search



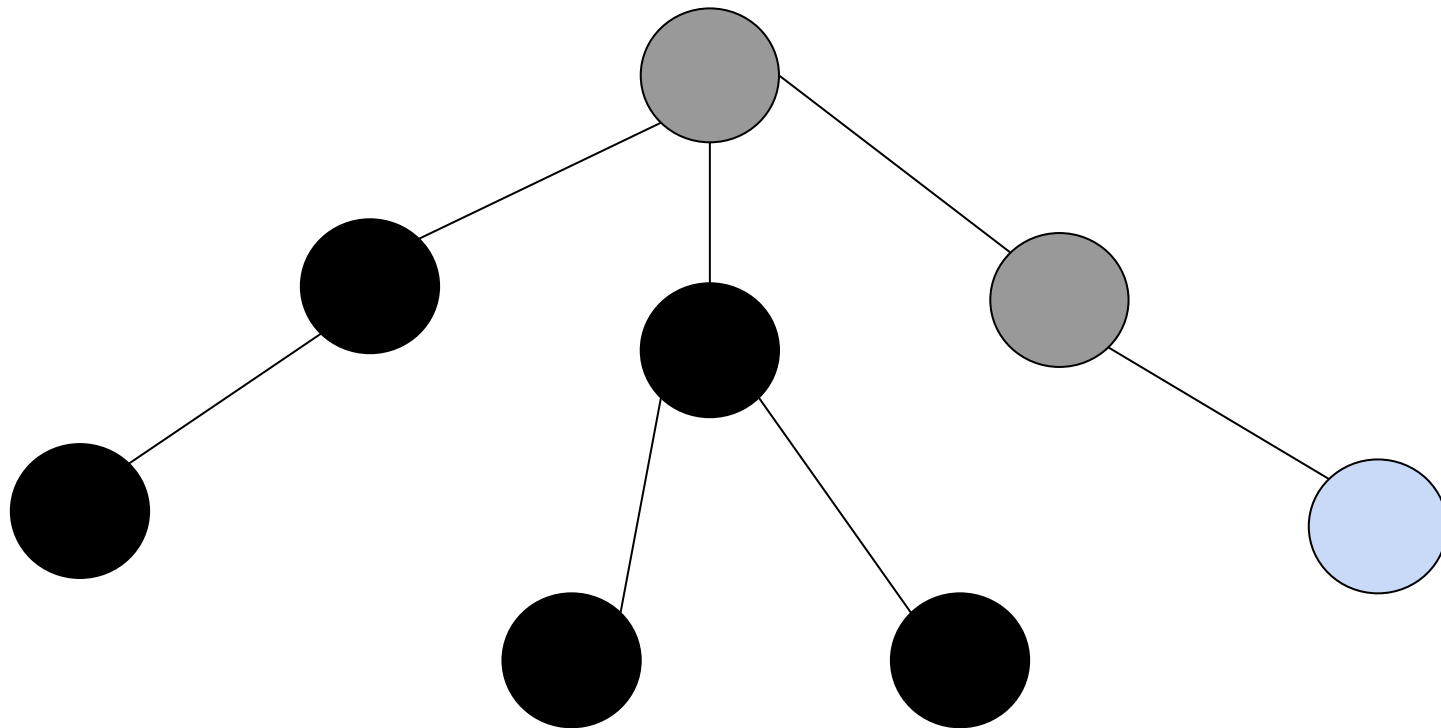
Depth First Search



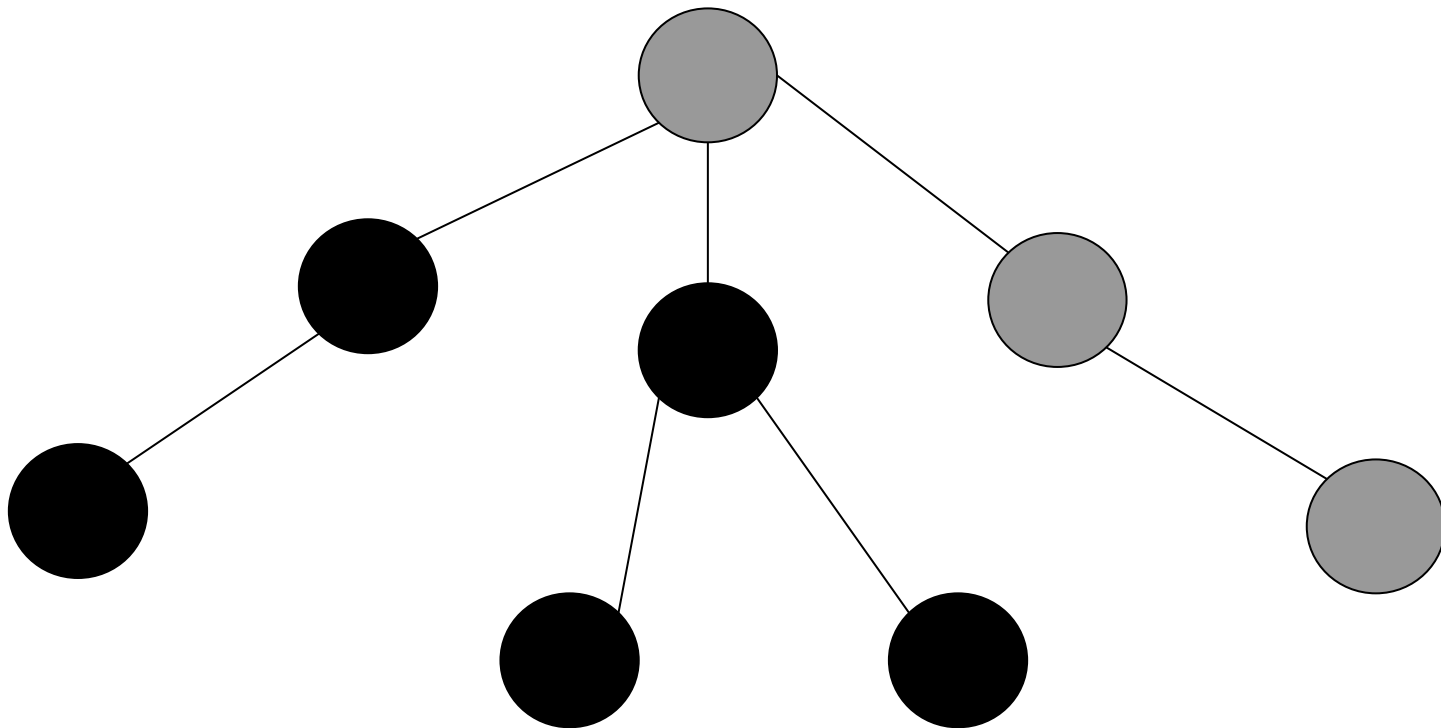
Depth First Search



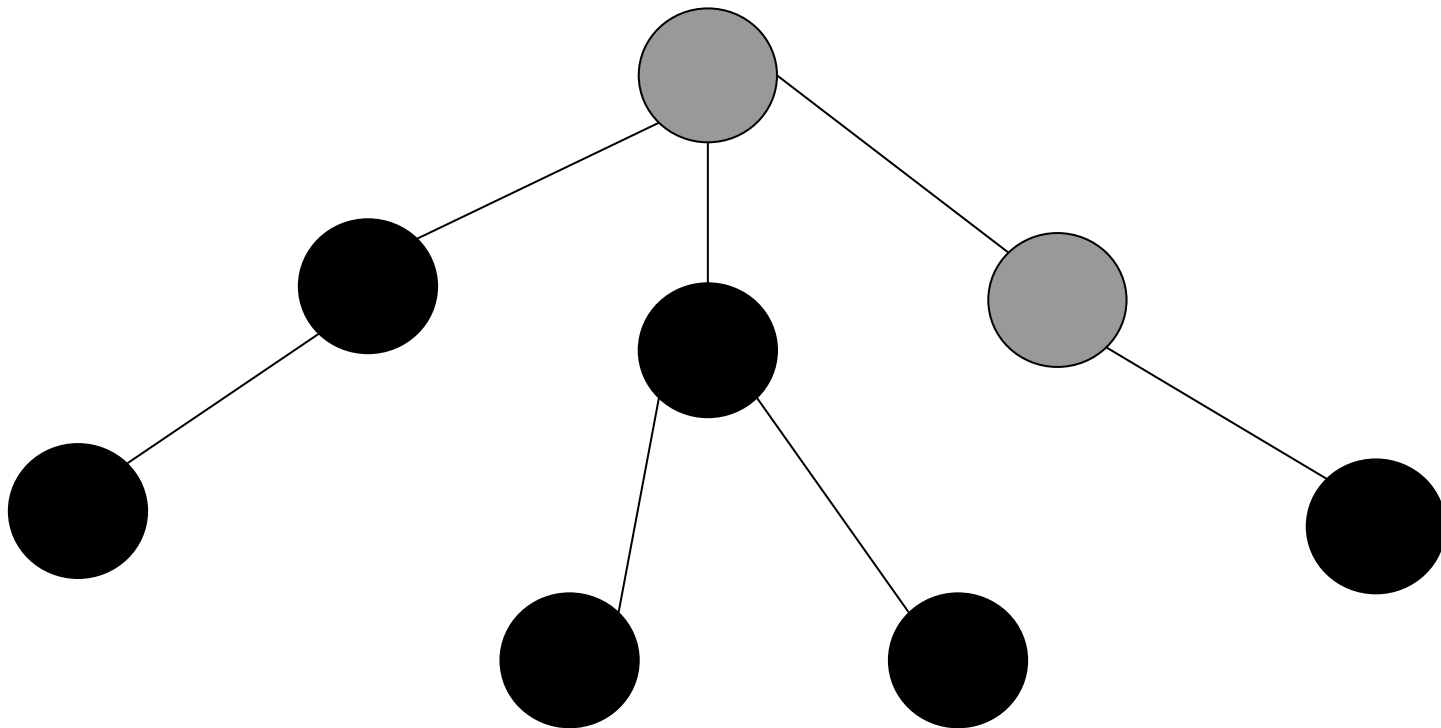
Depth First Search



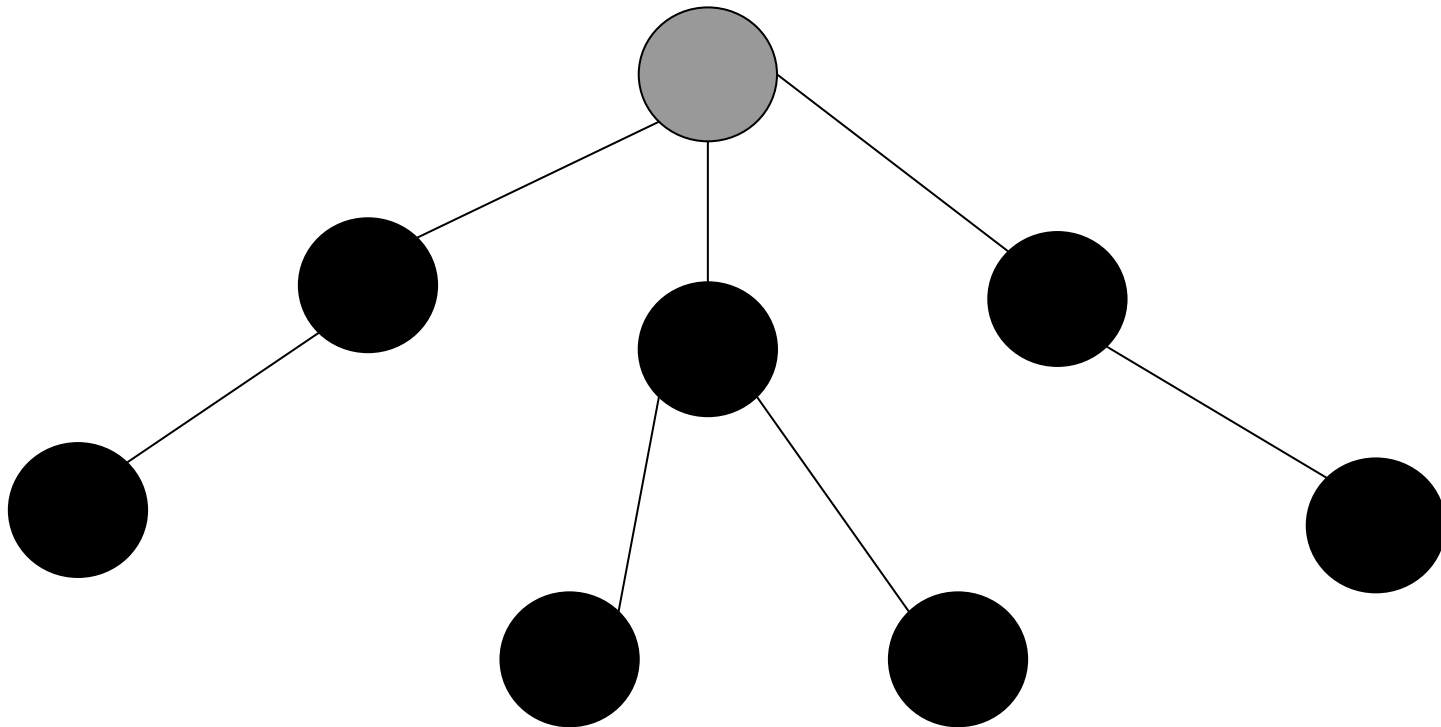
Depth First Search



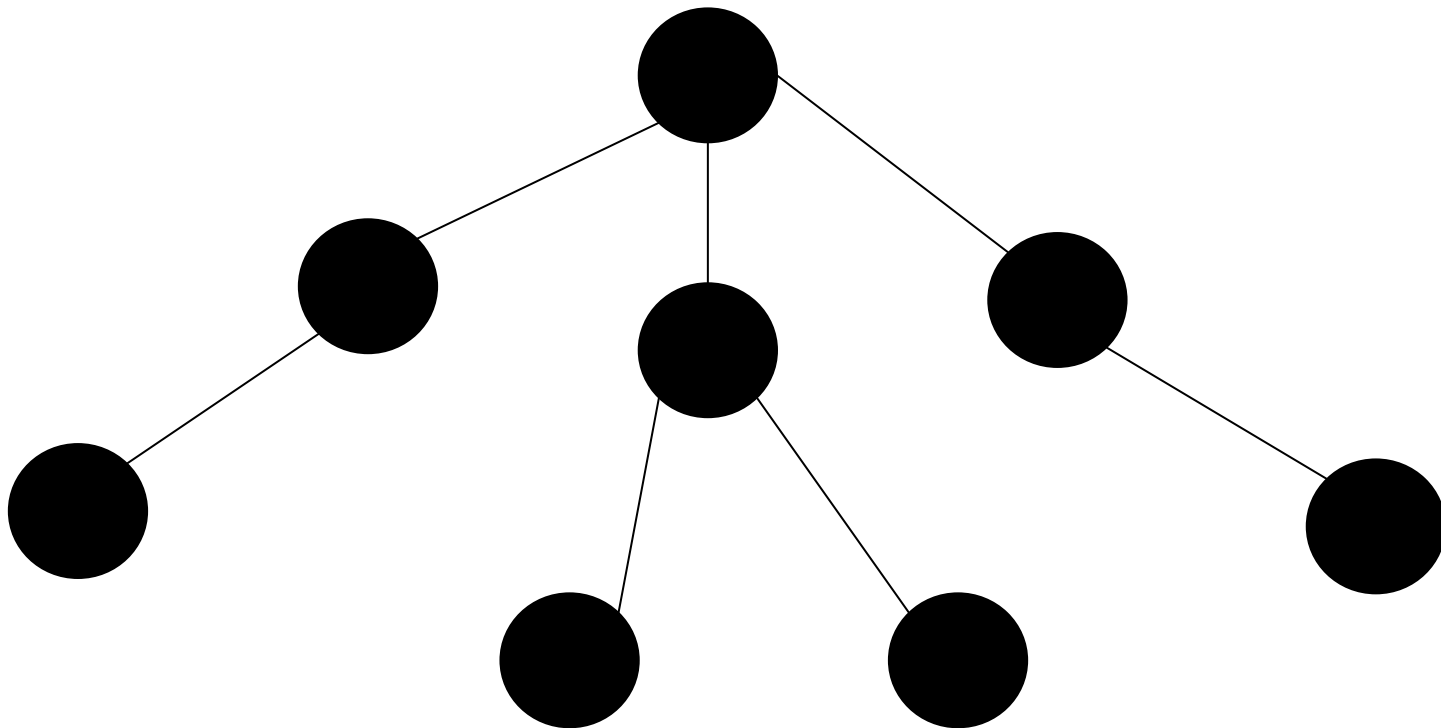
Depth First Search



Depth First Search

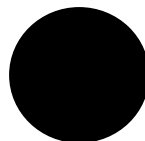
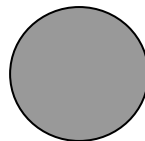
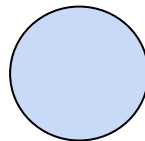


Depth First Search



Breadth-First Search: Colors

- White vertices - Unvisited
- Grey vertices - Visited but not fully explored
- Black vertices - Visited and fully explored



Properties of Vertex

```
class Vertex:
```

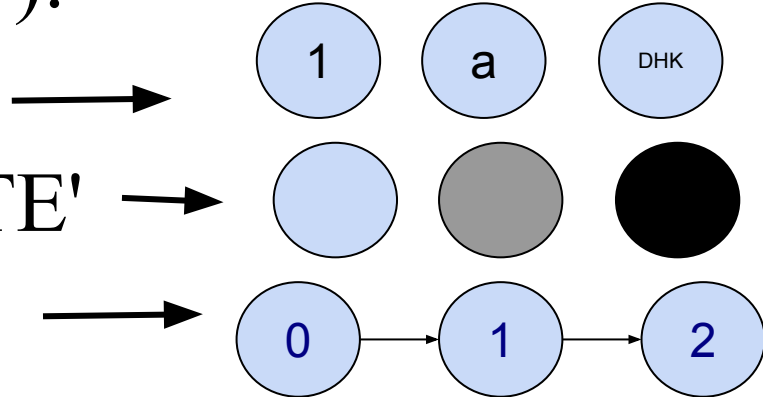
```
    def __init__(self, key):
```

```
        self.key = key
```

```
        self.color = 'WHITE'
```

```
        self.d = float('inf')
```

```
        self.p = None
```



We store the parent node here. For example, the parent of 2 is 1. So for the node of 2, we will store the node of 1 here

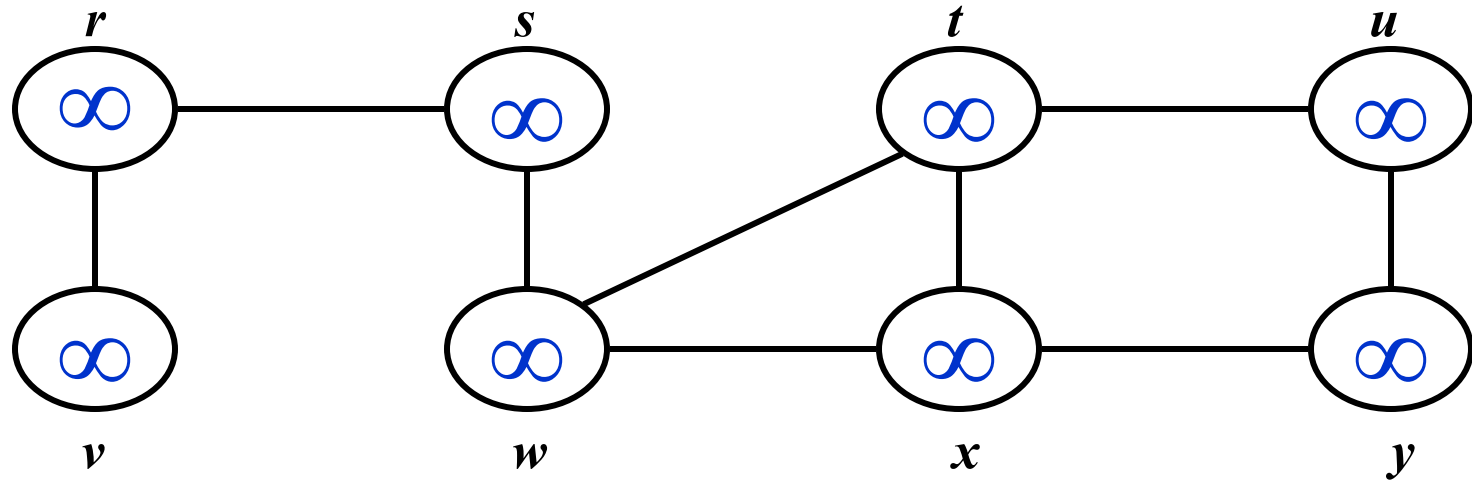
Breadth-First Search

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};      // Q is a queue (duh); initialize to s  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

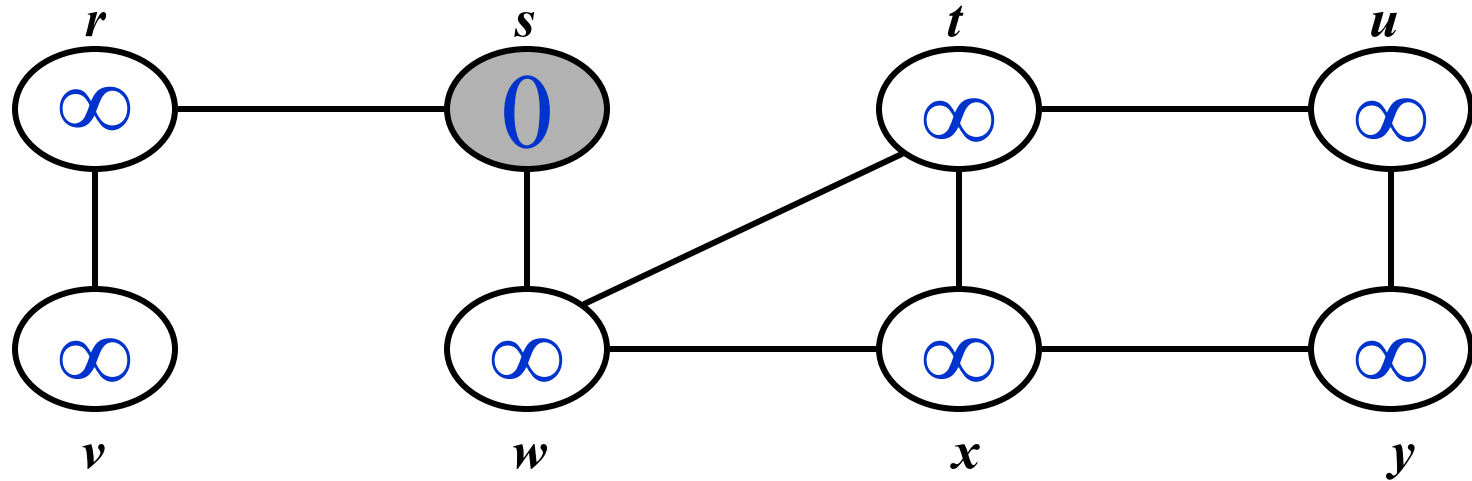
What does $v \rightarrow d$ represent?

What does $v \rightarrow p$ represent?

Breadth-First Search: Example

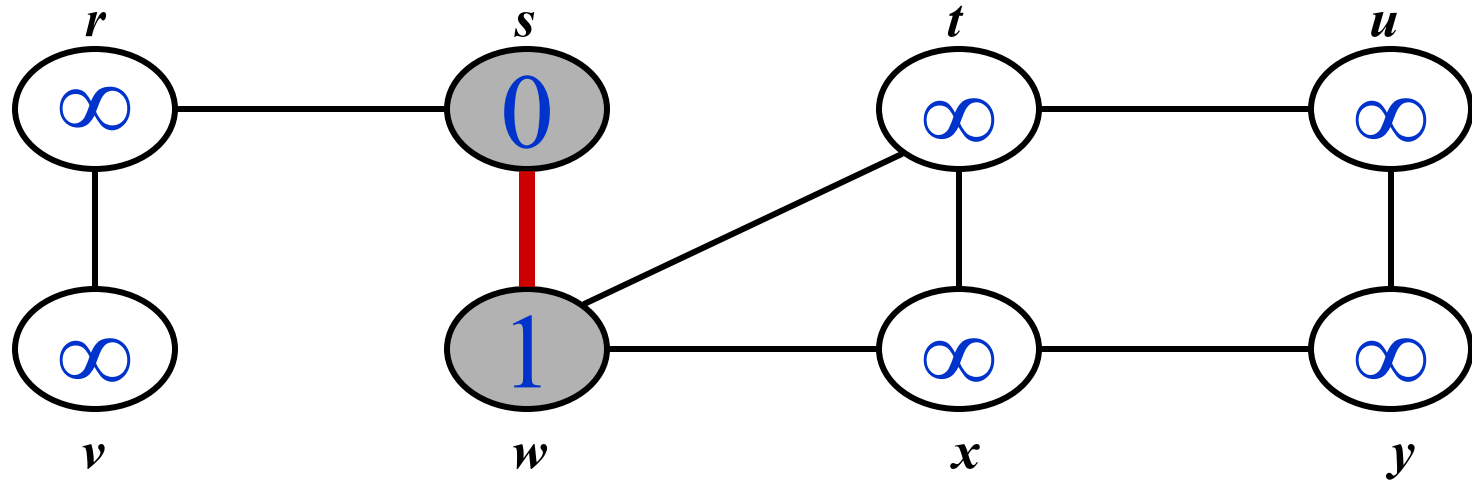


Breadth-First Search: Example



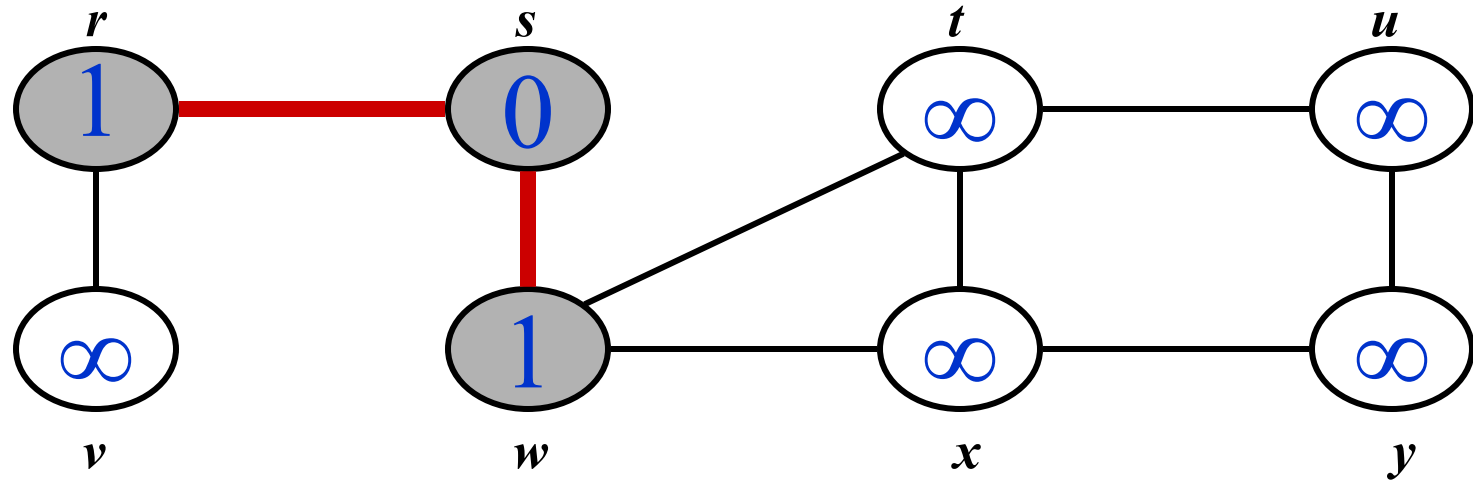
Q : s

Breadth-First Search: Example



Q : w

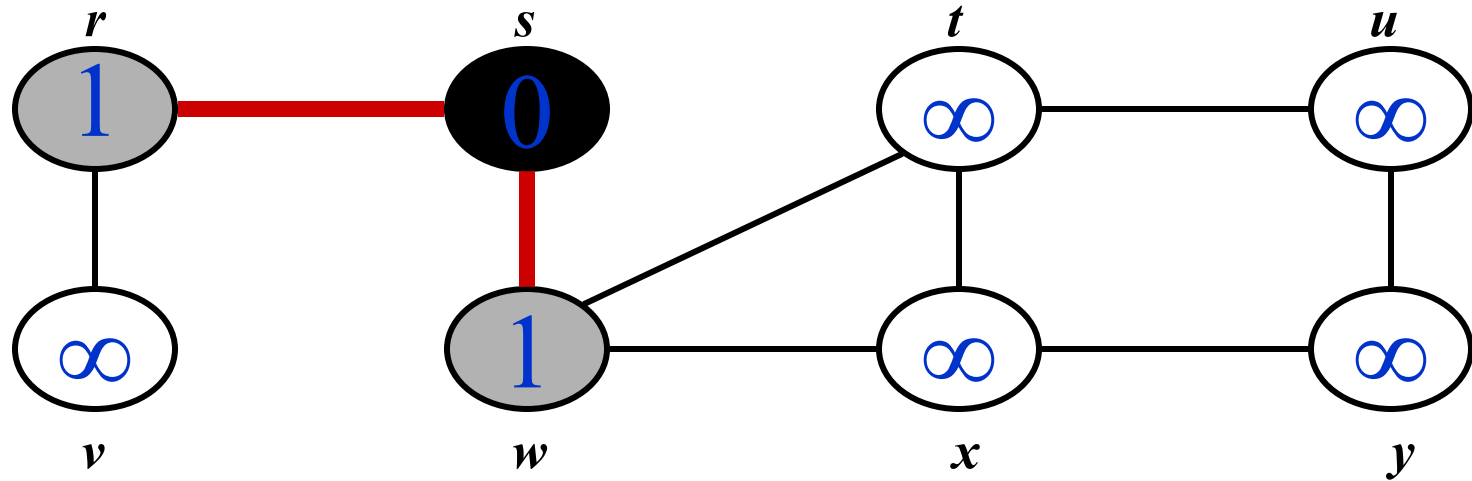
Breadth-First Search: Example



Q :

w	r
-----	-----

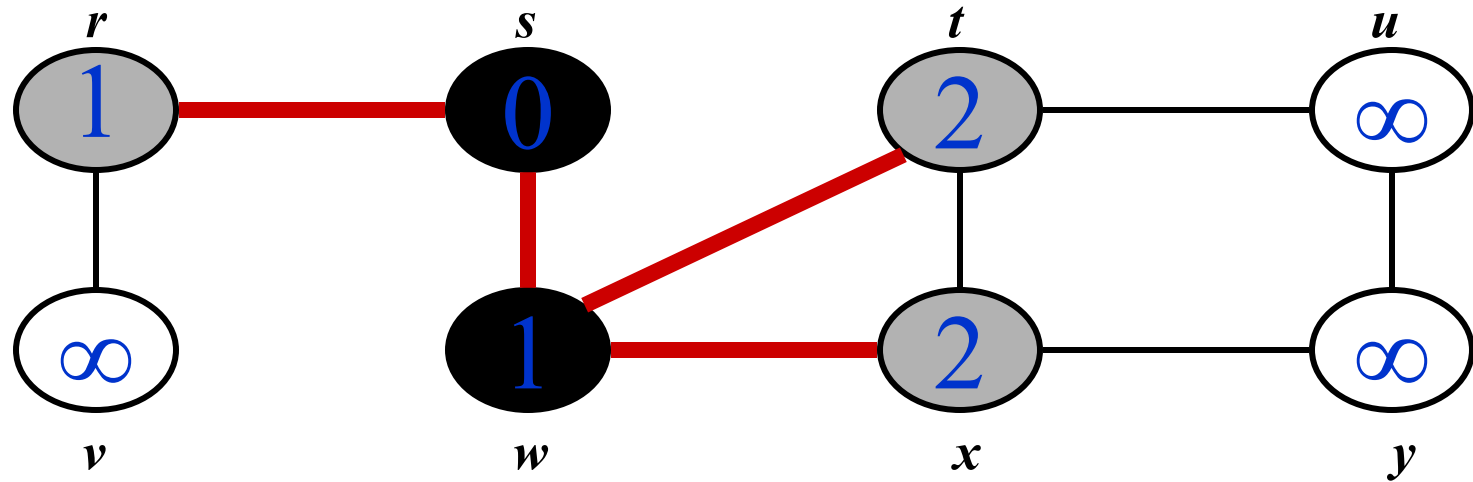
Breadth-First Search: Example



Q :

w	r
-----	-----

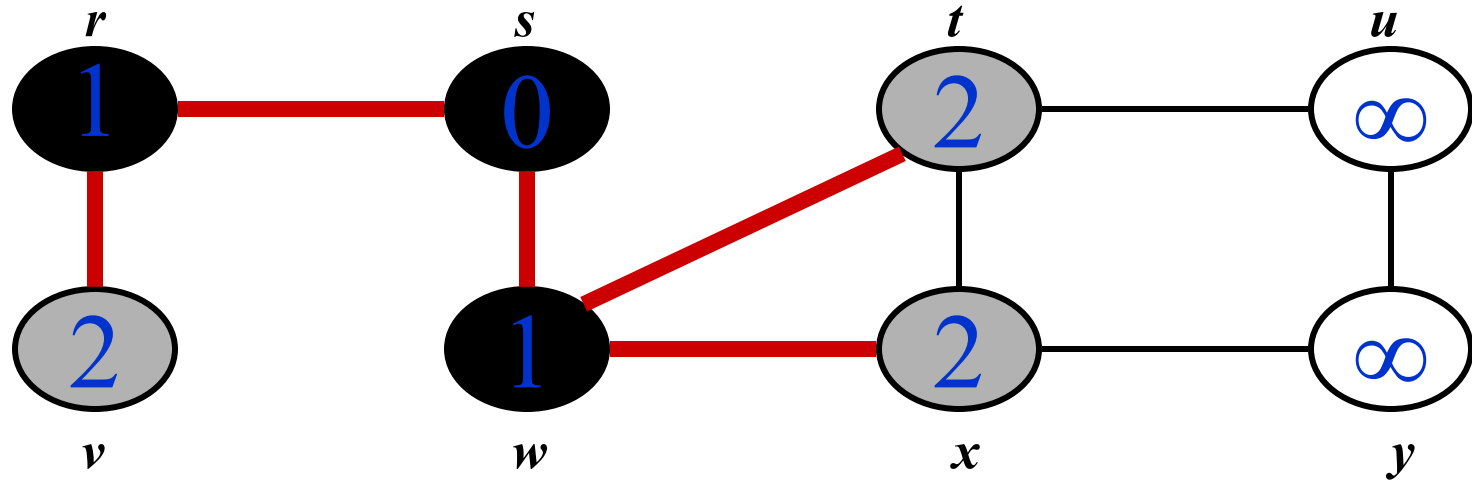
Breadth-First Search: Example



Q :

r	t	x
-----	-----	-----

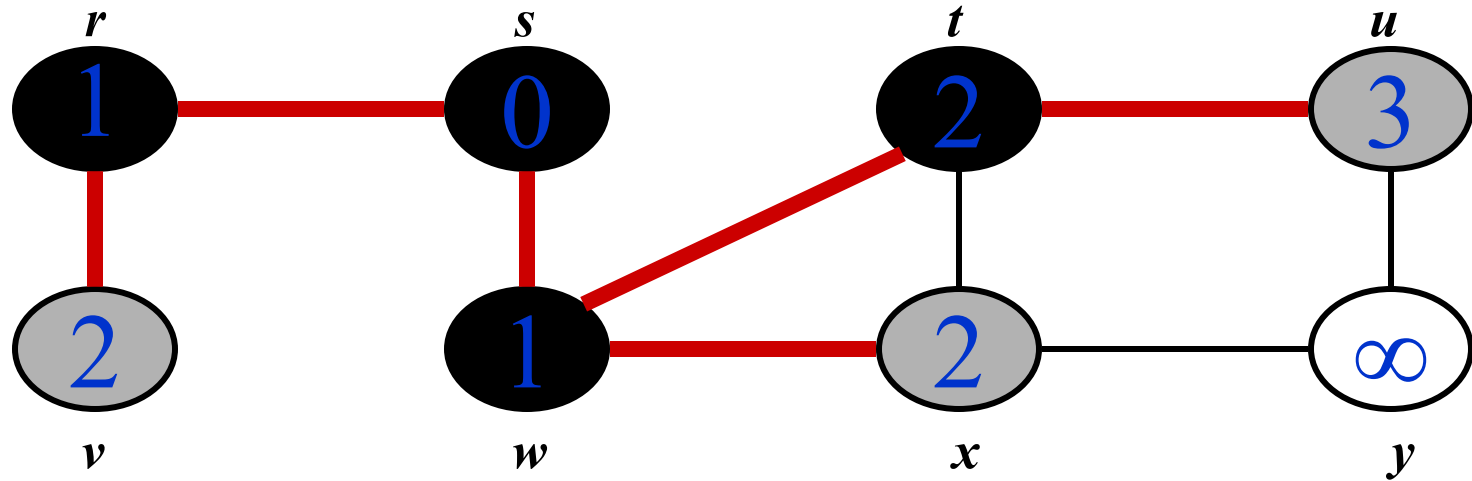
Breadth-First Search: Example



Q :

t	x	v
-----	-----	-----

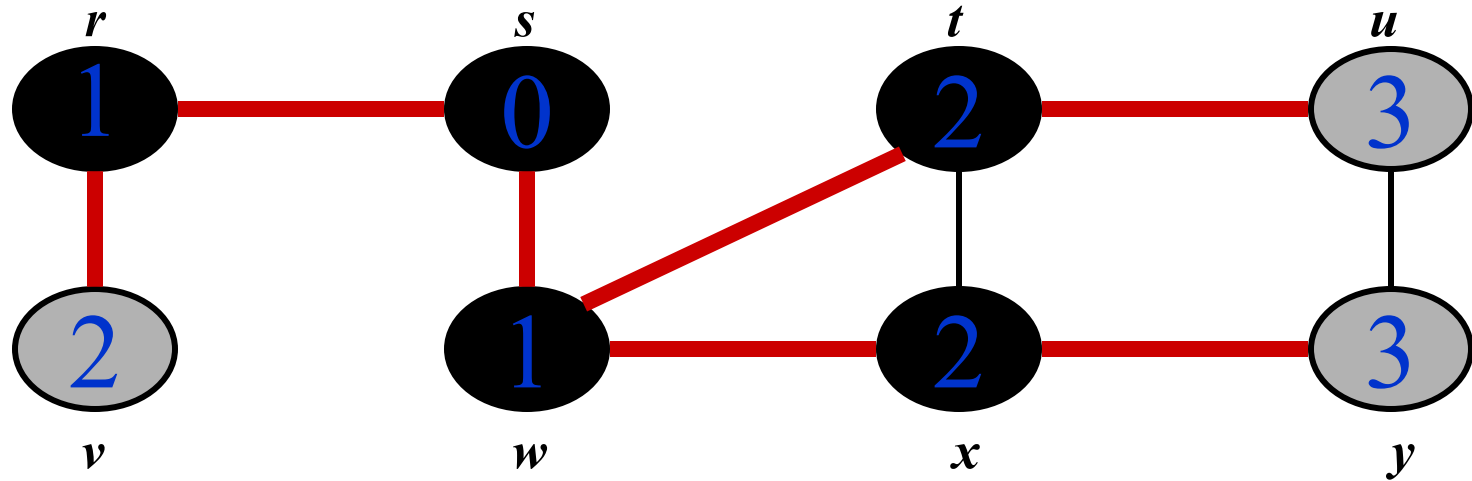
Breadth-First Search: Example



Q :

x	v	u
-----	-----	-----

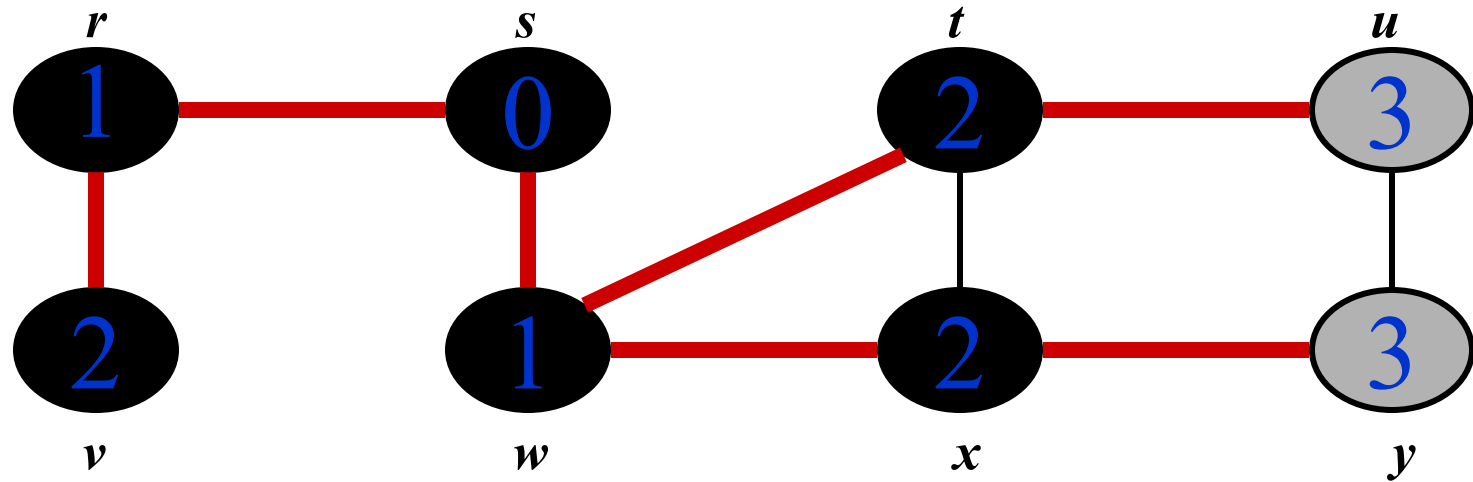
Breadth-First Search: Example



Q :

v	u	y
-----	-----	-----

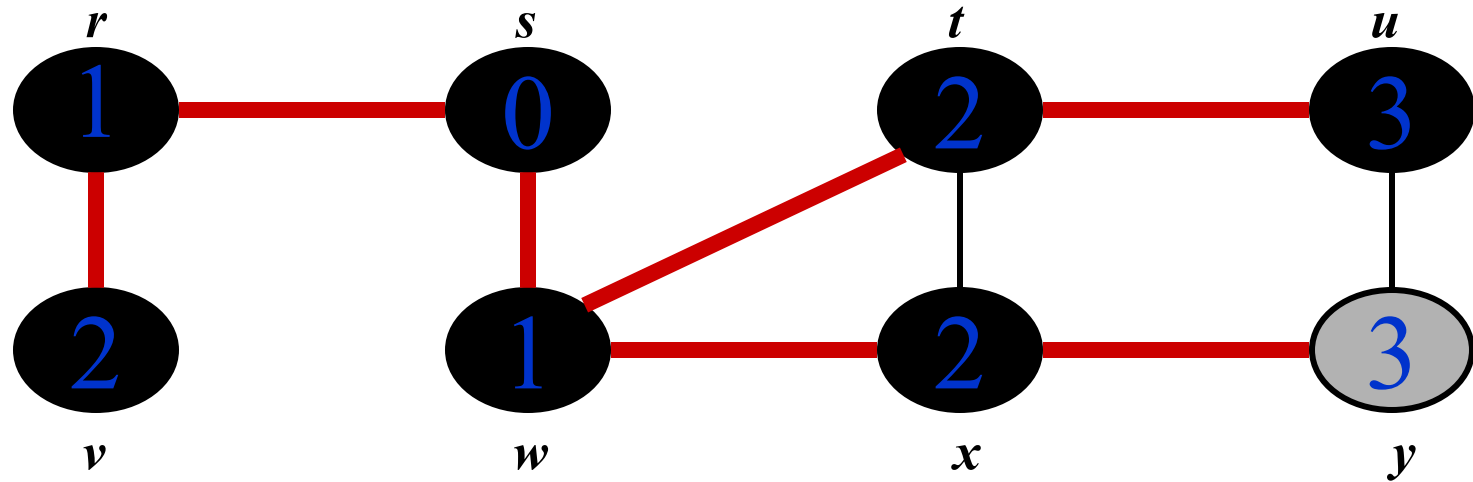
Breadth-First Search: Example



Q :

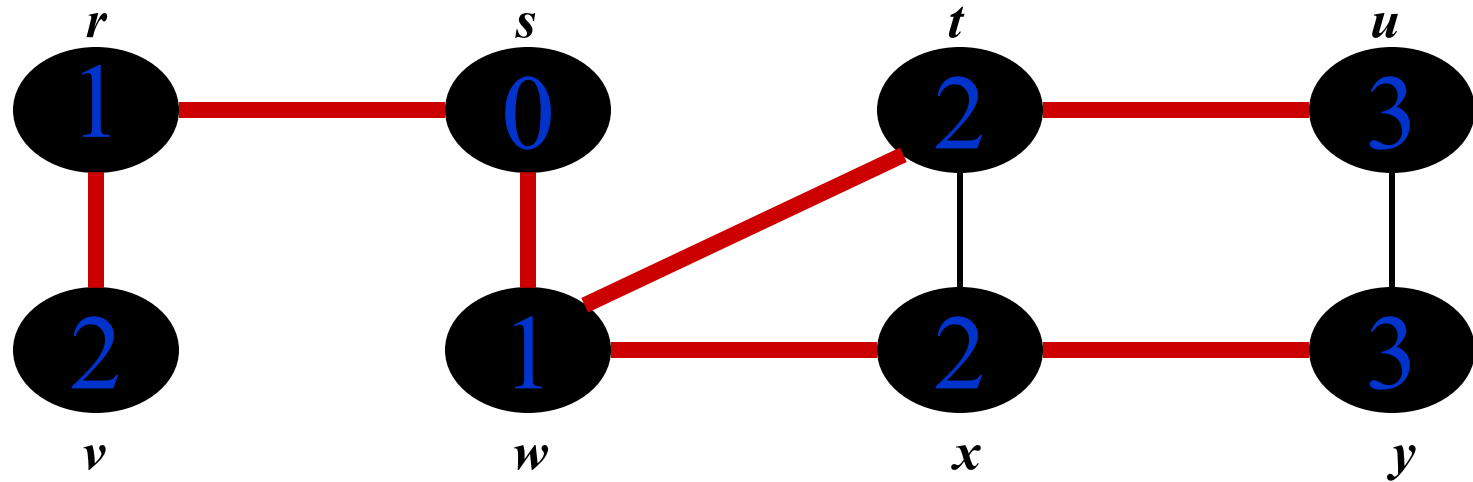
u	y
-----	-----

Breadth-First Search: Example



Q : y

Breadth-First Search: Example



$Q: \emptyset$

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

← *Touch every vertex: $O(V)$*

← *u = every vertex, but only once*

So v = every vertex that
appears in some other vert's
adjacency list

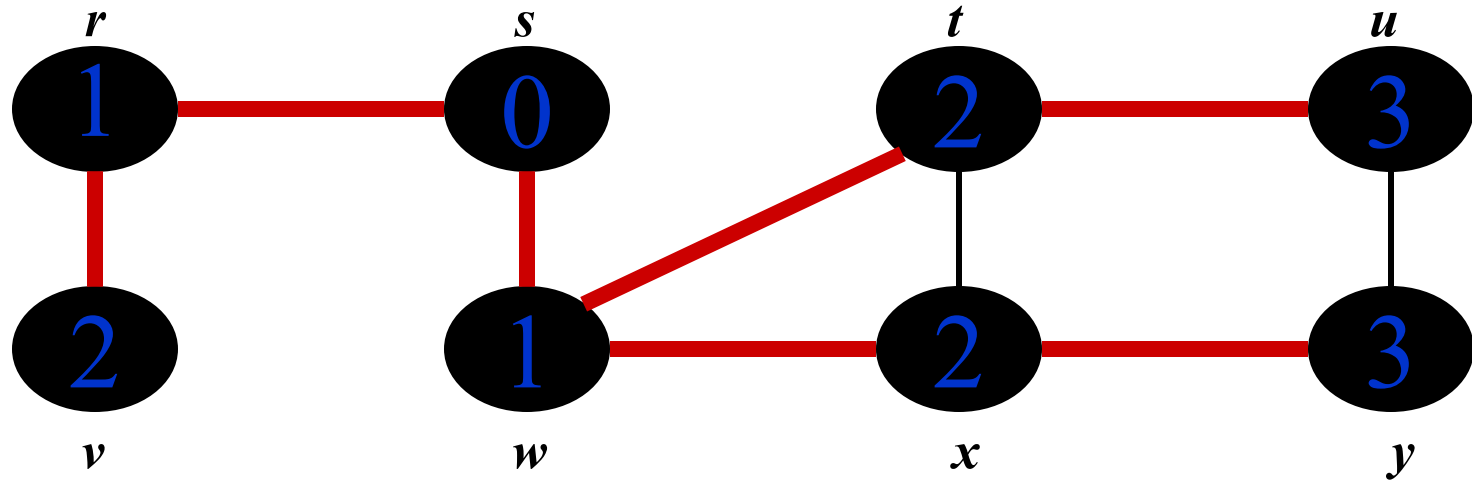
What will be the running time?

Total running time: $O(V+E)$

Finding Shortest Path Using BFS

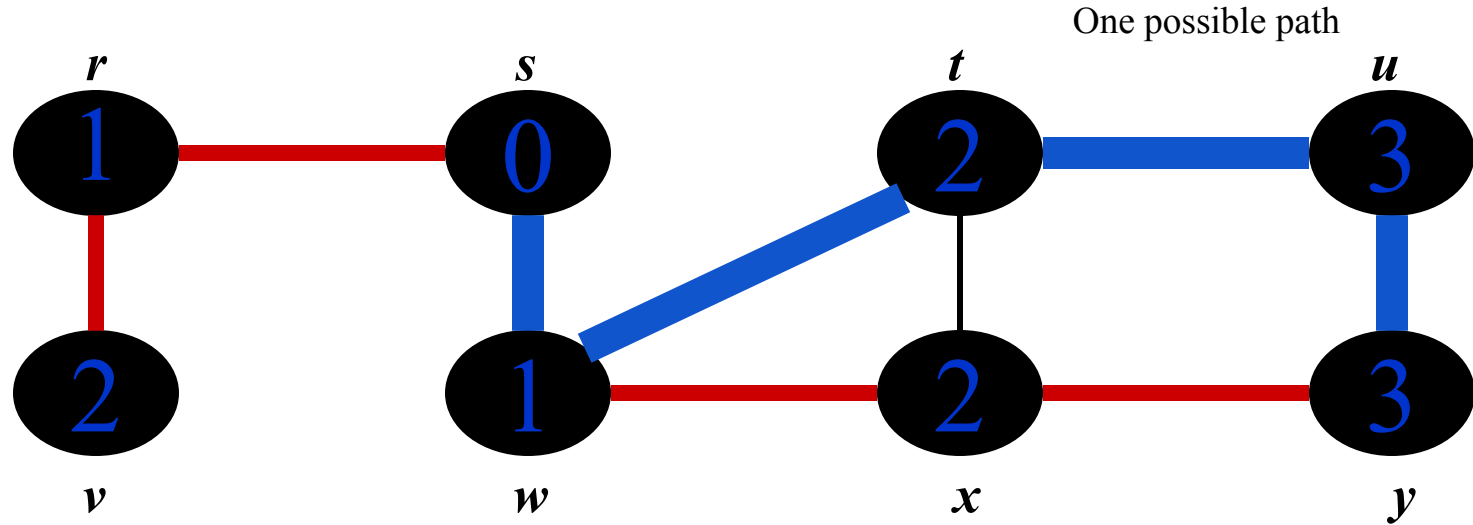
- We are storing the parent of all nodes, right?
- For finding shortest path between u and v , just start from v and backtrack using the parent of each node until you reach u !

Finding Shortest Path Using BFS



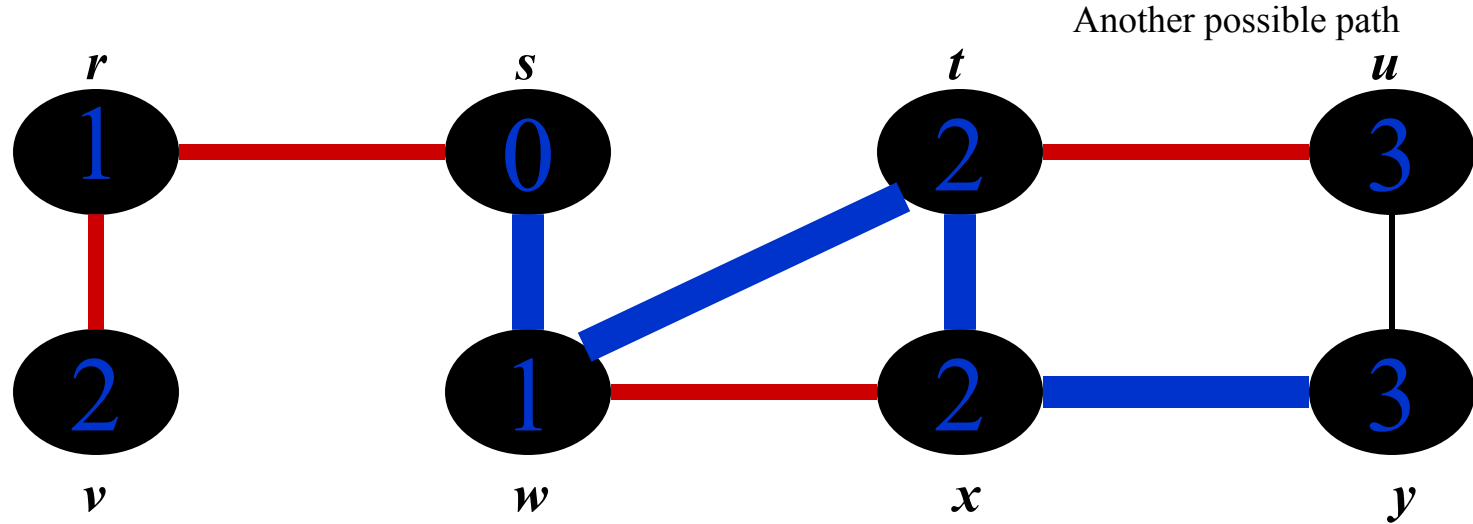
Question: Find shortest path between (s,y)

Finding Shortest Path Using BFS



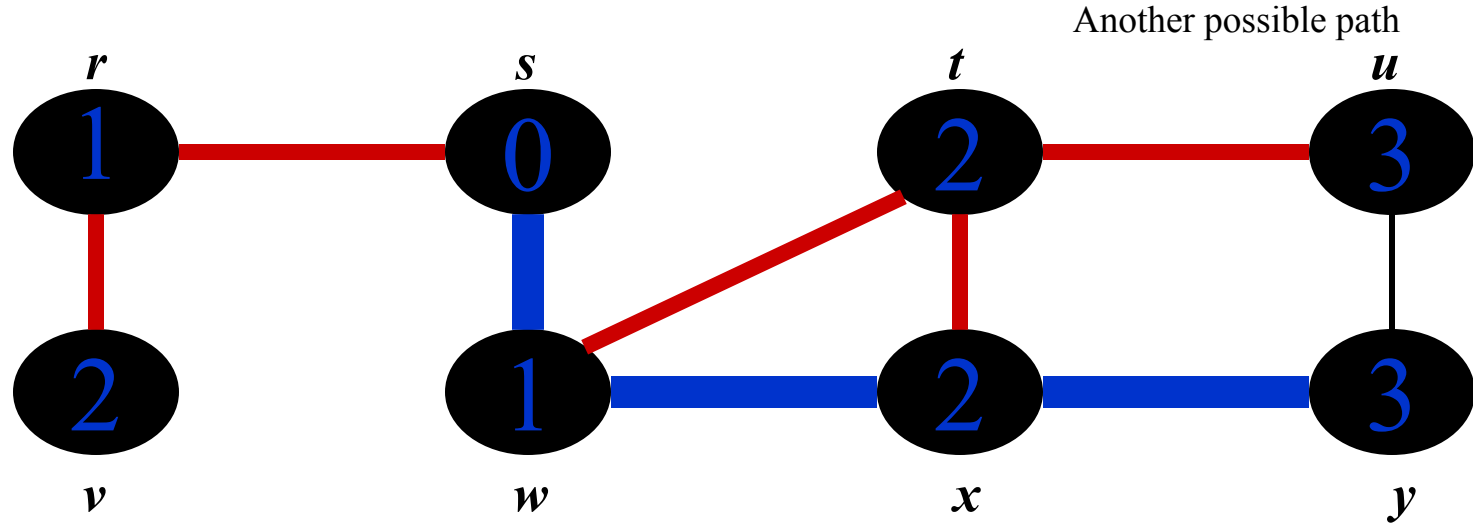
Question: Find shortest path between (s,y)

Finding Shortest Path Using BFS



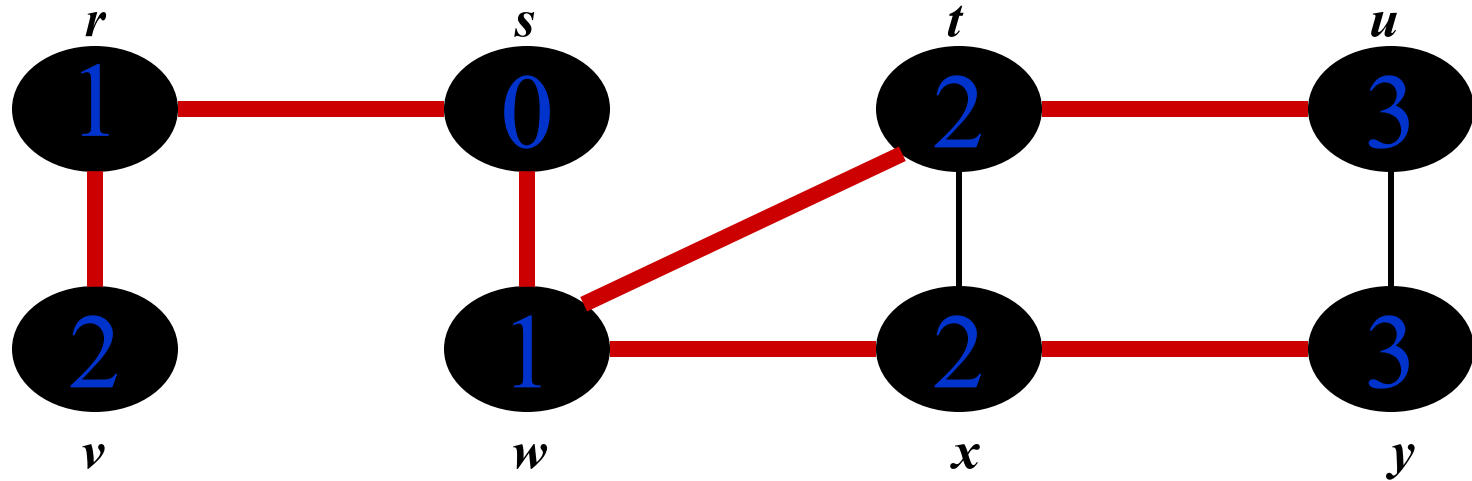
Question: Find shortest path between (s,y)

Finding Shortest Path Using BFS



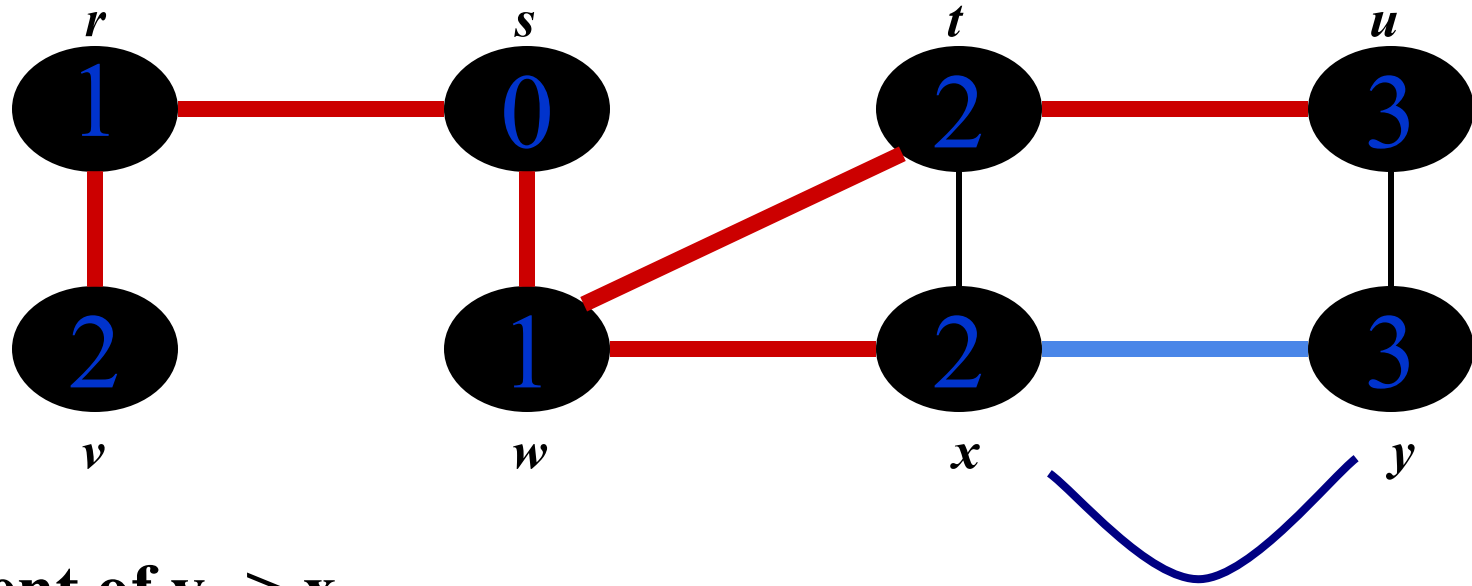
Question: Find shortest path between (s,y)

Finding Shortest Path Using BFS



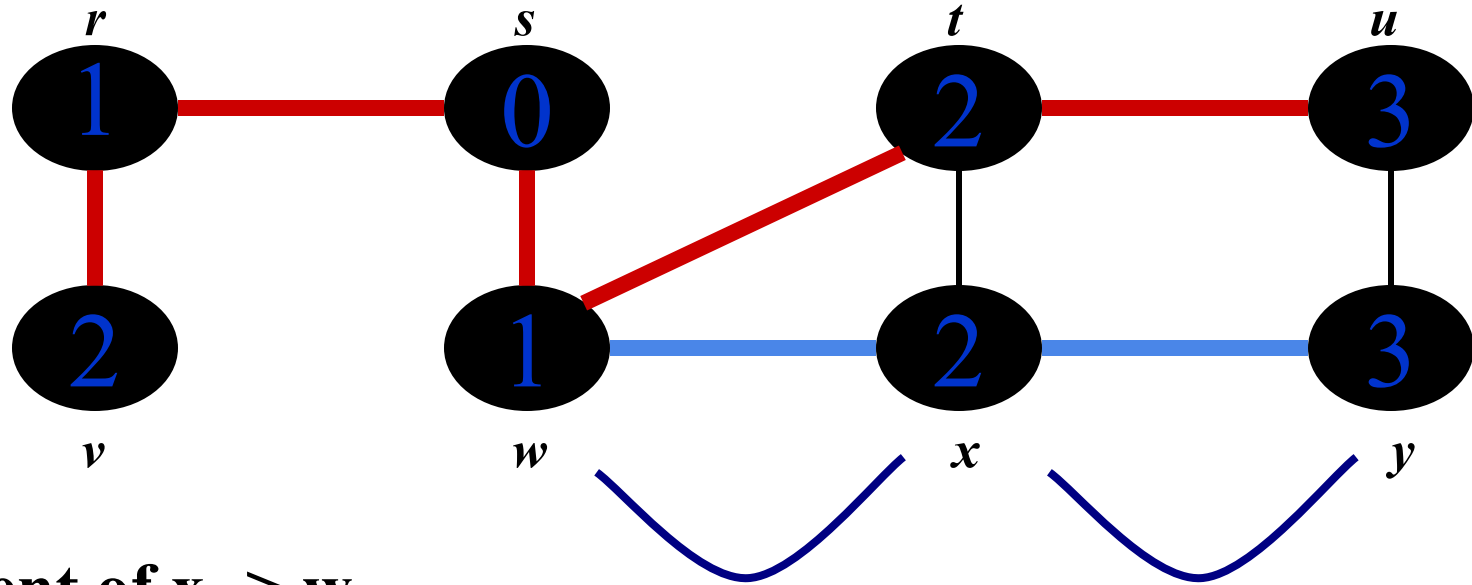
Start from y

Finding Shortest Path Using BFS



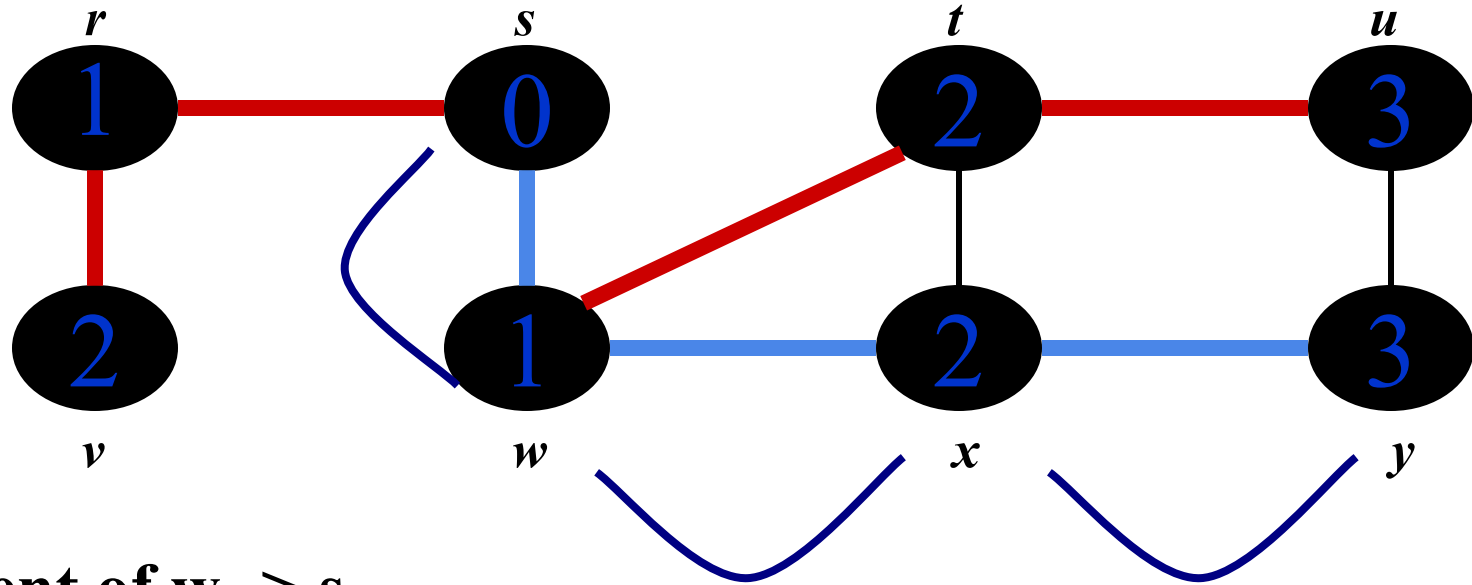
Parent of y $\rightarrow$ x

Finding Shortest Path Using BFS



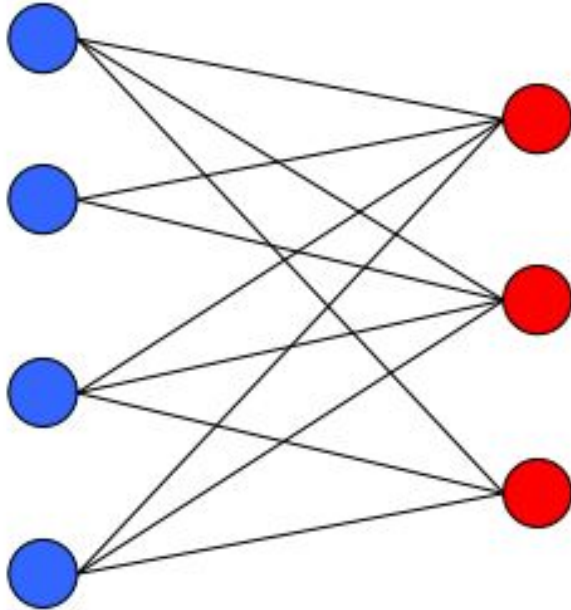
Parent of $x \rightarrow w$

Finding Shortest Path Using BFS



Parent of $w \rightarrow s$

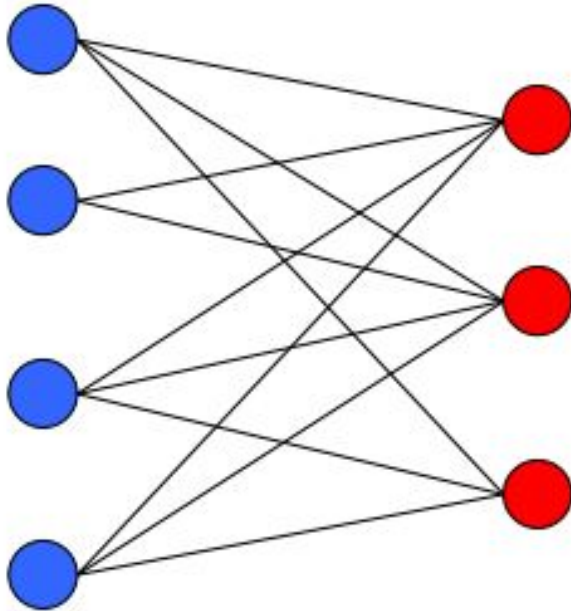
Bipartite Graphs



No adjacent vertices
have the same color

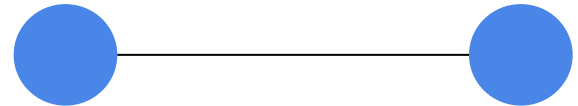
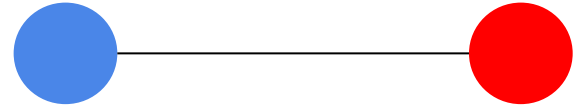
a bipartite graph

Bipartite Graphs

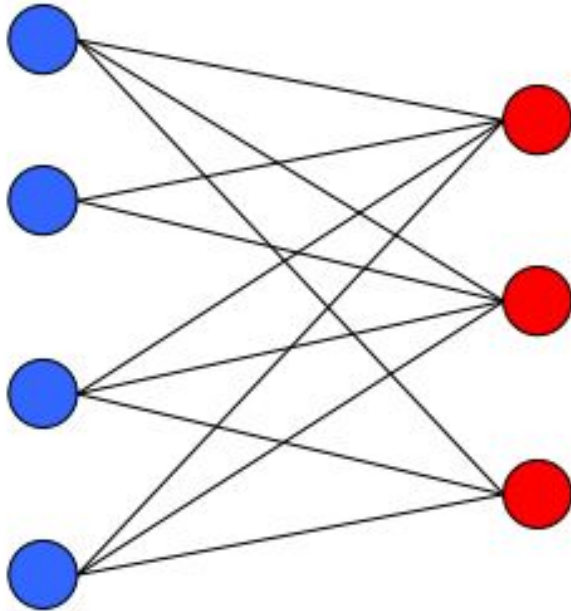


a bipartite graph

No adjacent vertices
have the same color

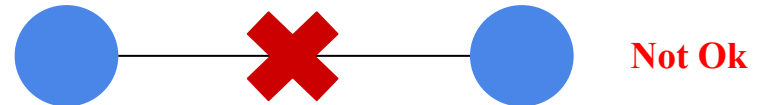
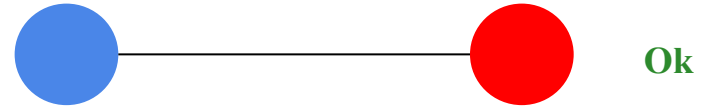


Bipartite Graphs

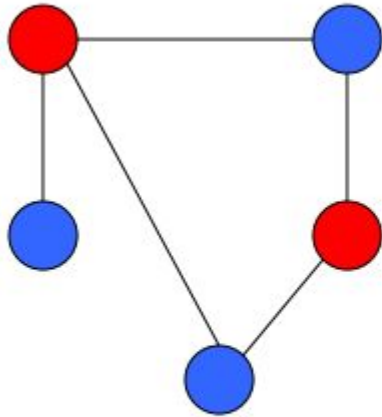


a bipartite graph

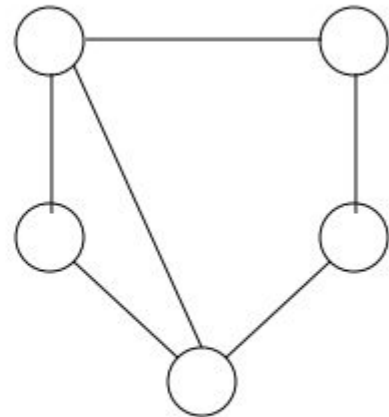
No adjacent vertices
have the same color



Bipartite Graphs

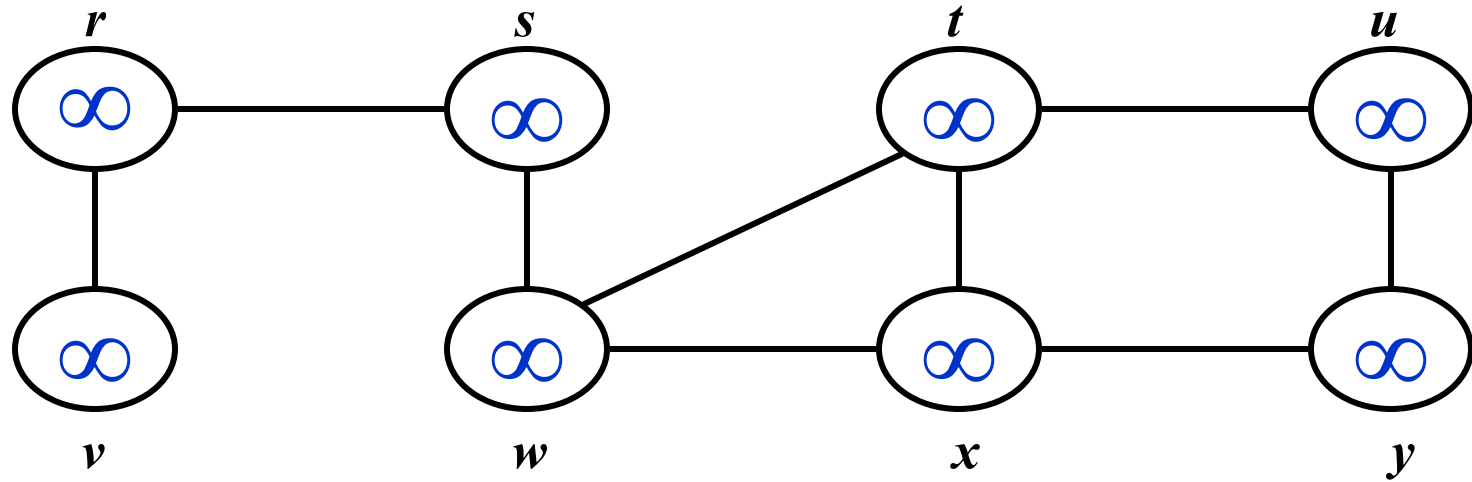


bipartite
(2-colorable)

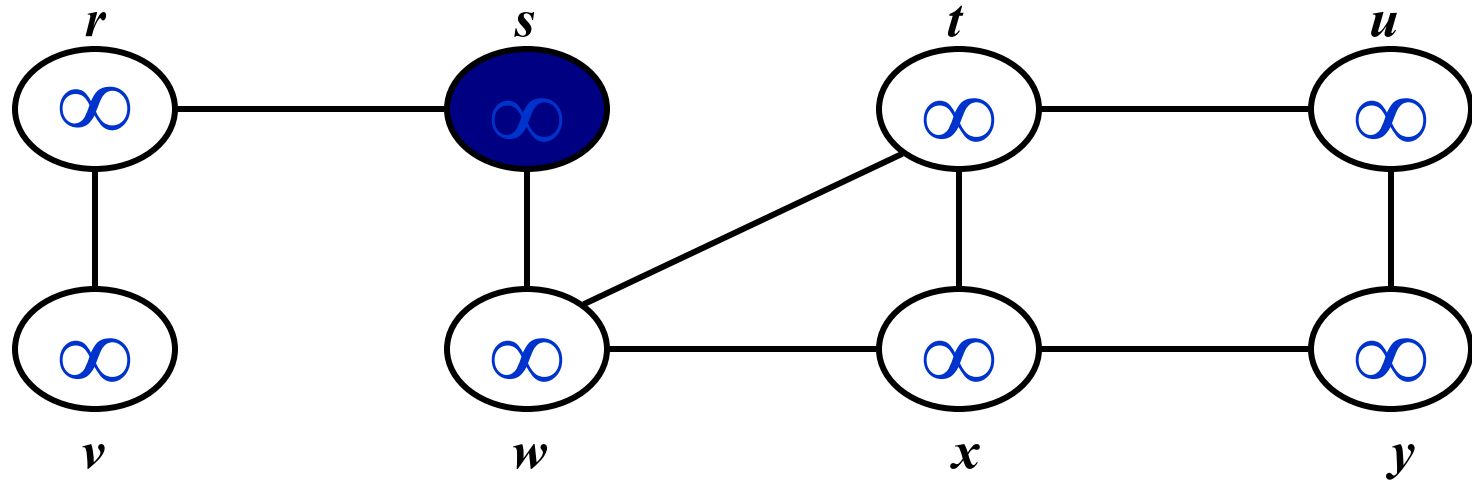


not bipartite
(not 2-colorable)

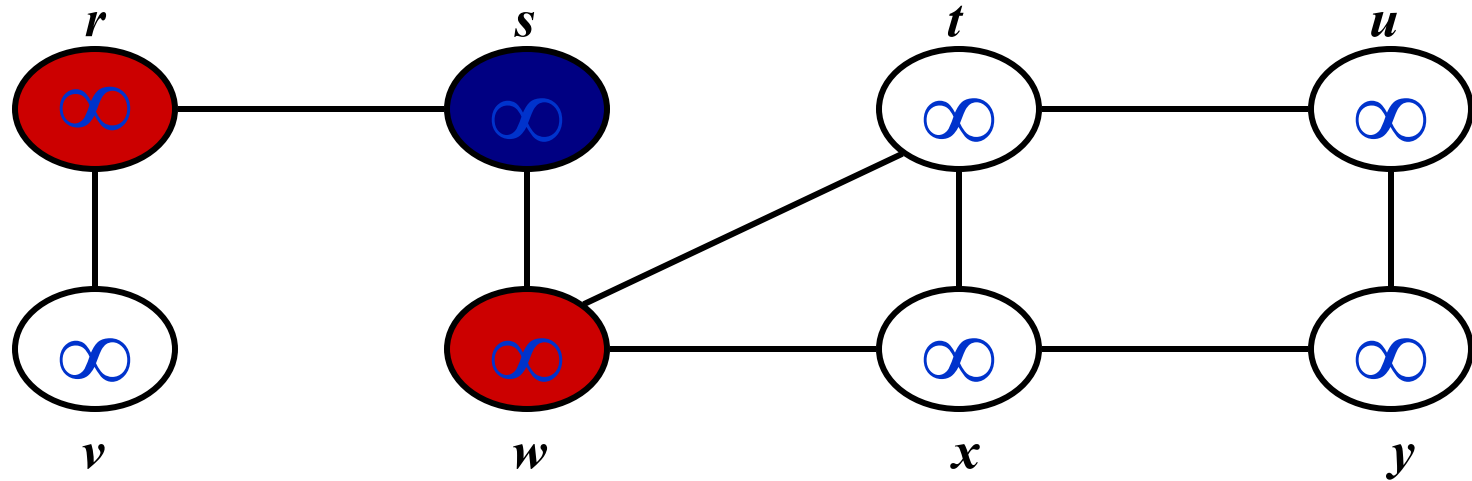
Detecting Bipartite Graphs with BFS



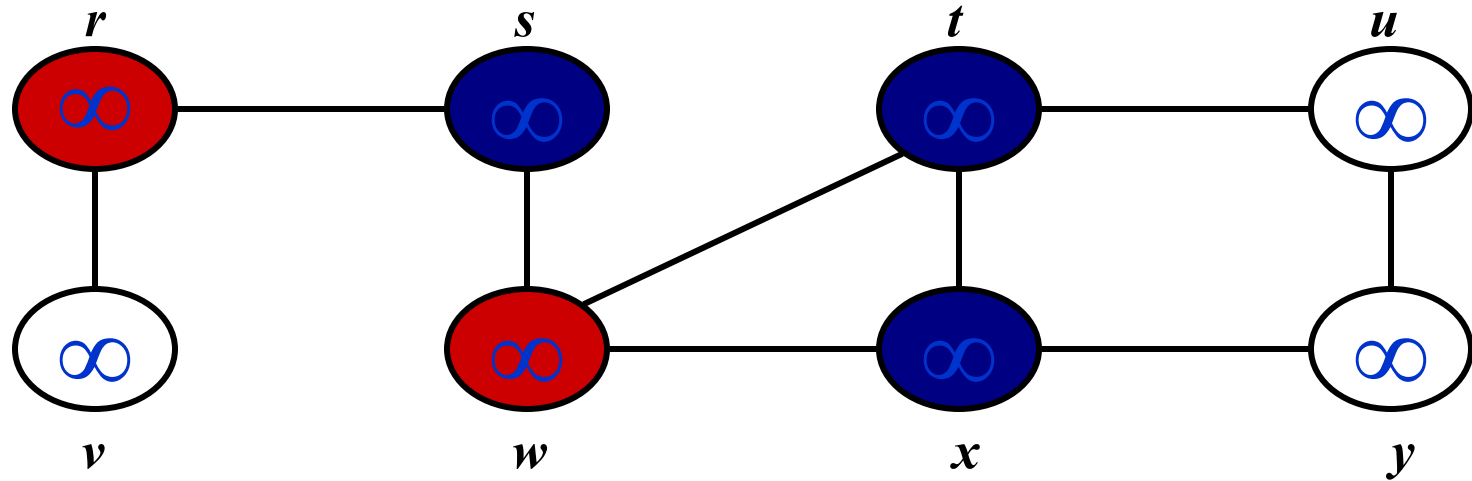
Detecting Bipartite Graphs with BFS



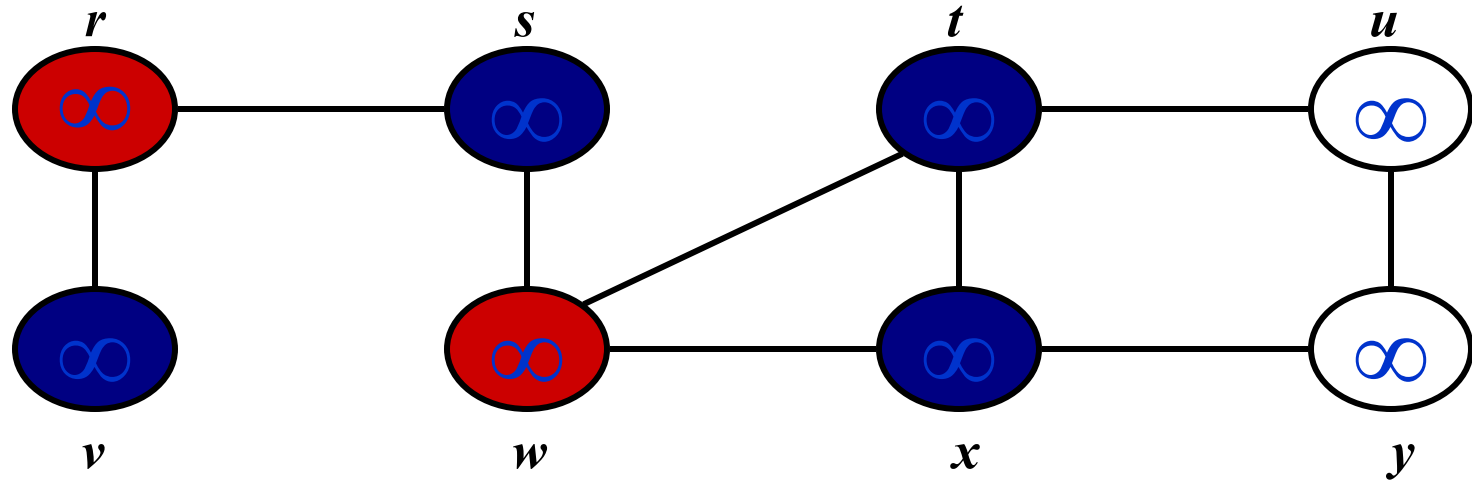
Detecting Bipartite Graphs with BFS



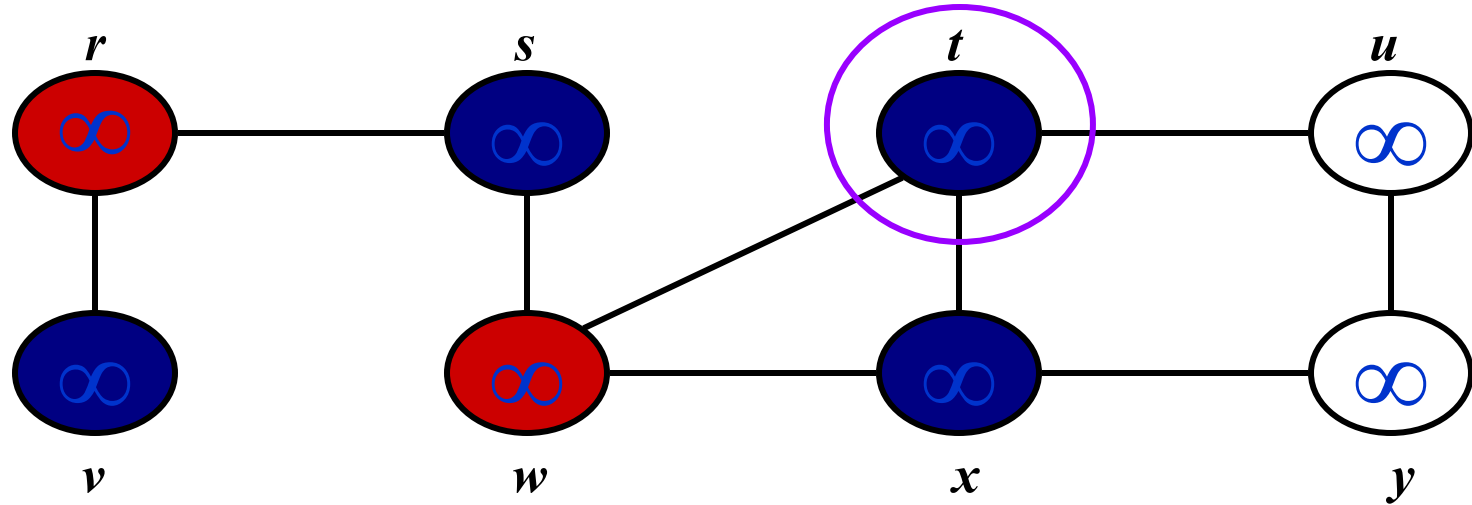
Detecting Bipartite Graphs with BFS



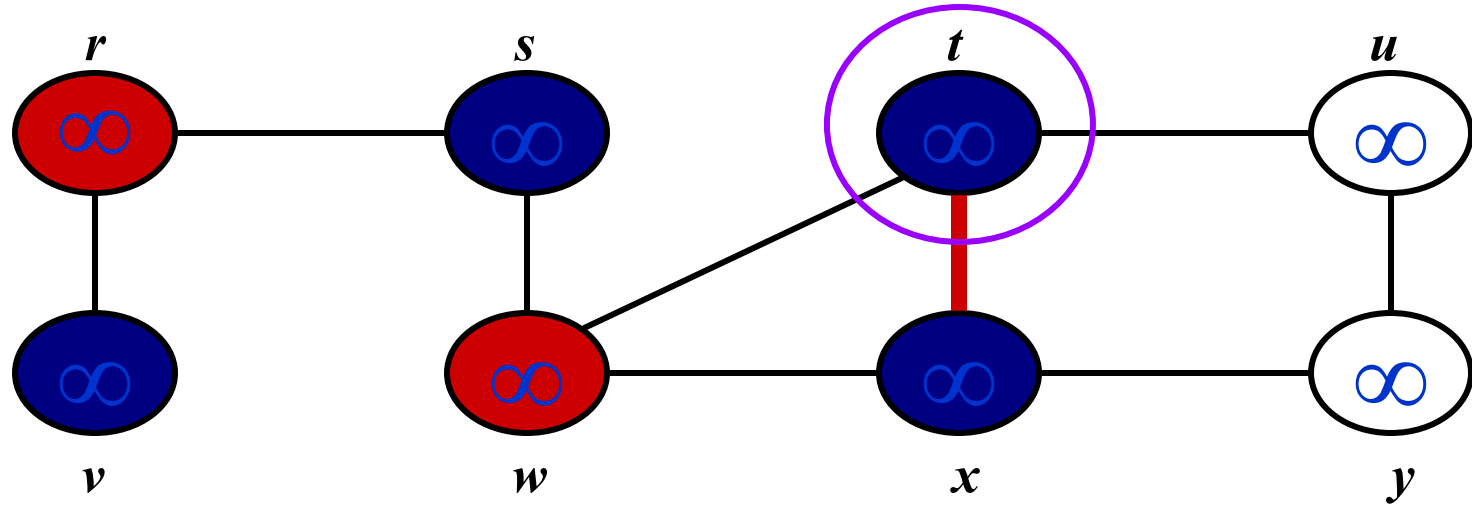
Detecting Bipartite Graphs with BFS



Detecting Bipartite Graphs with BFS



Detecting Bipartite Graphs with BFS



Same colored vertex encountered!
Not bipartite

Detecting Bipartite Graphs with BFS

```
BipartiteBicolorableBFS(G, s):
```

```
    for each vertex u in G.V:
```

```
        u.color = NIL
```

```
    s.color = 0
```

```
    Q = new Queue()
```

```
    Q.enqueue(s)
```

```
    while Q is not empty:
```

```
        u = Q.dequeue()
```

```
        for each v in G.Adj[u]:
```

```
            if v.color == NIL:
```

```
                v.color = 1 - u.color
```

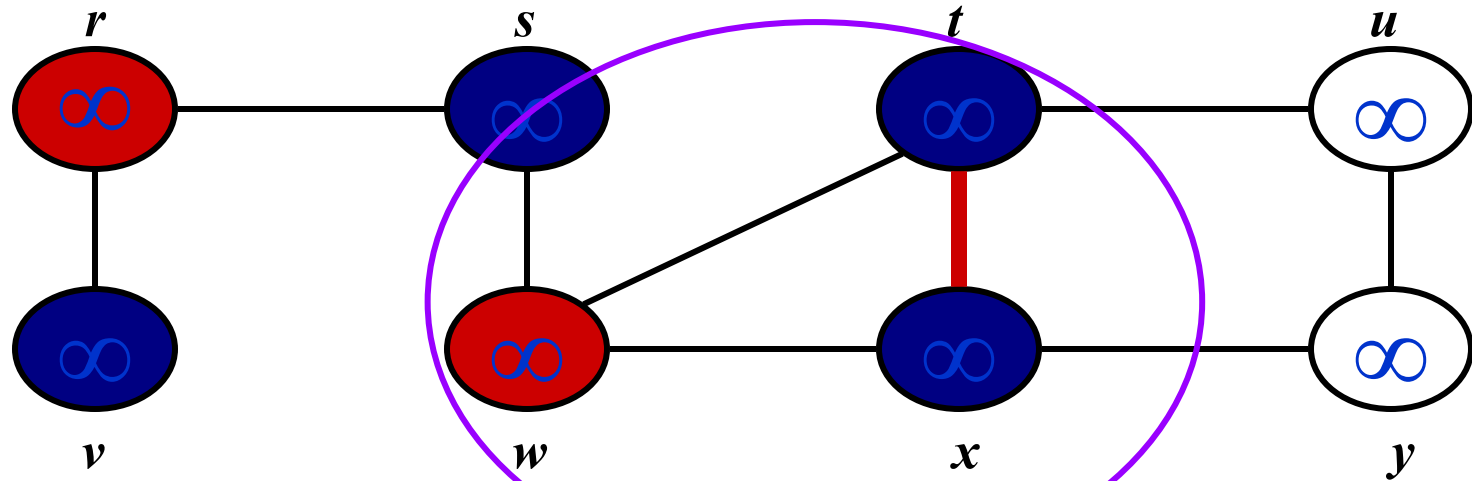
```
                Q.enqueue(v)
```

```
            else if v.color == u.color:
```

```
                return False
```

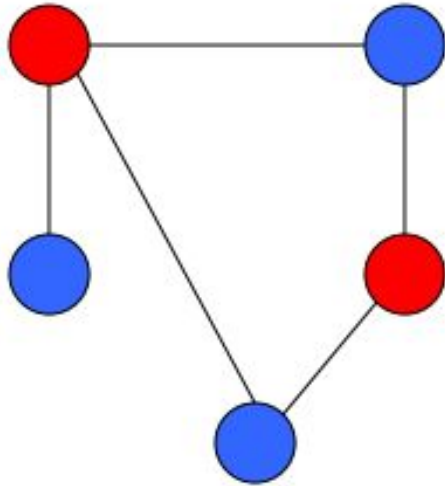
```
    return True
```

Detecting Odd Cycles in Graphs with BFS



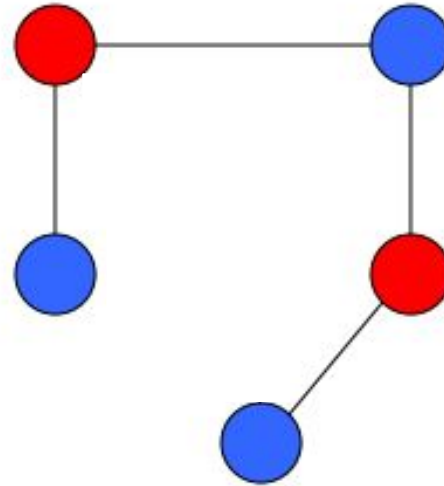
Same colored vertex encountered!
Not bipartite, odd length cycle

Detecting Odd Cycles in Graphs with BFS



bipartite
(2-colorable)

bipartite, even length cycle



bipartite
(2-colorable)

bipartite, no cycle

Depth-First Search

- *Depth-first search* is another strategy for exploring a graph
 - Explore “deeper” in the graph whenever possible
 - Edges are explored out of the most recently discovered vertex v that still has unexplored edges
 - When all of v 's edges have been explored, backtrack to the vertex from which v was discovered

Depth-First Search

- Vertices initially colored white
- Then colored gray when discovered
- Then black when finished

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->d represent?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->f represent?

Depth-First Search: The Code

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

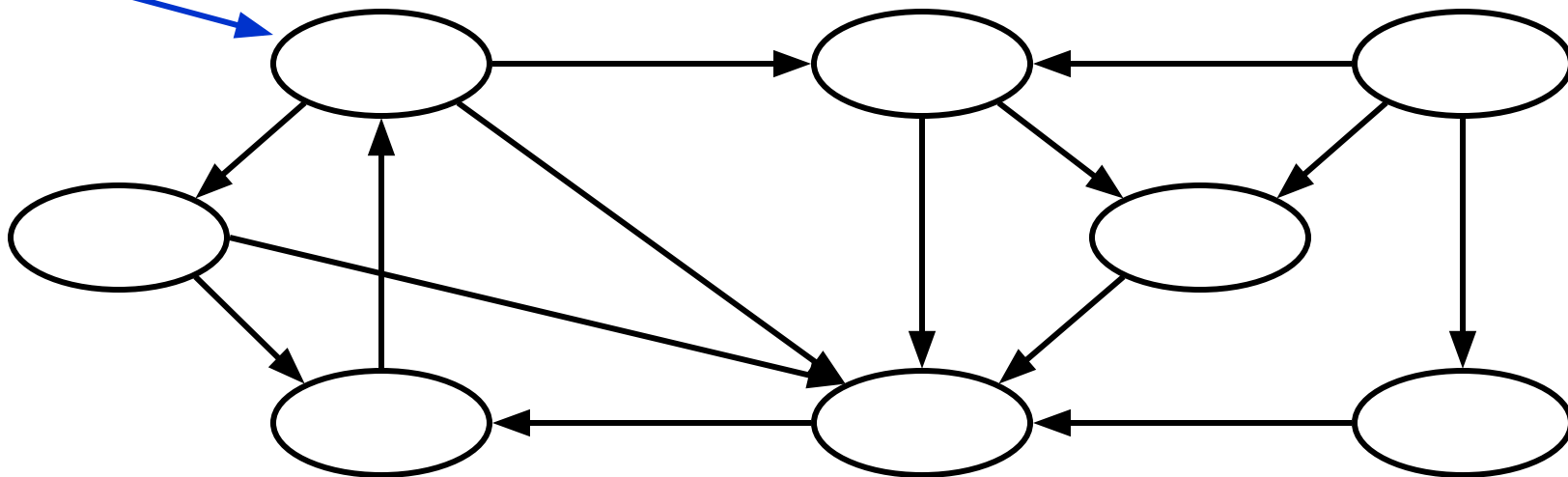
discovery time u->d and finishing time u->f

Depth-First Search: The Code

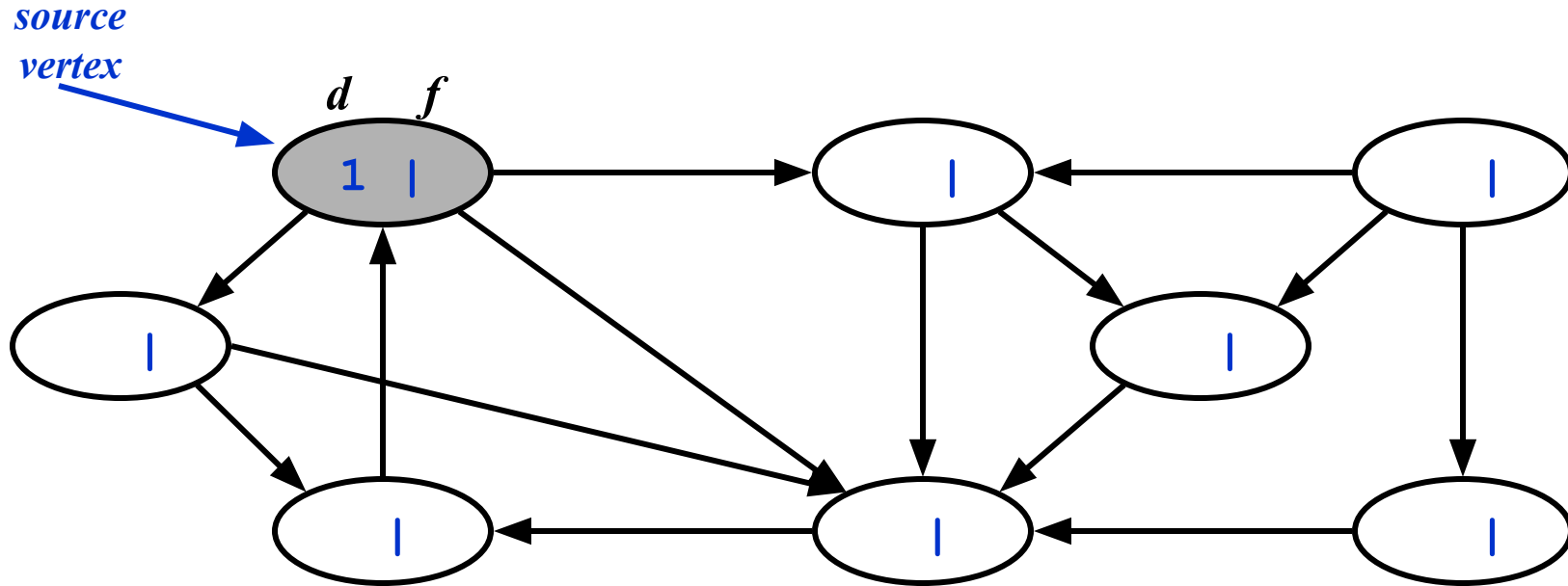
```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

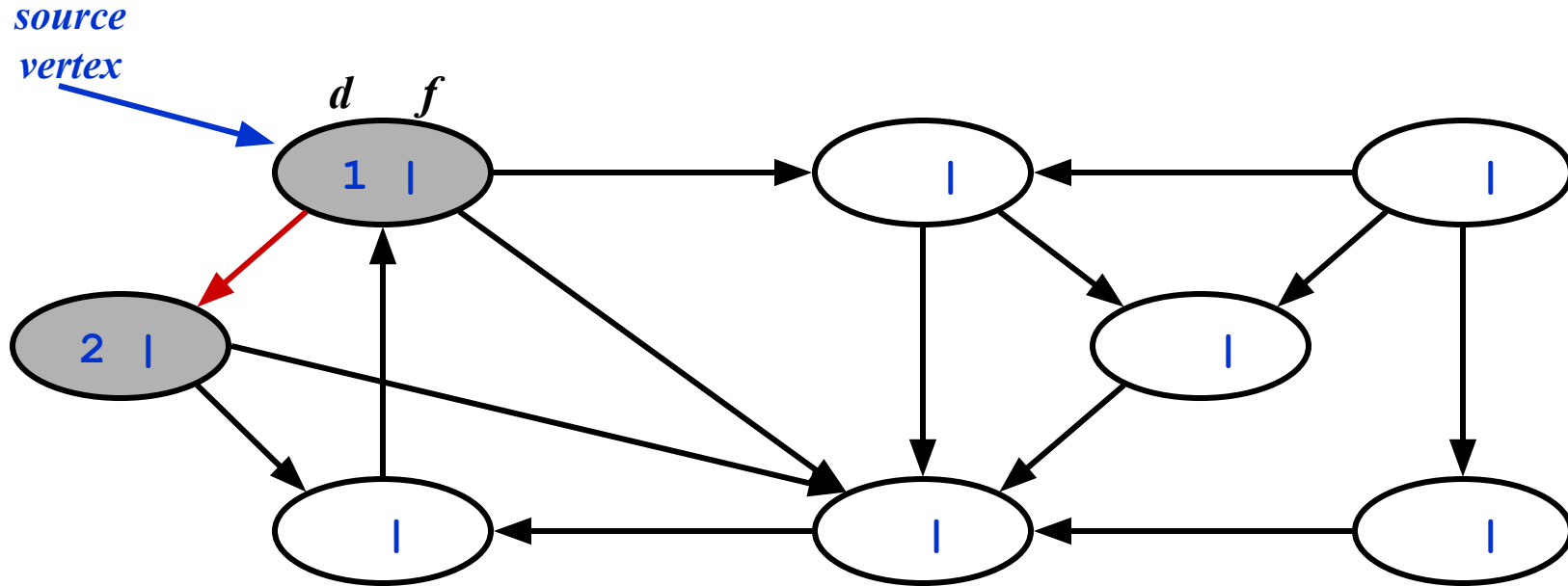
Running time of DFS = $O(V+E)$



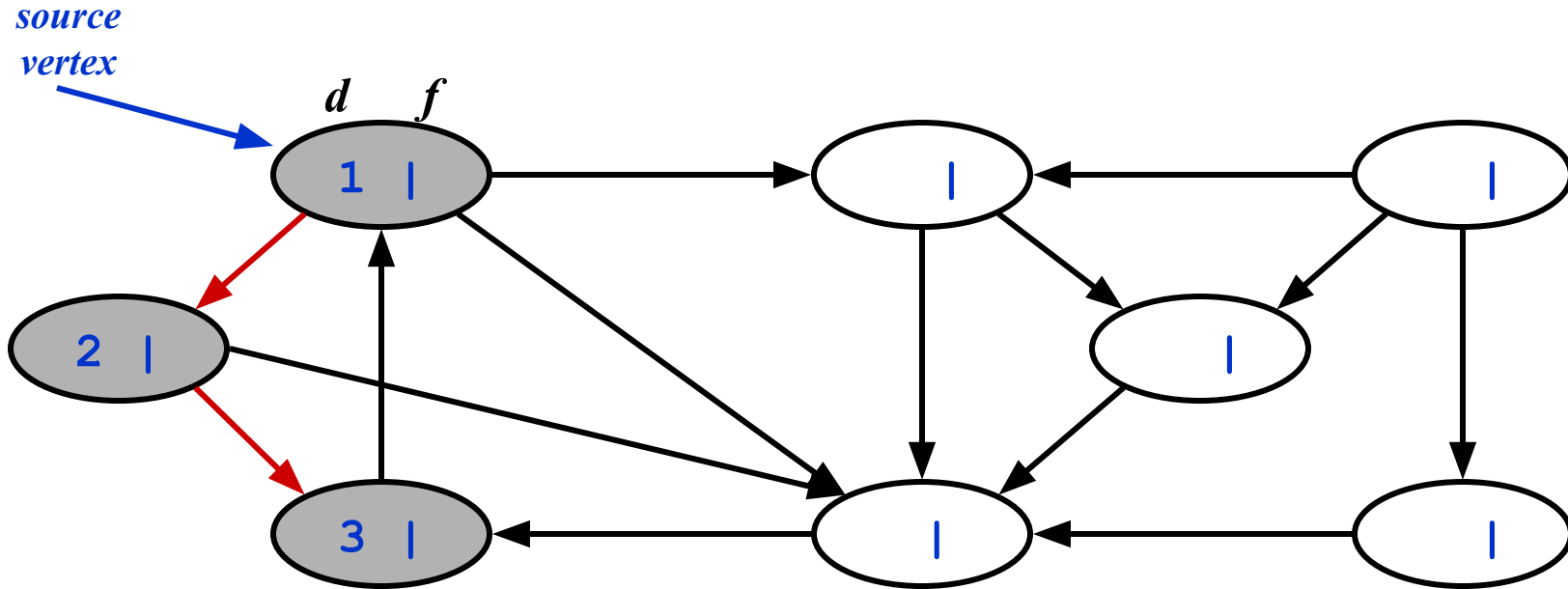
DFS Example



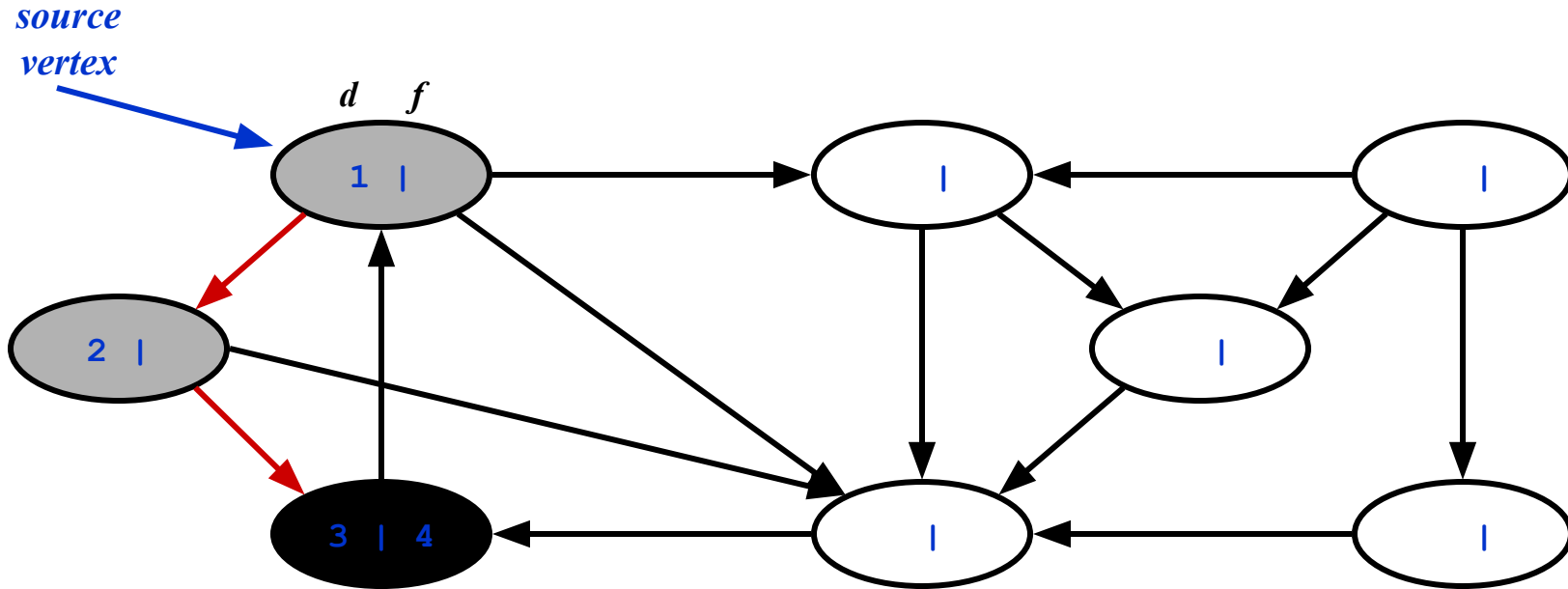
DFS Example



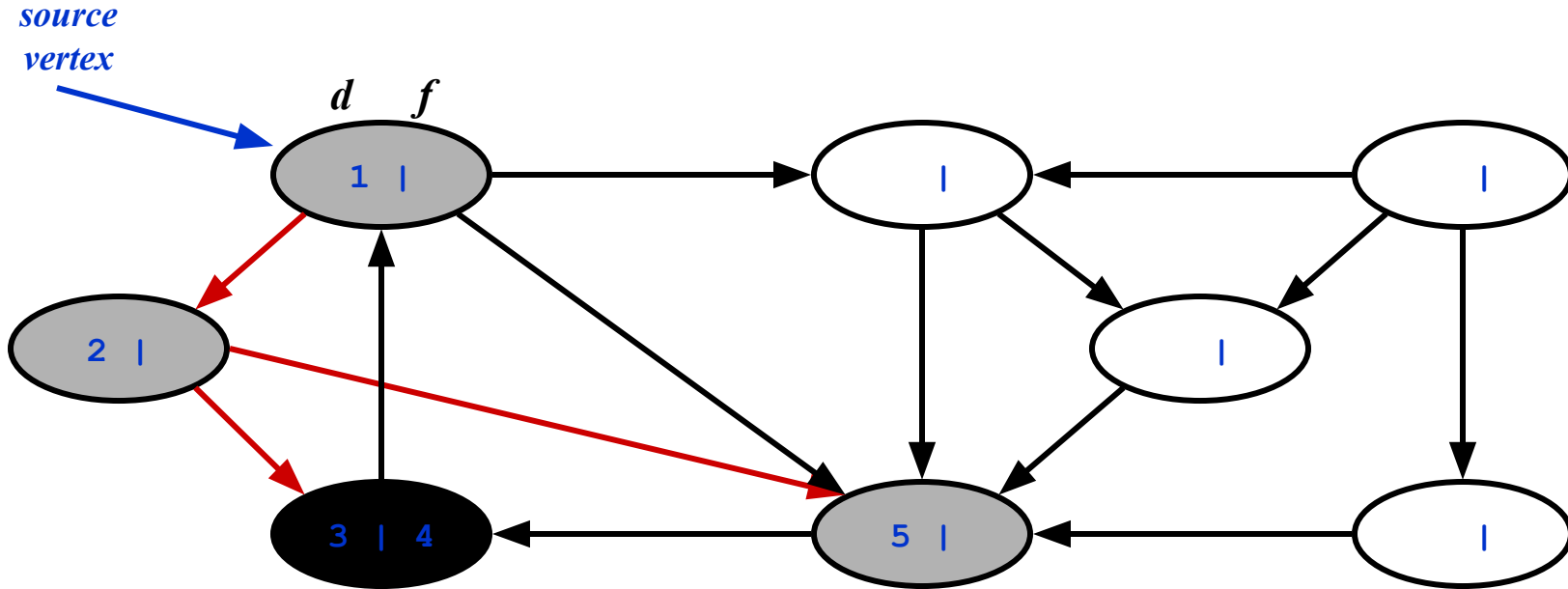
DFS Example



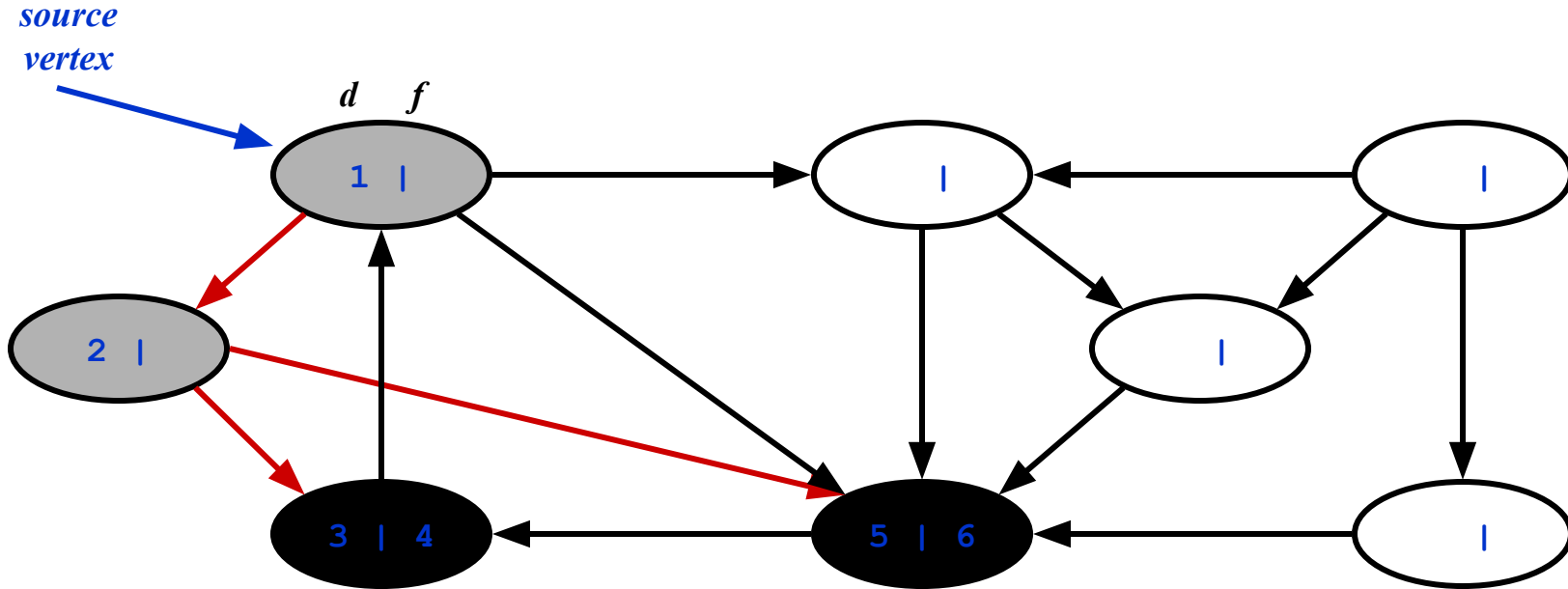
DFS Example



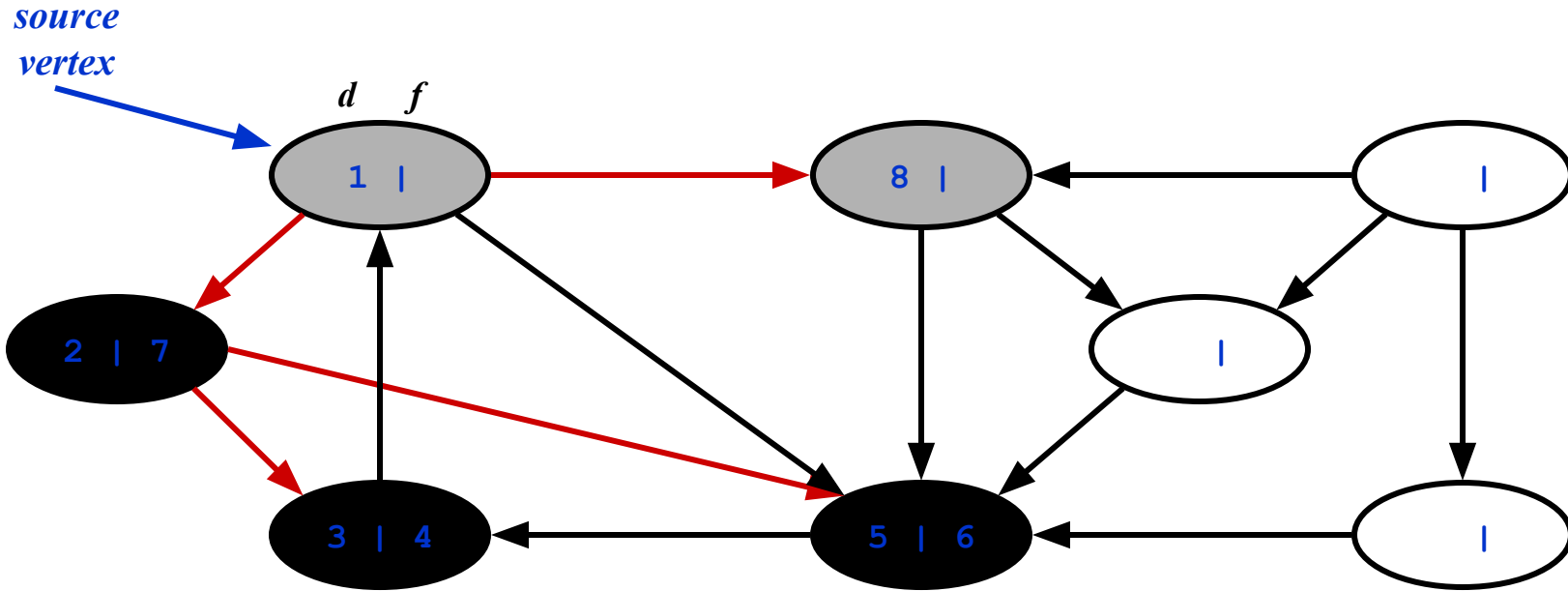
DFS Example



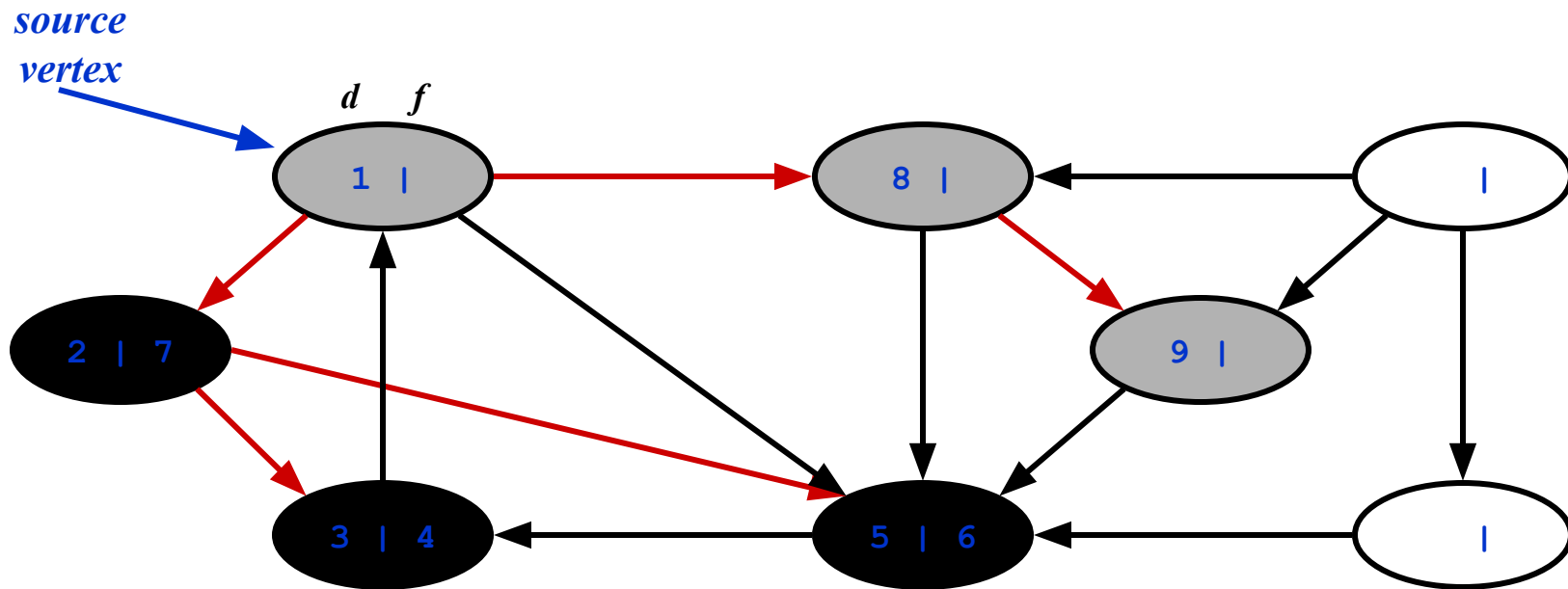
DFS Example



DFS Example

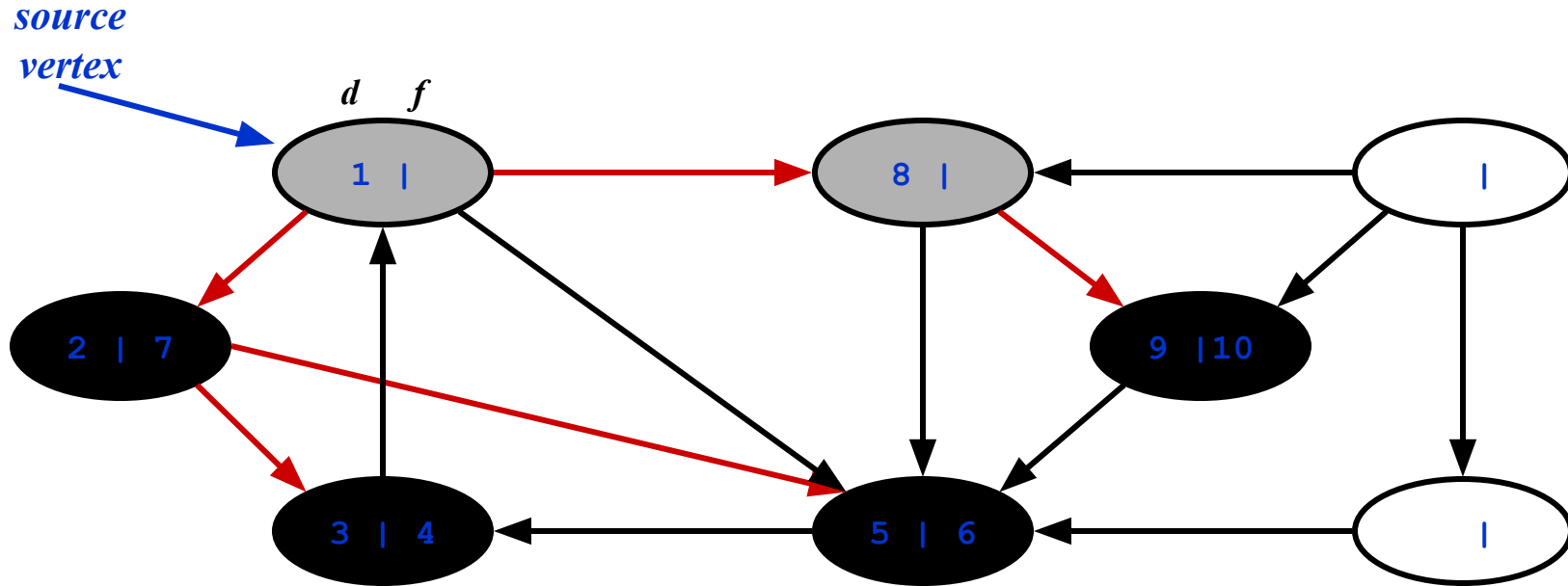


DFS Example

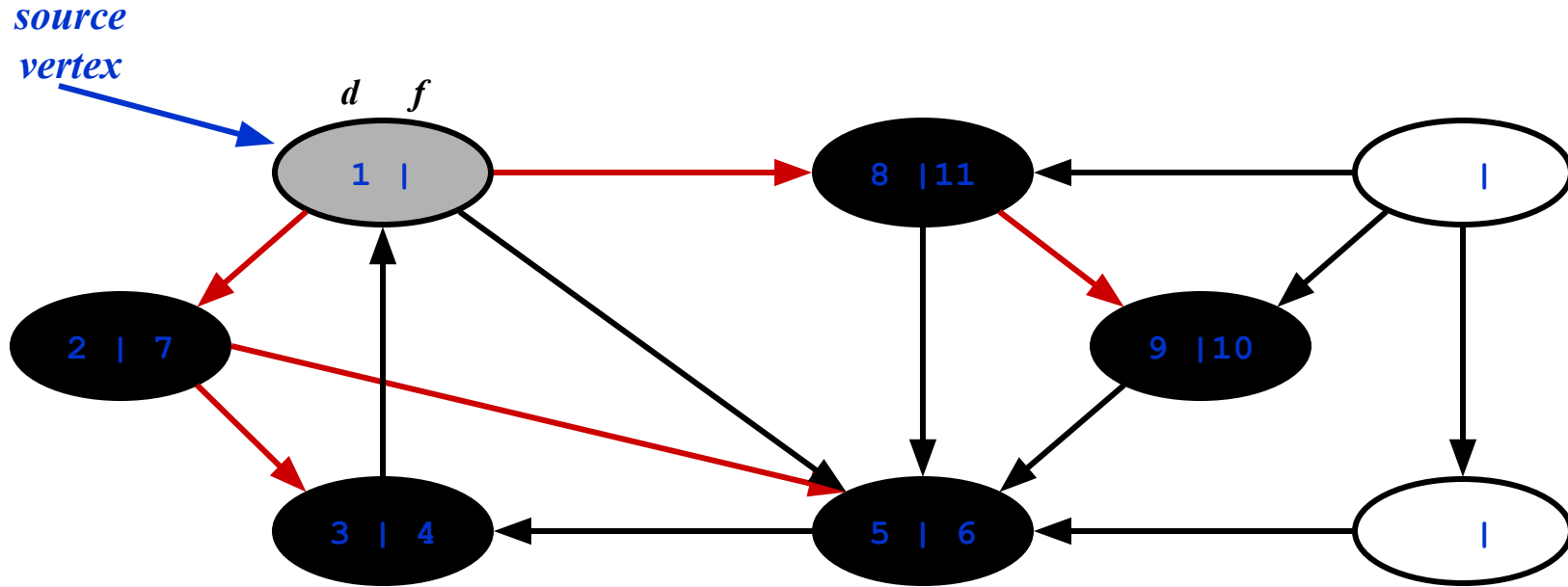


*What is the structure of the grey vertices?
What do they represent?*

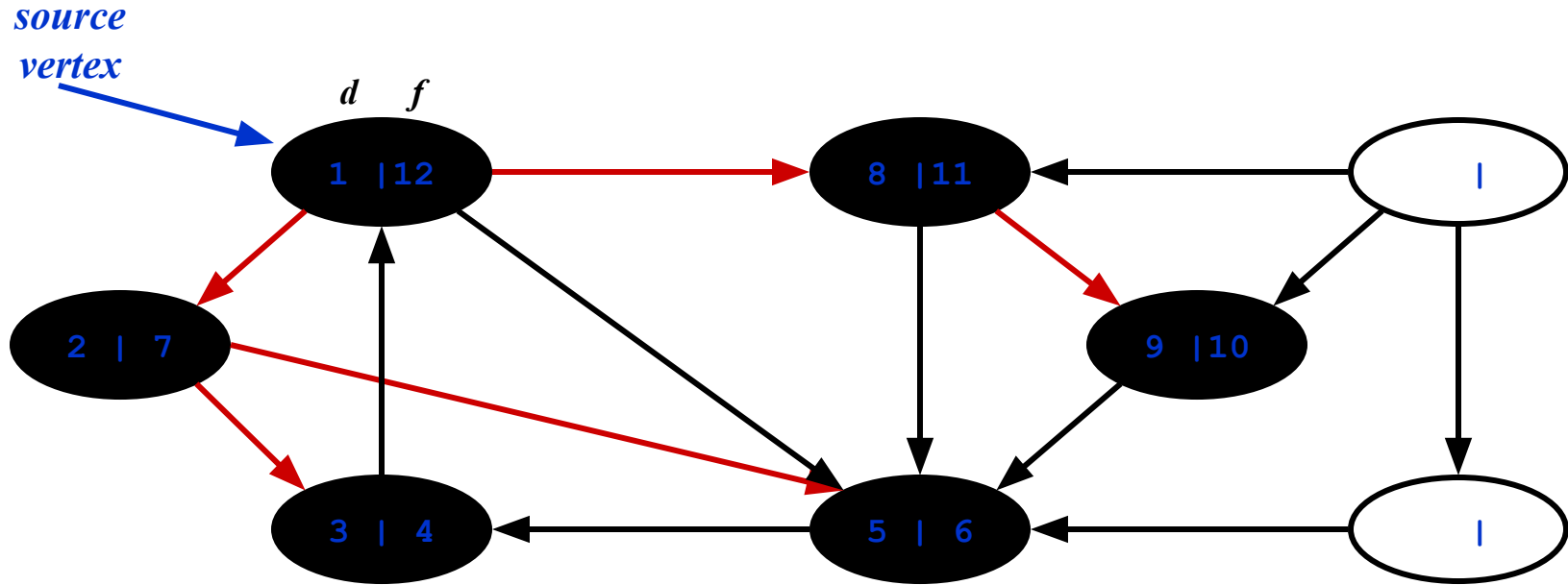
DFS Example



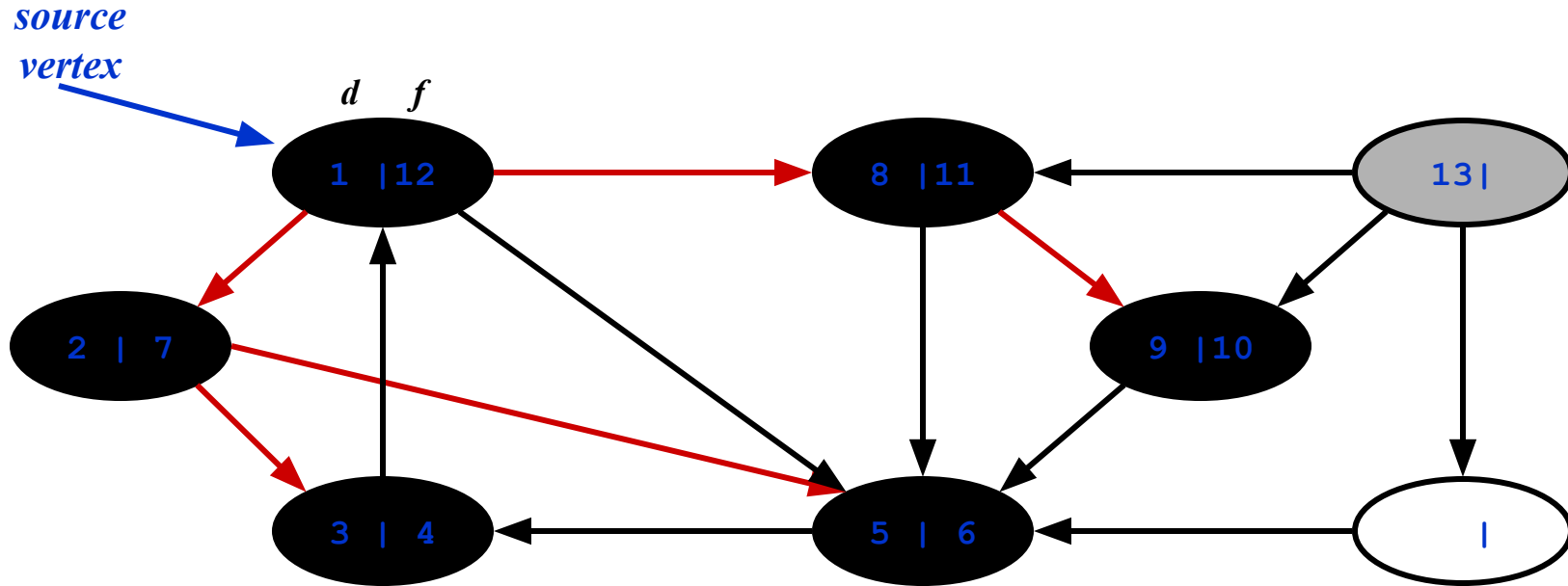
DFS Example



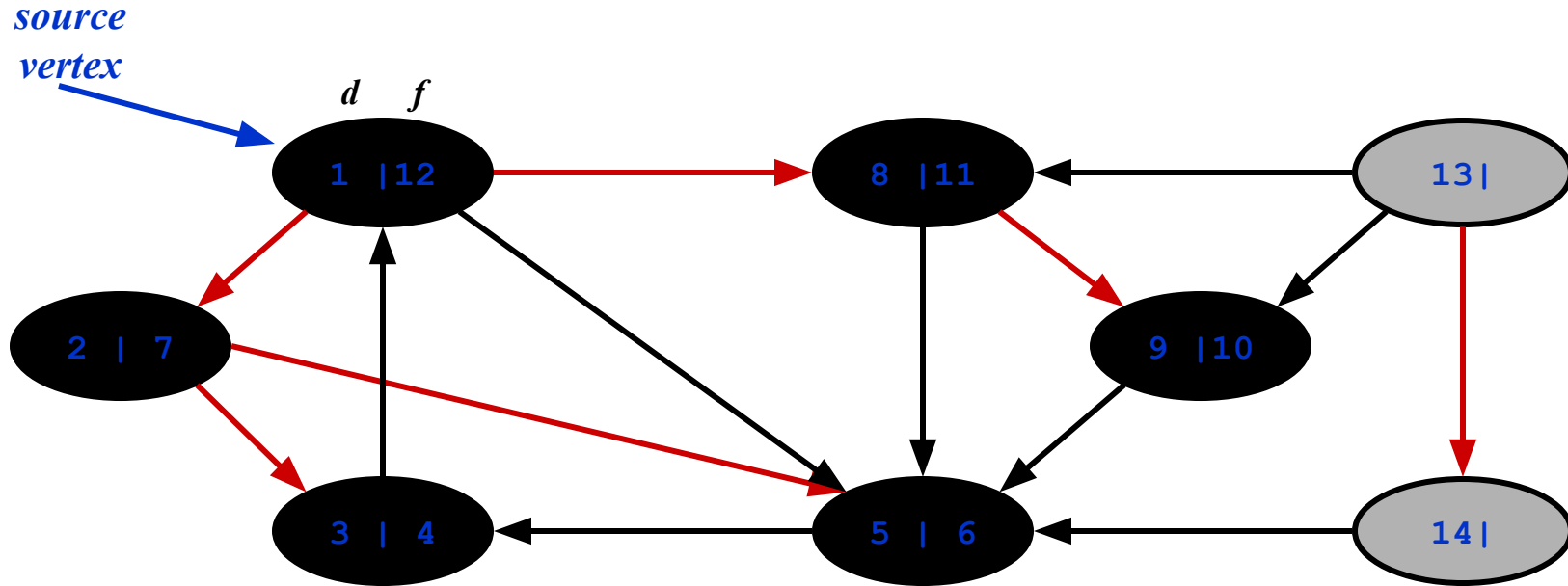
DFS Example



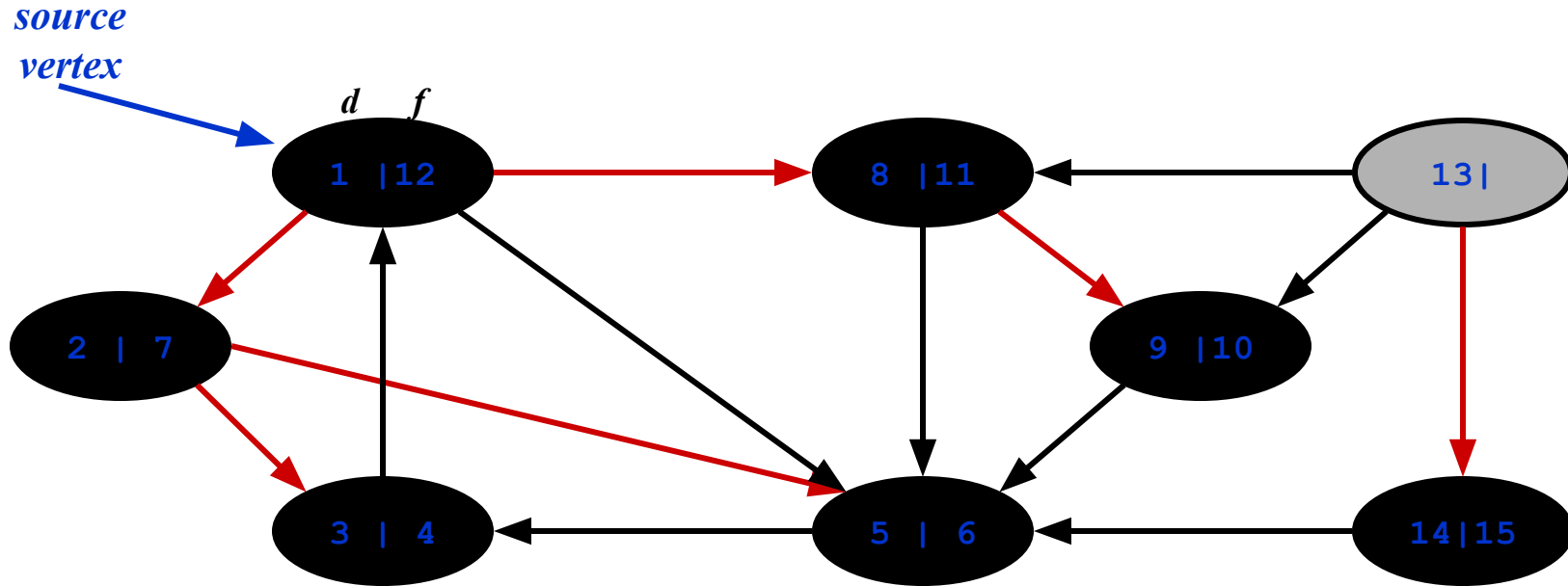
DFS Example



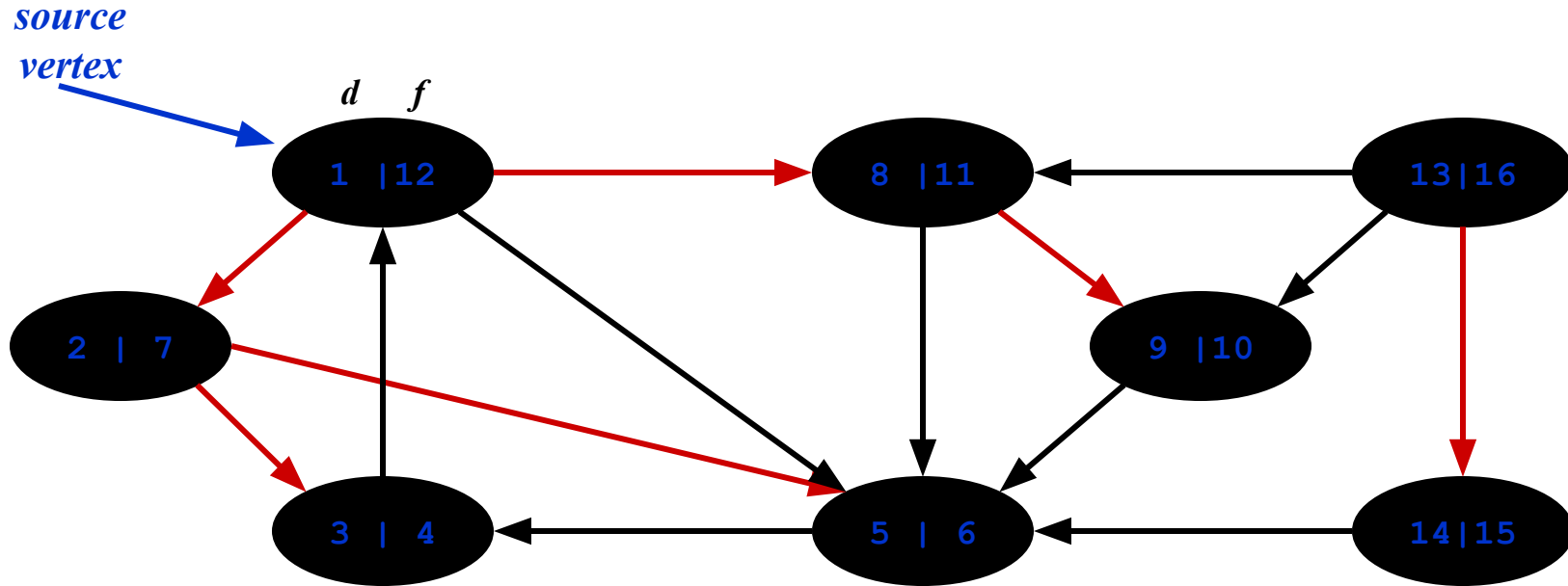
DFS Example



DFS Example



DFS Example



DFS: Kinds of edges

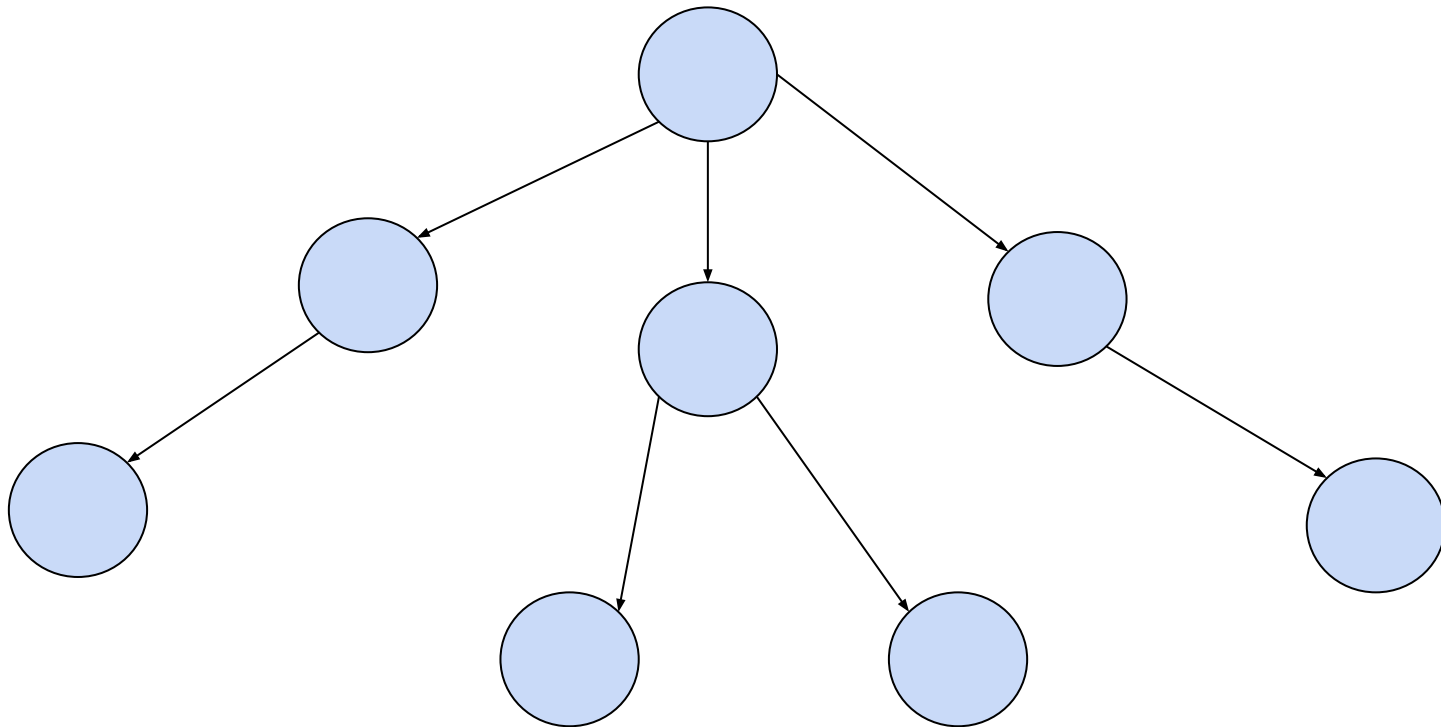
- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter WHITE vertex
 - *Back edge*: encounter GREY vertex
 - *Cross edge*: encounter BLACK vertex

CLASSIFYING EDGES OF A DIGRAPH

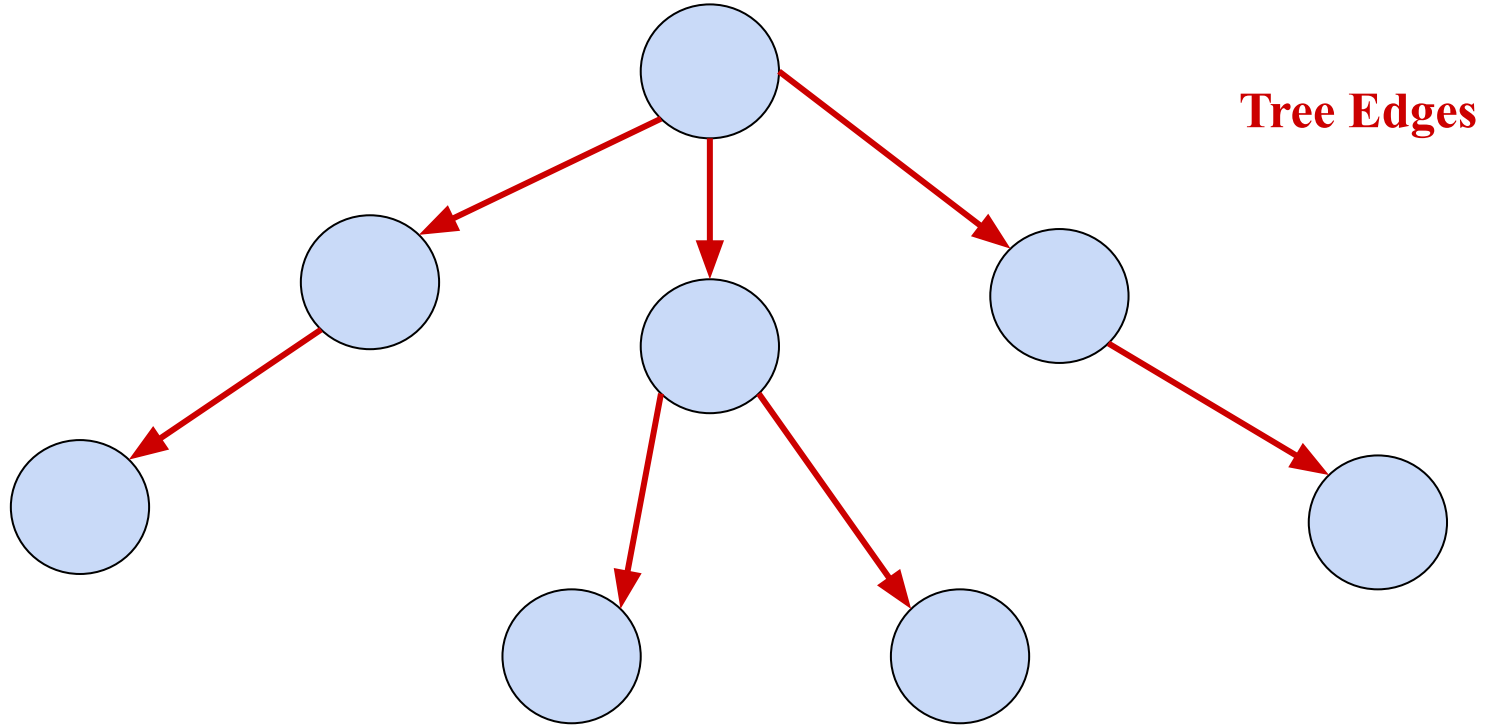
Edge (u, v) is:

- Tree edge – if v is white
- Back edge – if v is gray
- Forward or cross - if v is black
 - Forward edge – if v is black and $d[u] < d[v]$
(v was discovered after u)
 - Cross edge – if v is black and $d[u] > d[v]$
(u discovered after v)

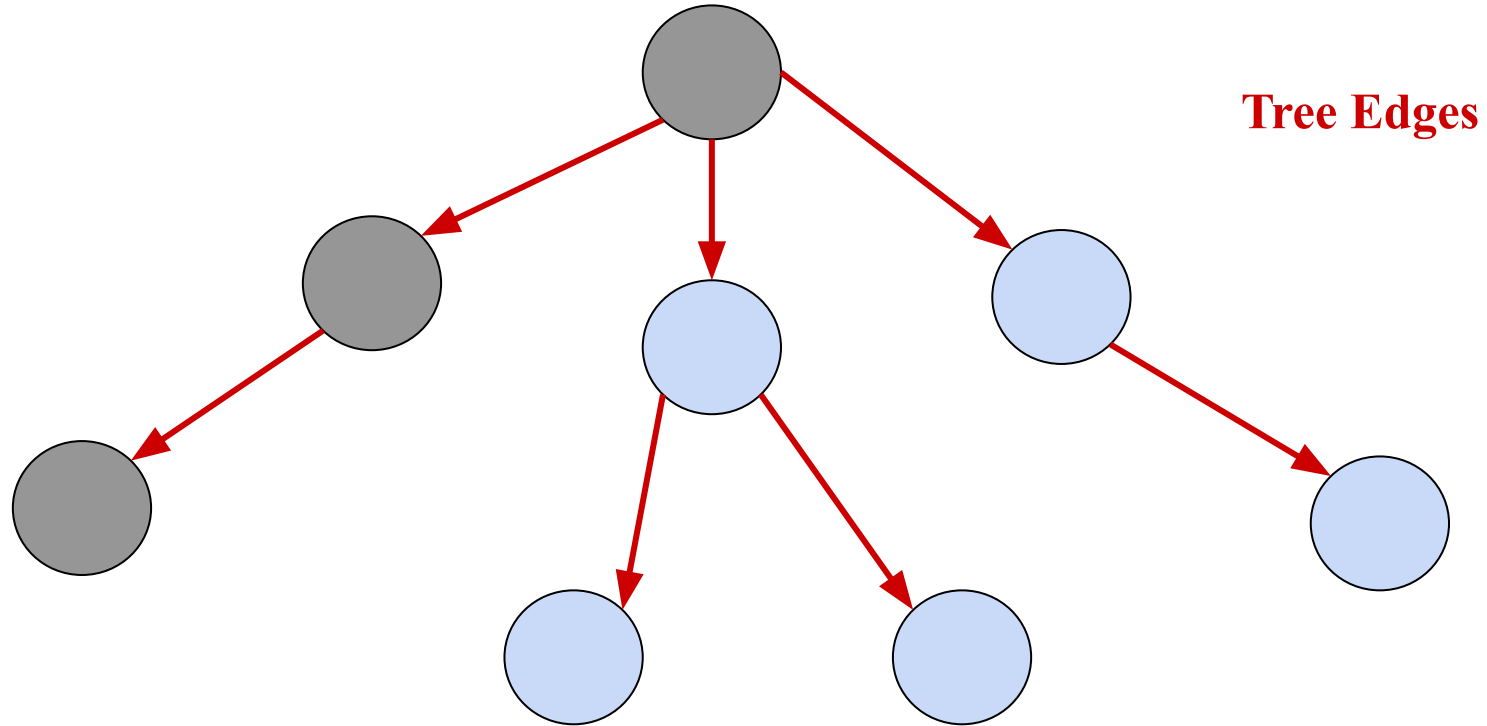
Depth First Search



Depth First Search

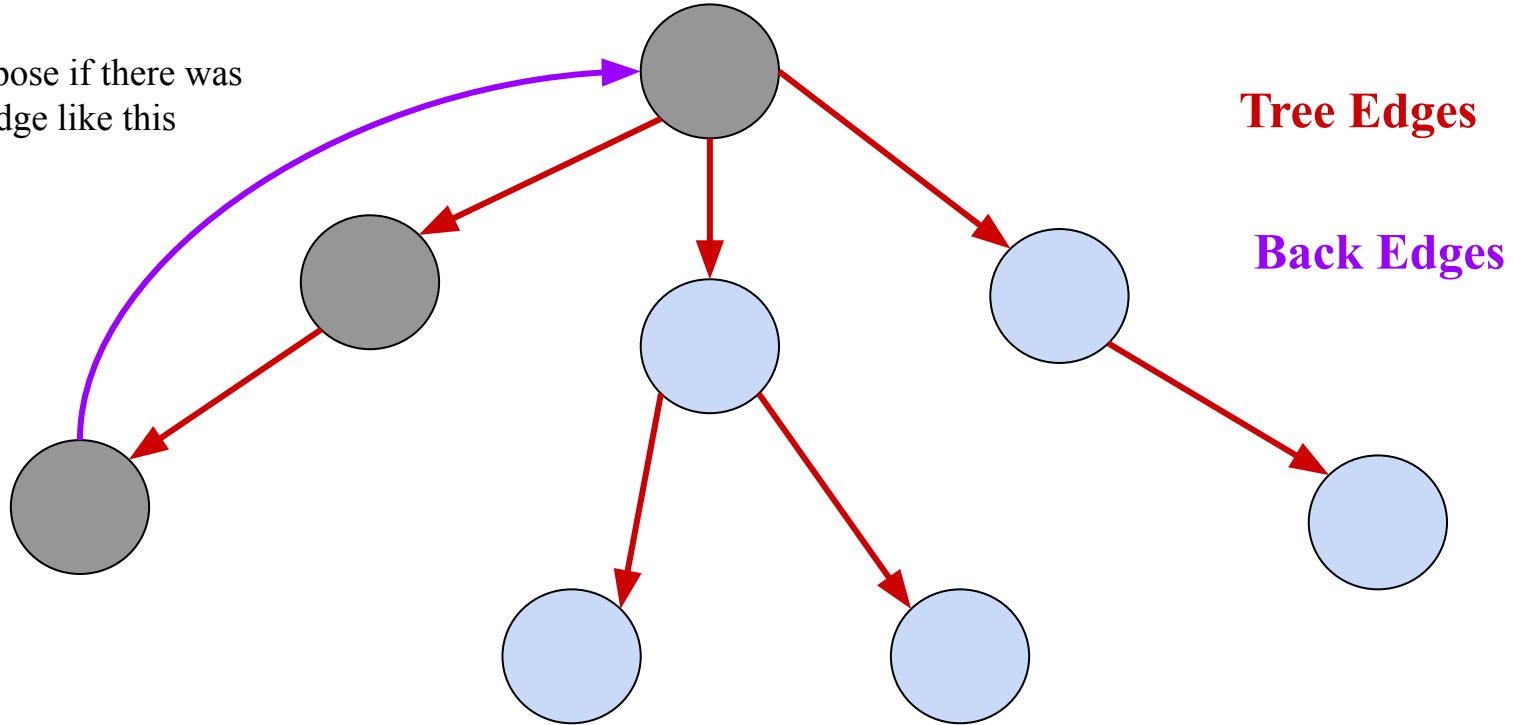


Depth First Search

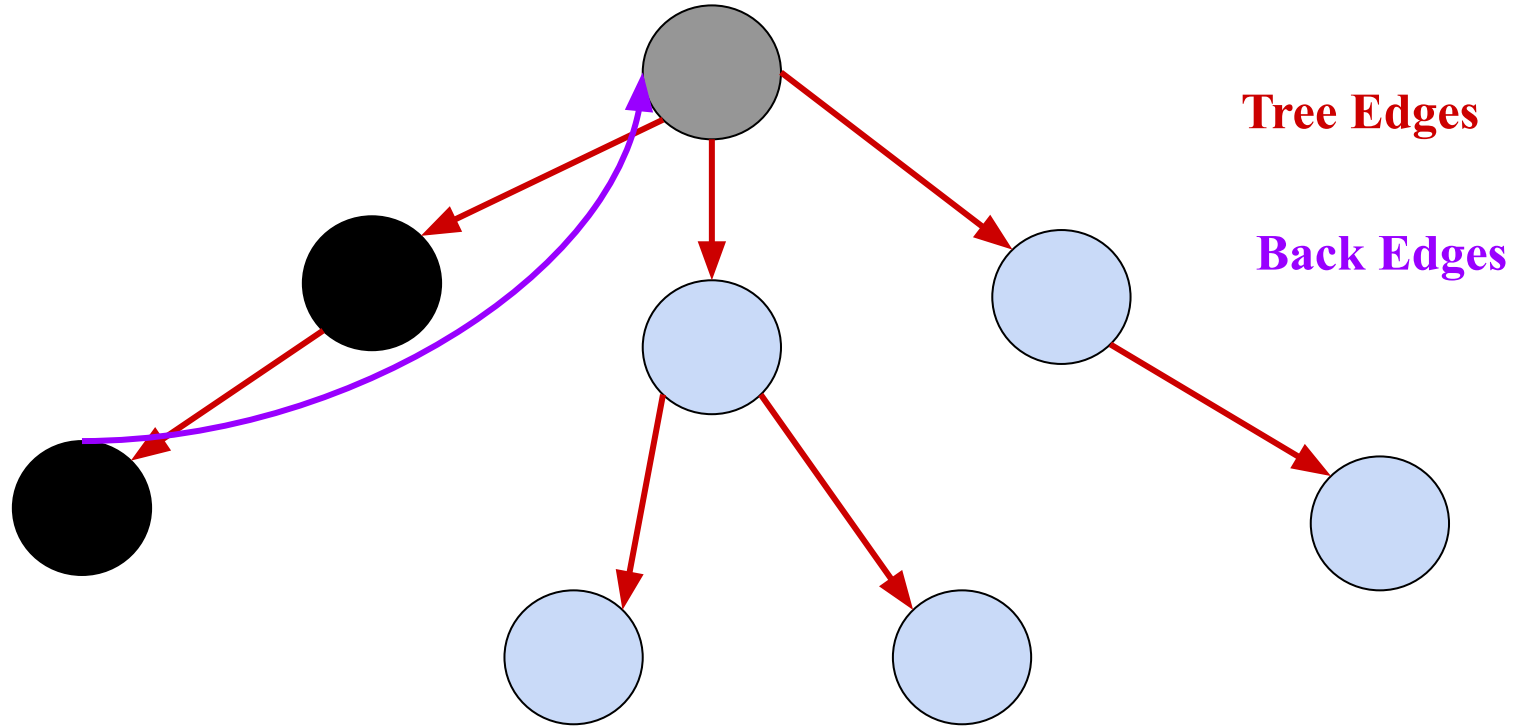


Depth First Search

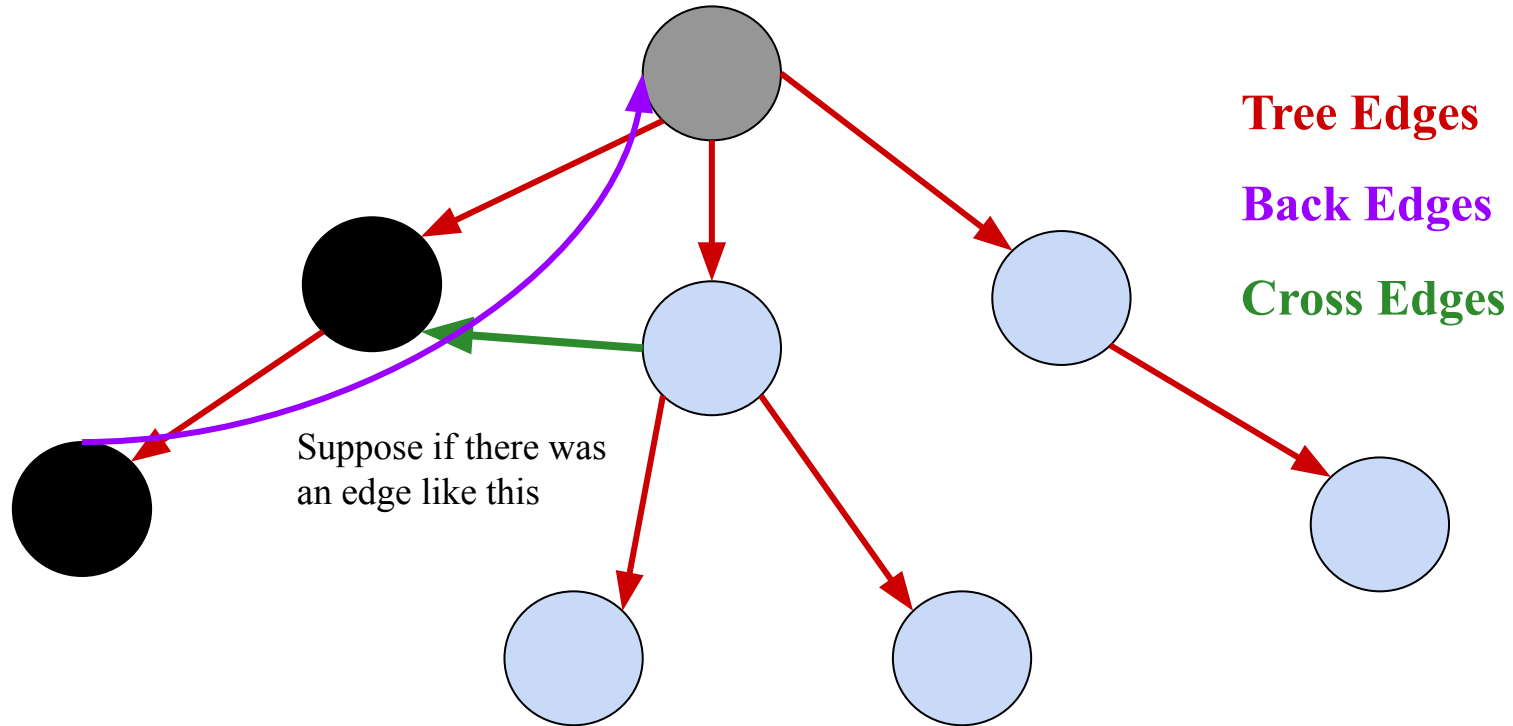
Suppose if there was
an edge like this



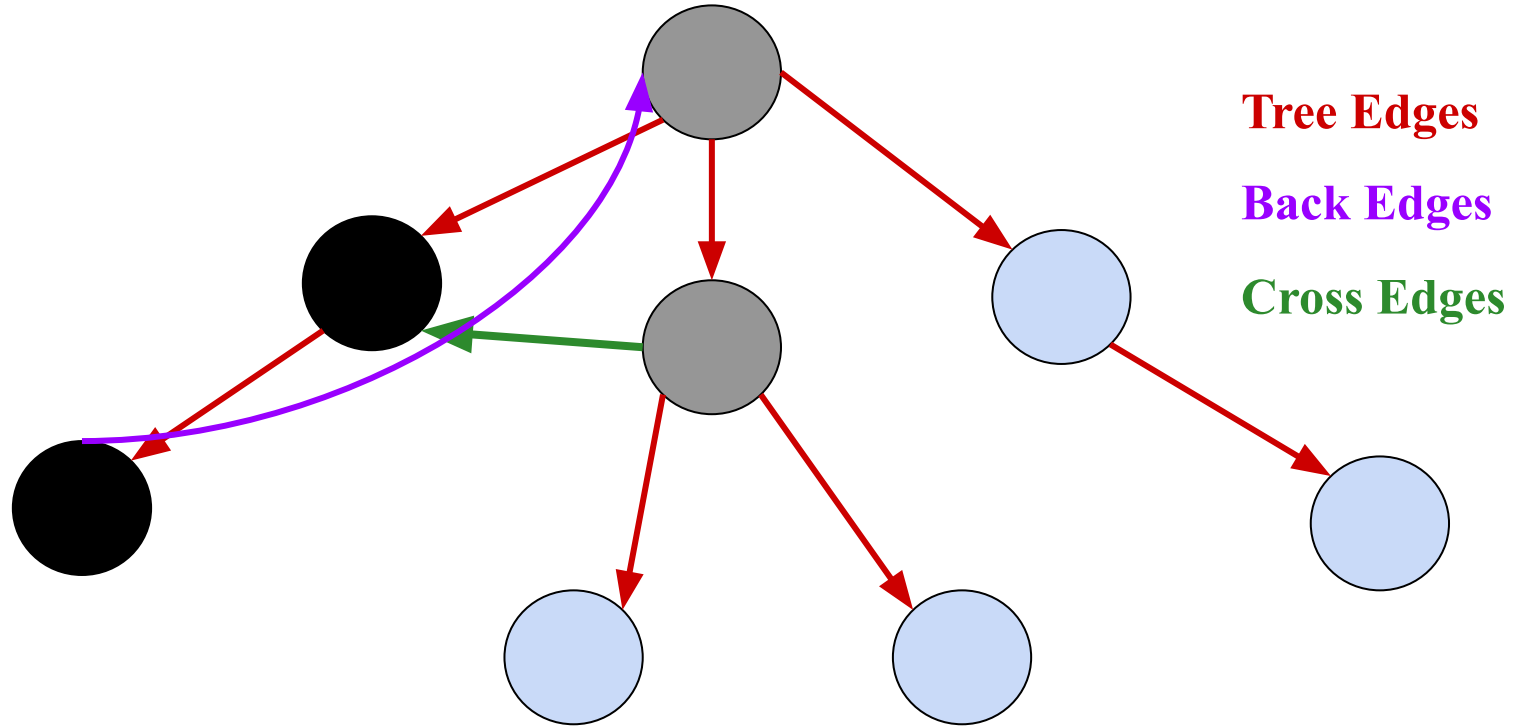
Depth First Search



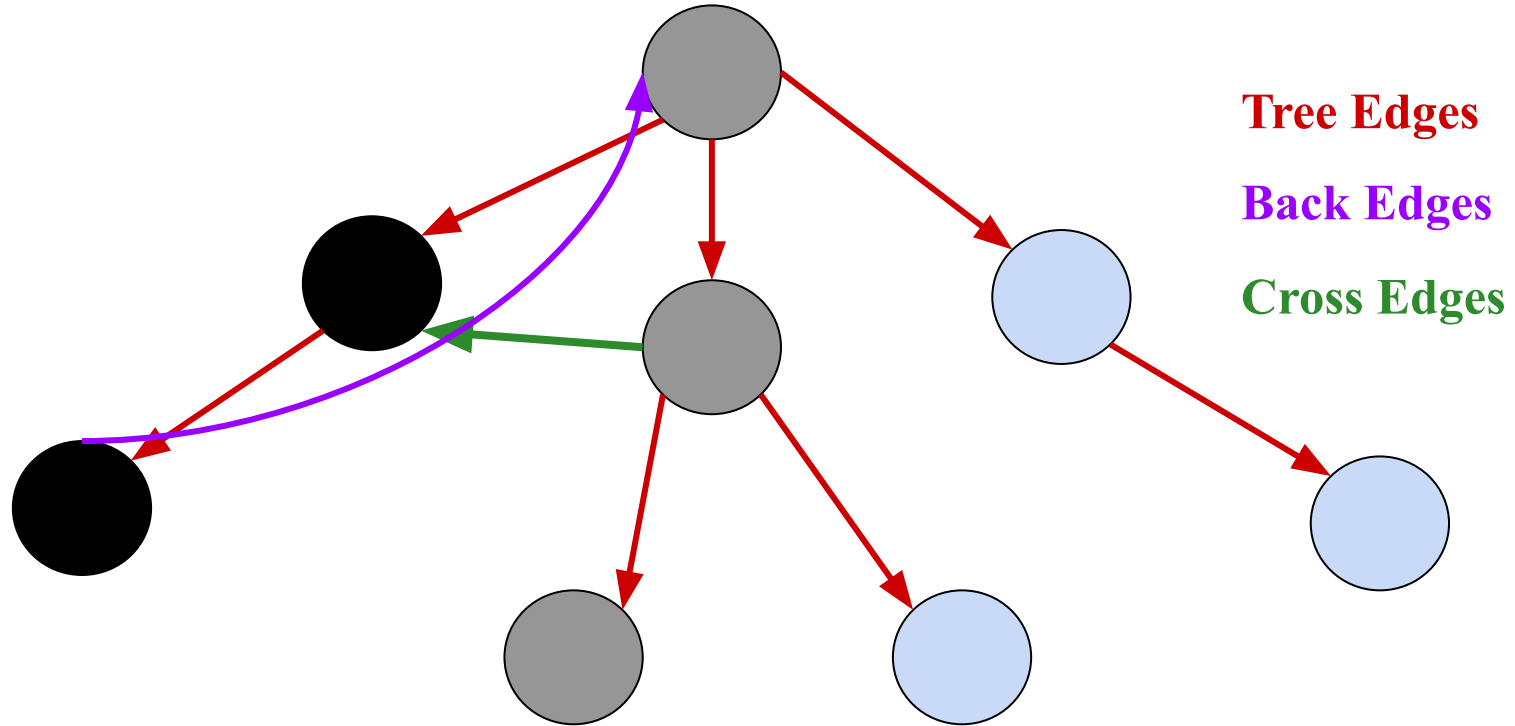
Depth First Search



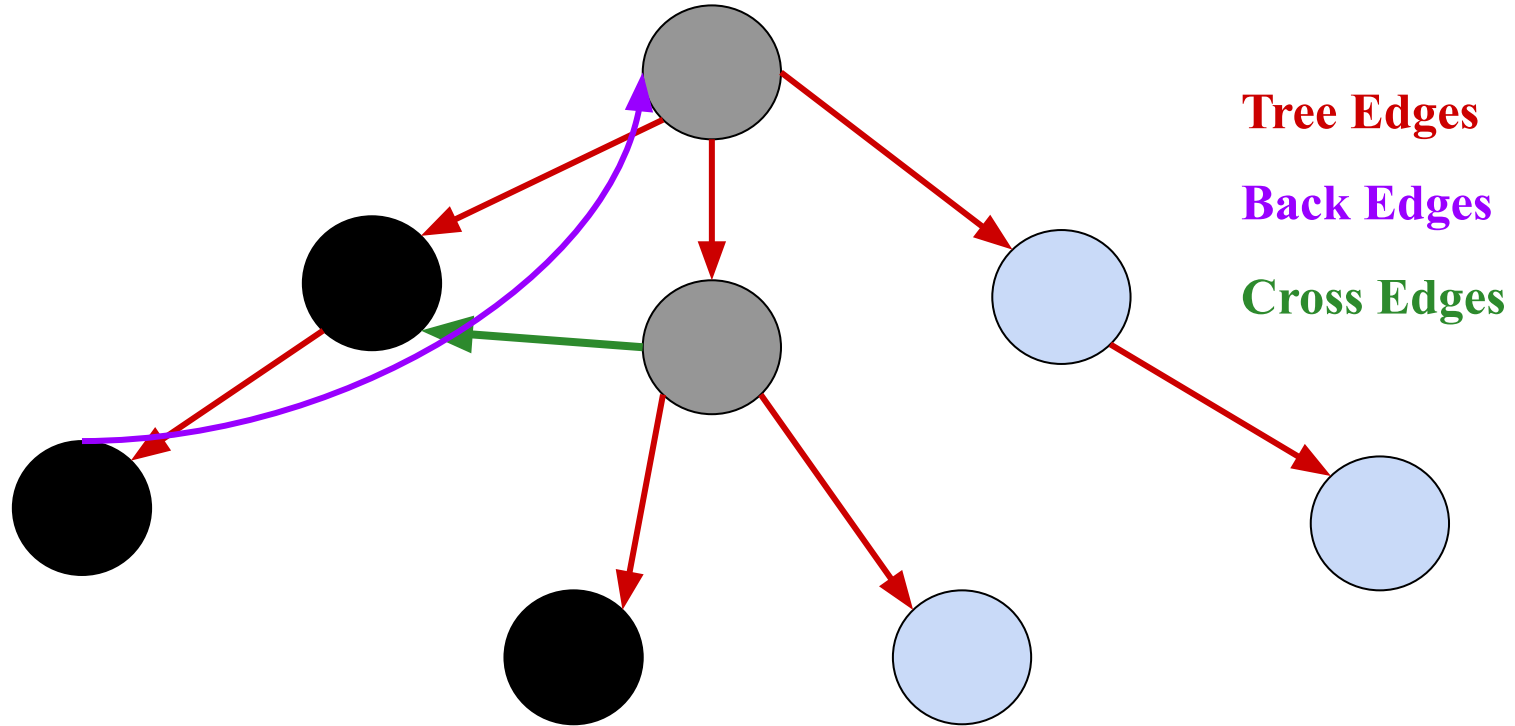
Depth First Search



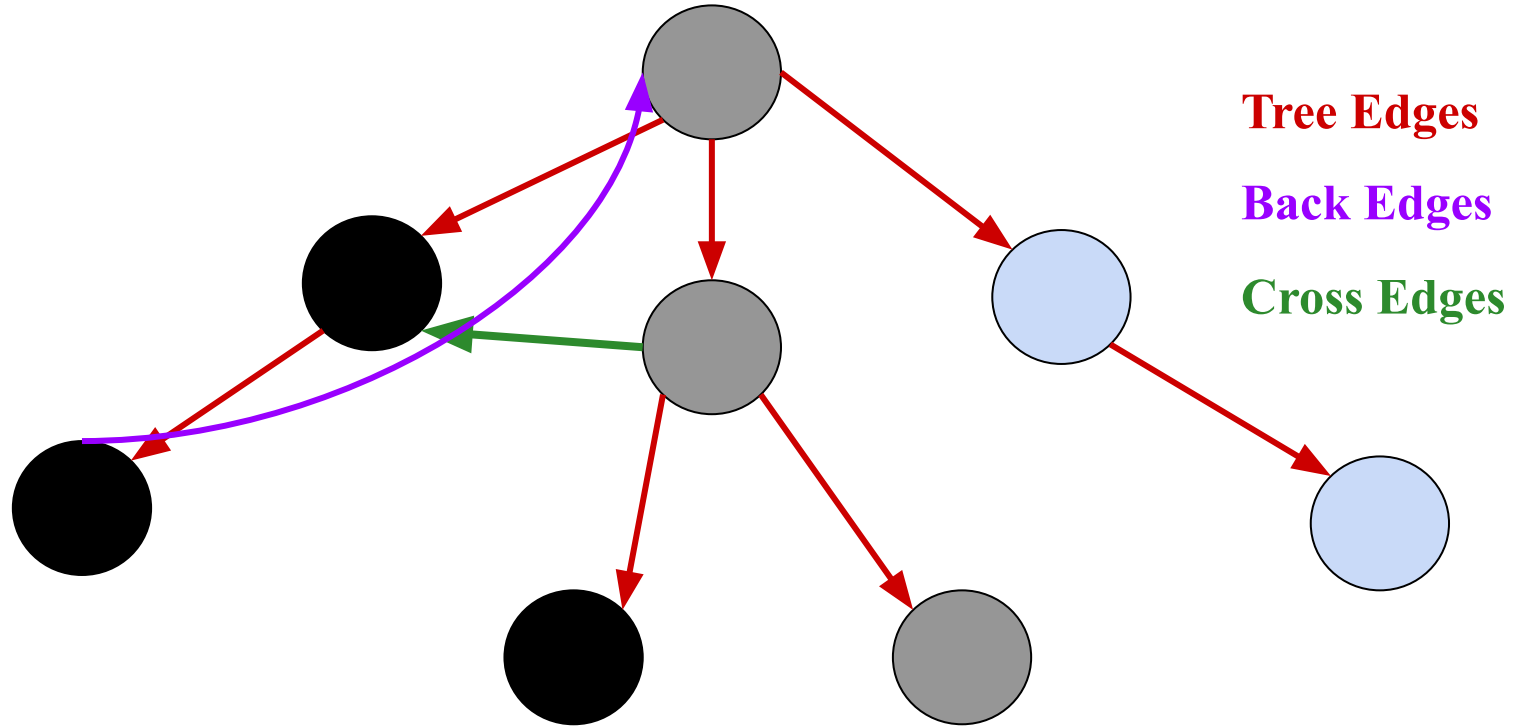
Depth First Search



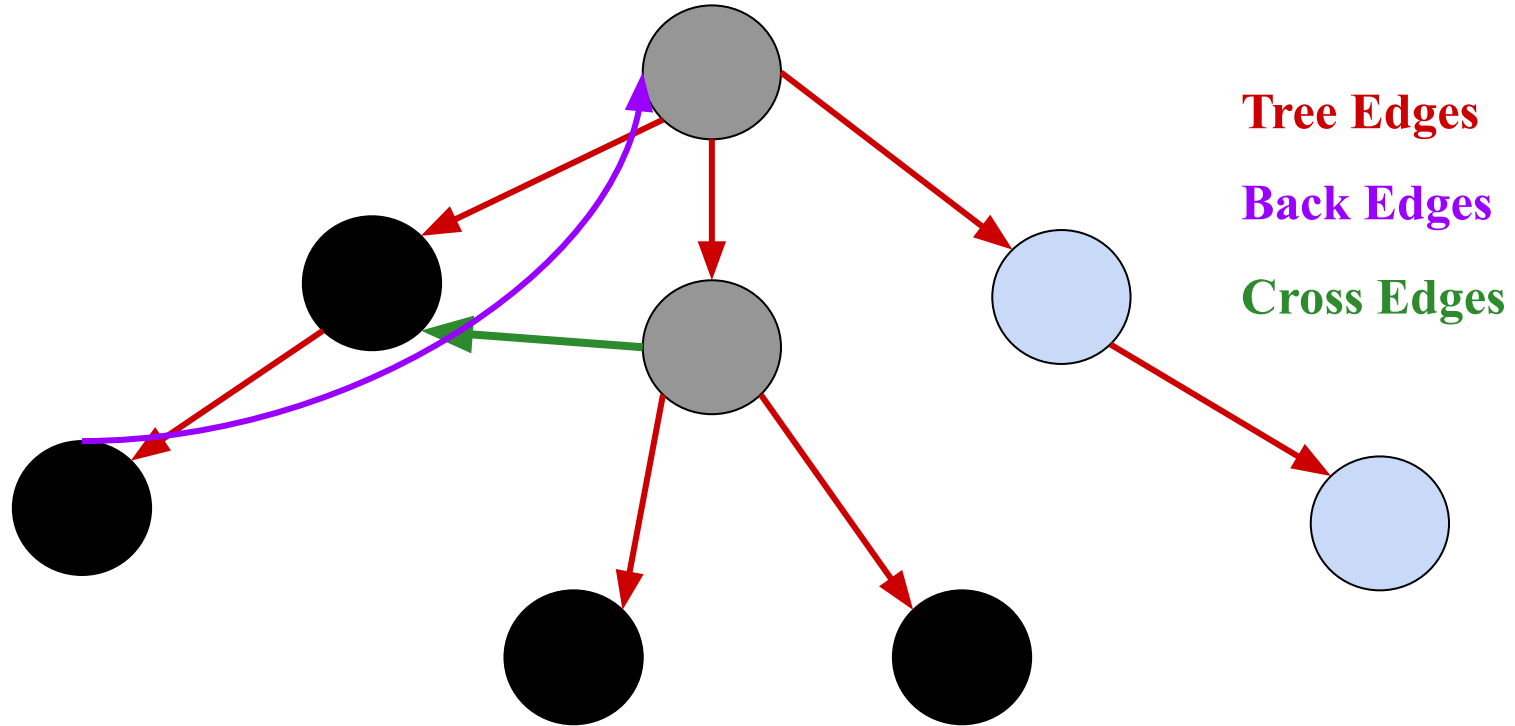
Depth First Search



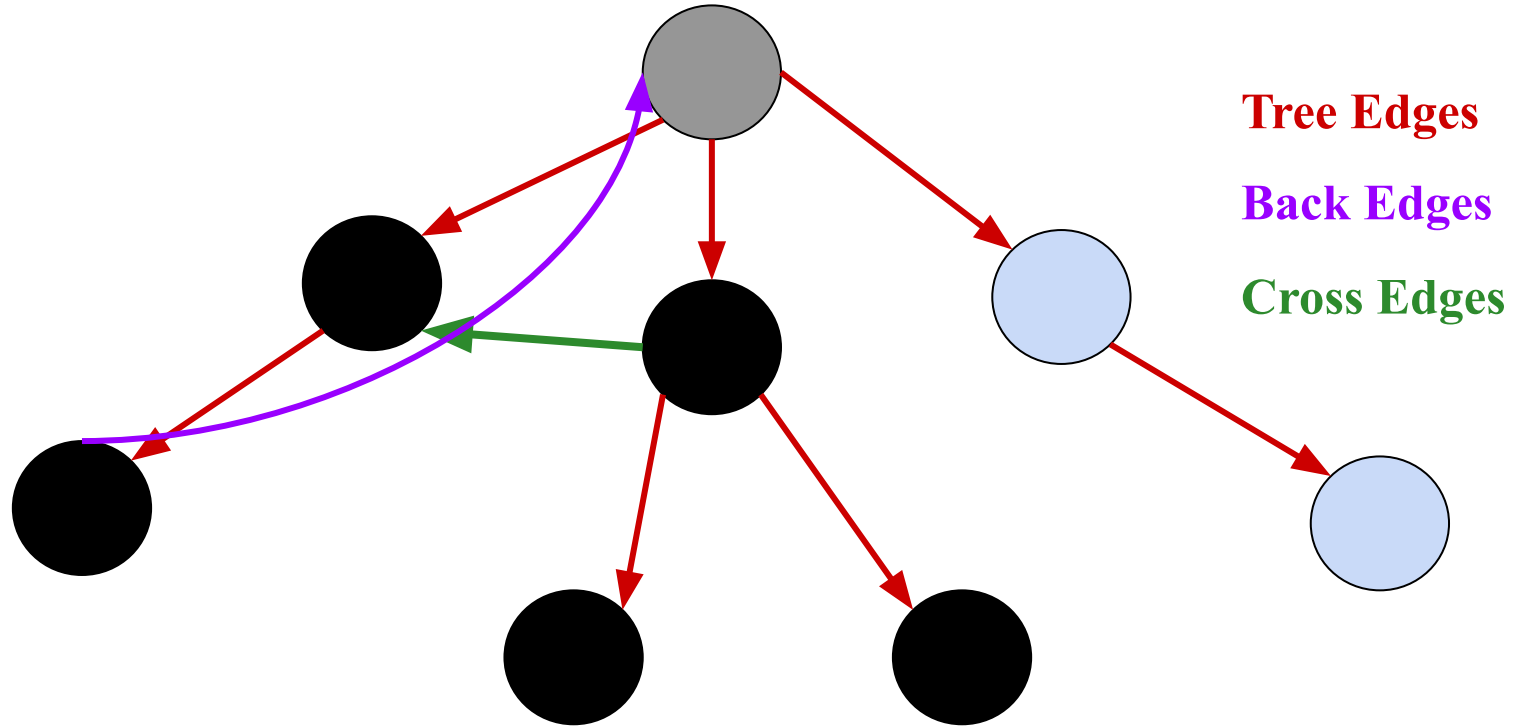
Depth First Search



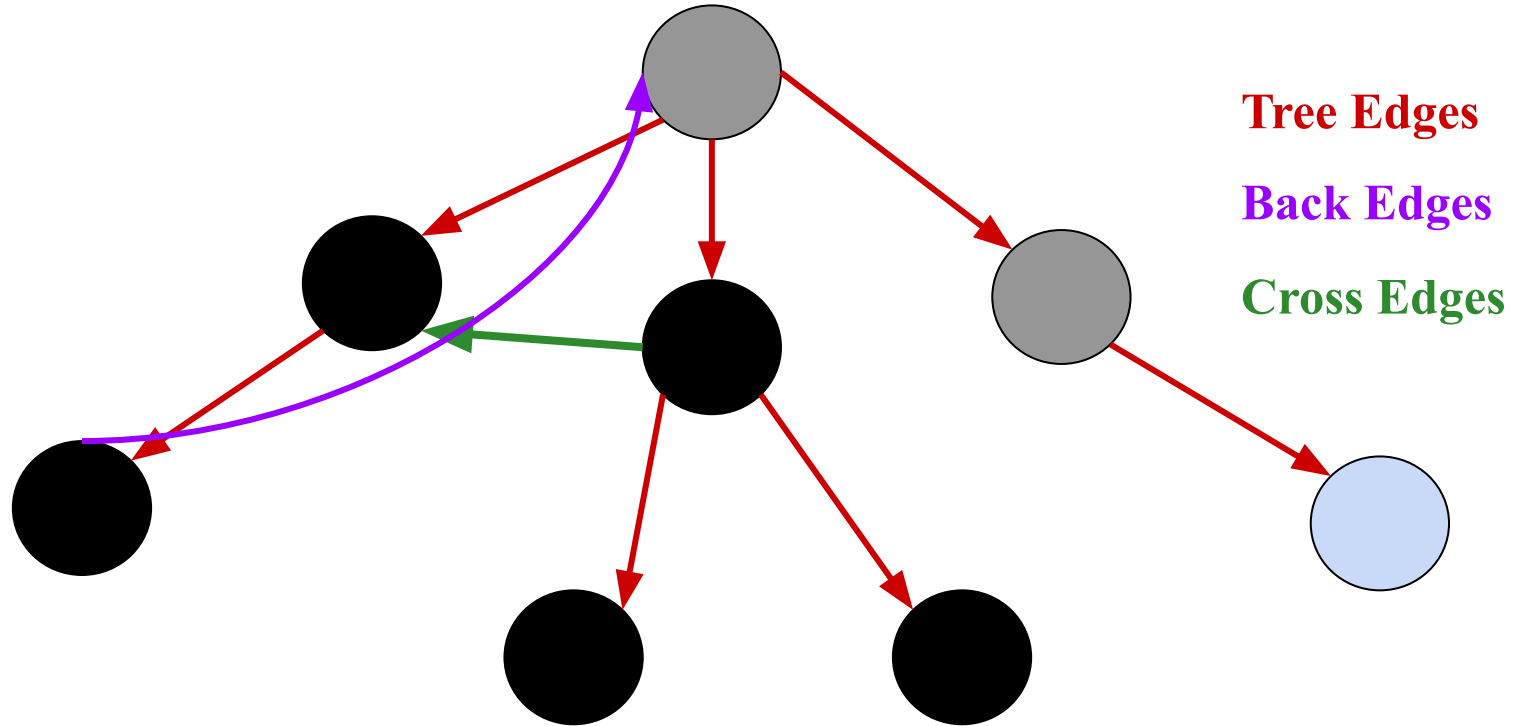
Depth First Search



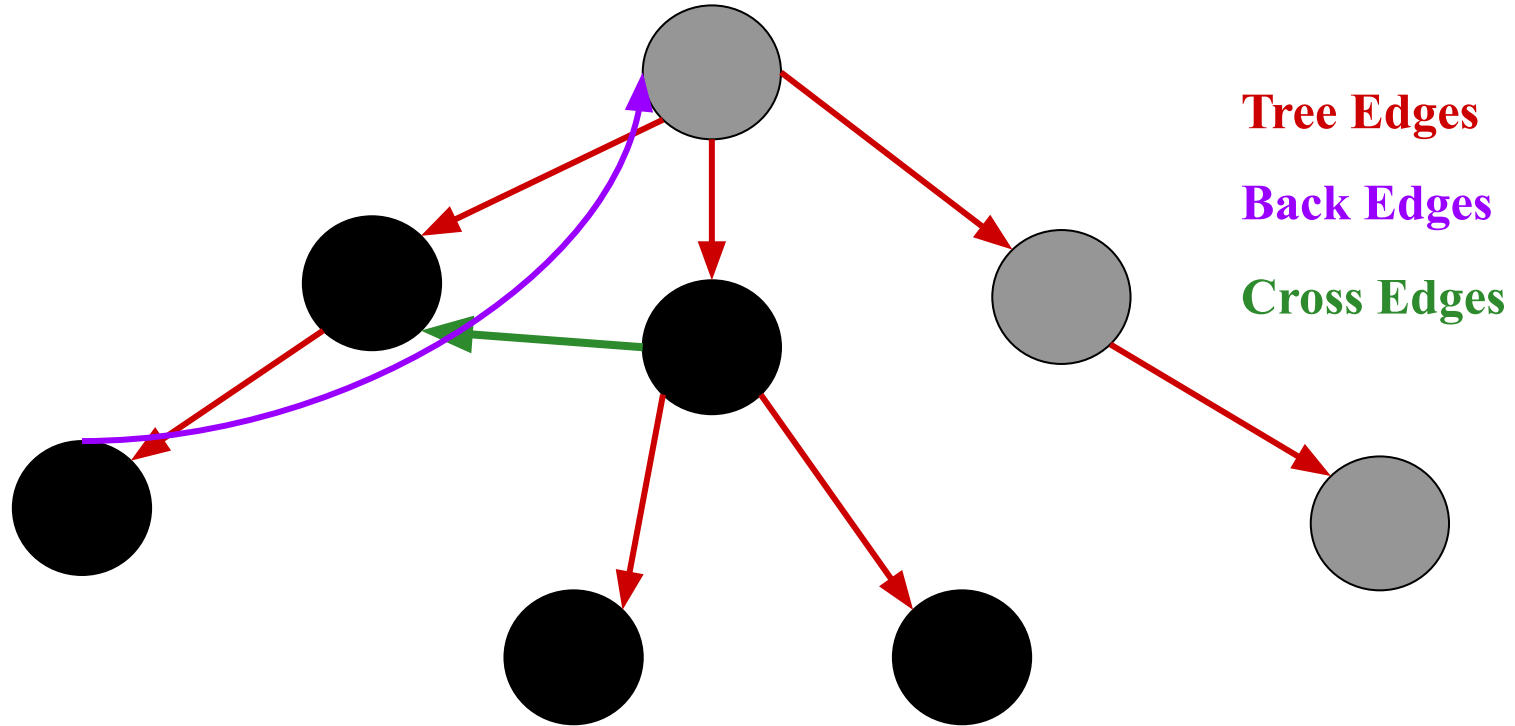
Depth First Search



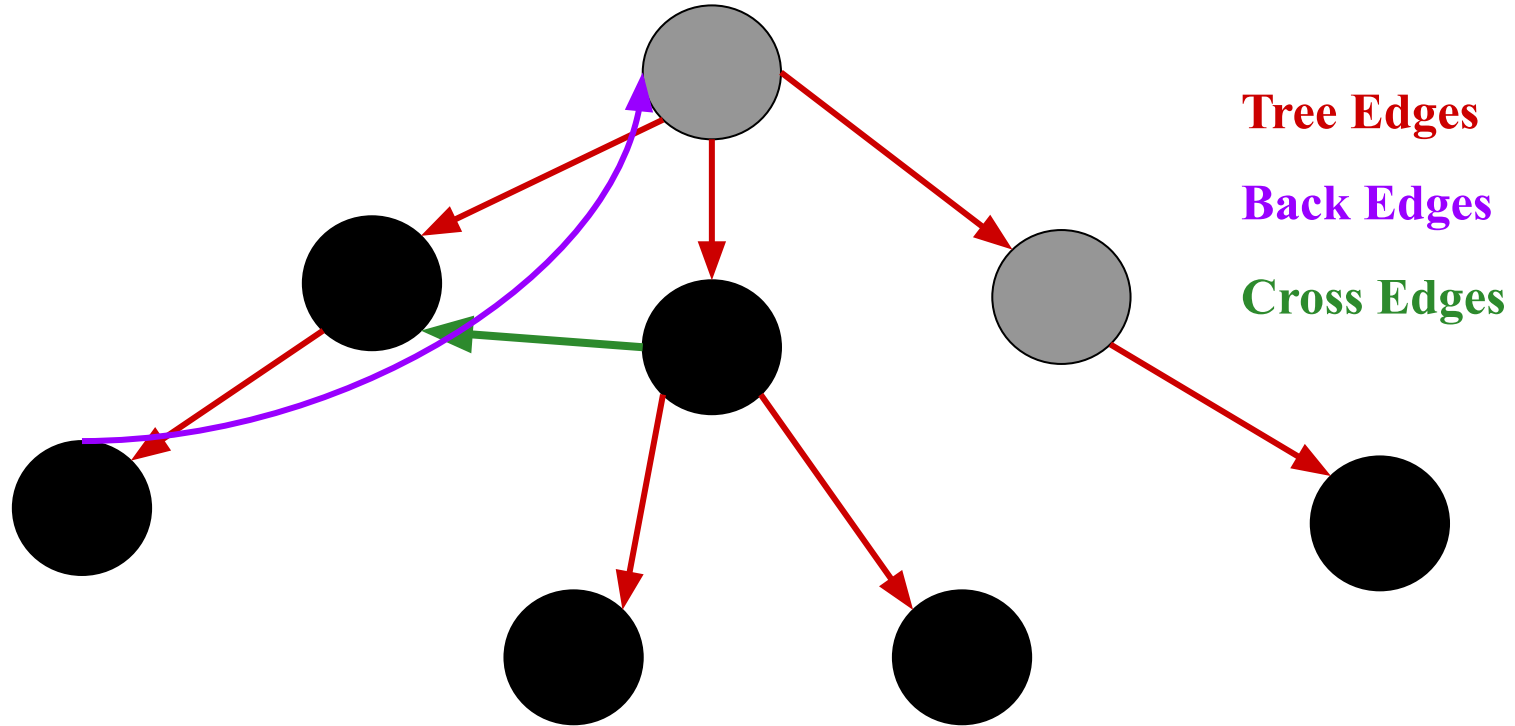
Depth First Search



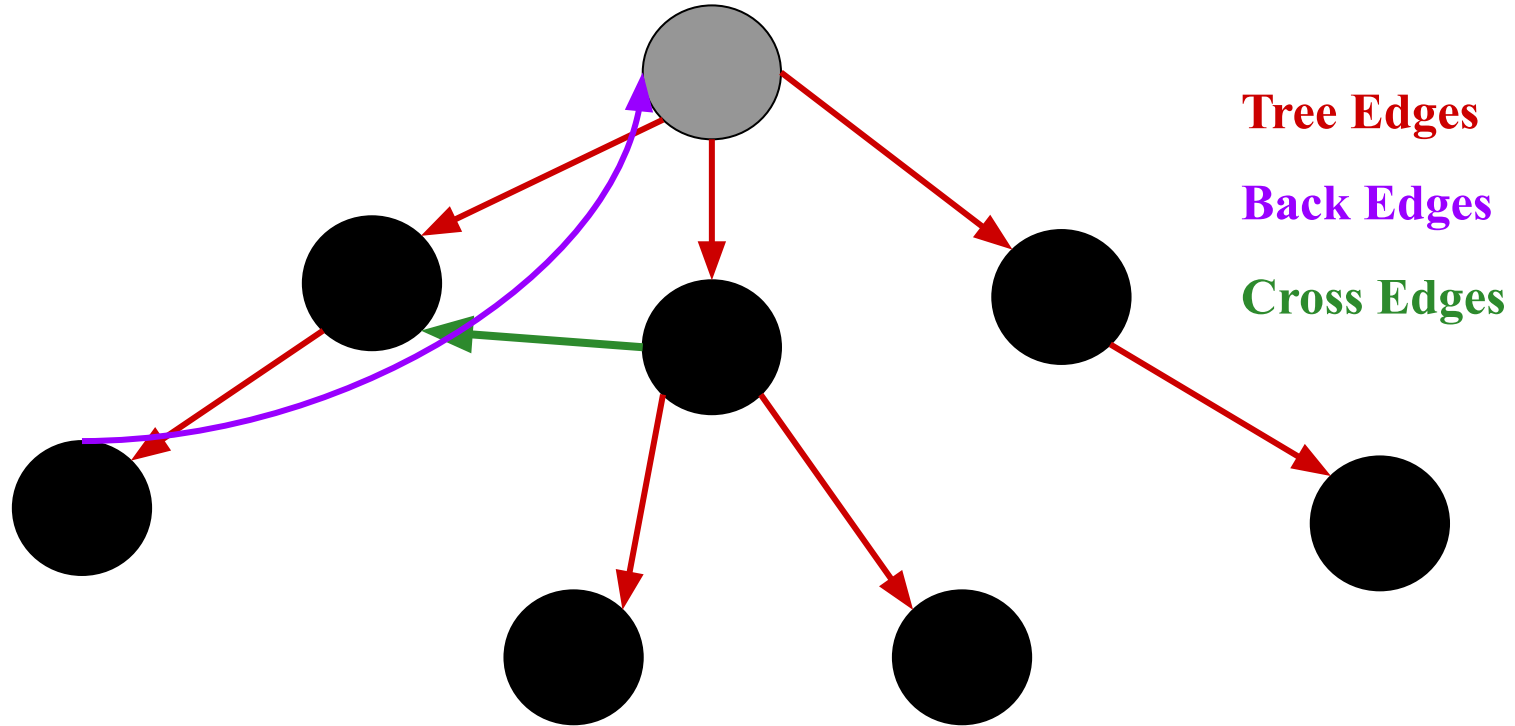
Depth First Search



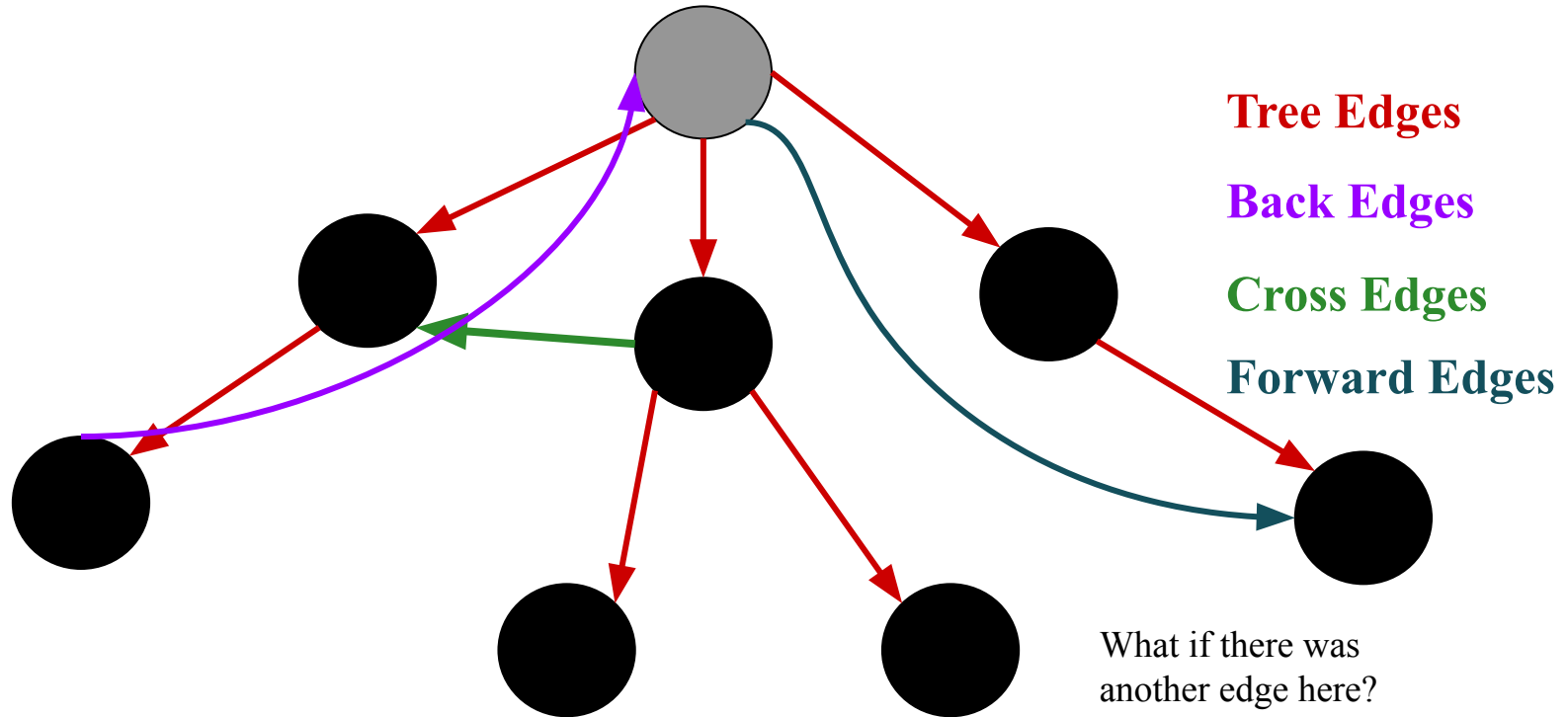
Depth First Search



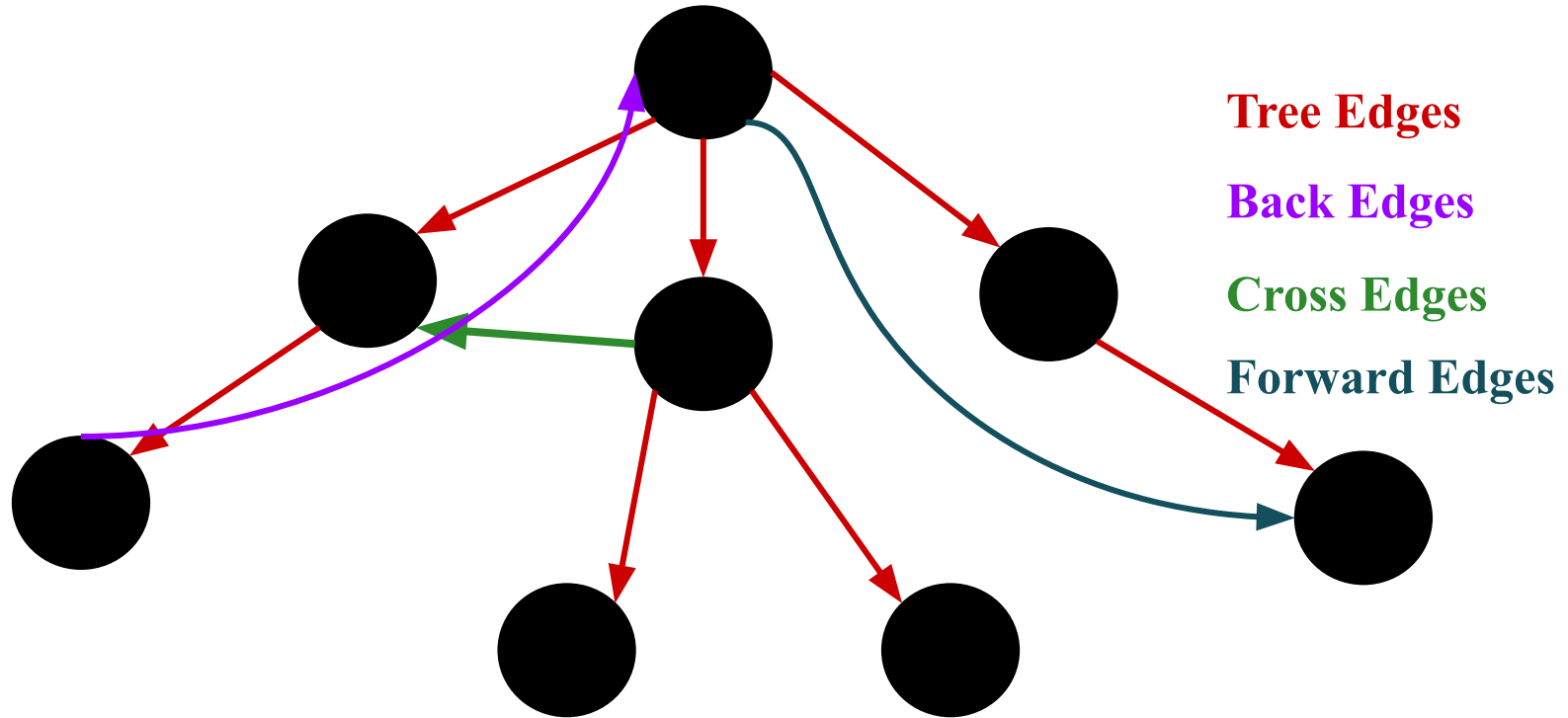
Depth First Search



Depth First Search

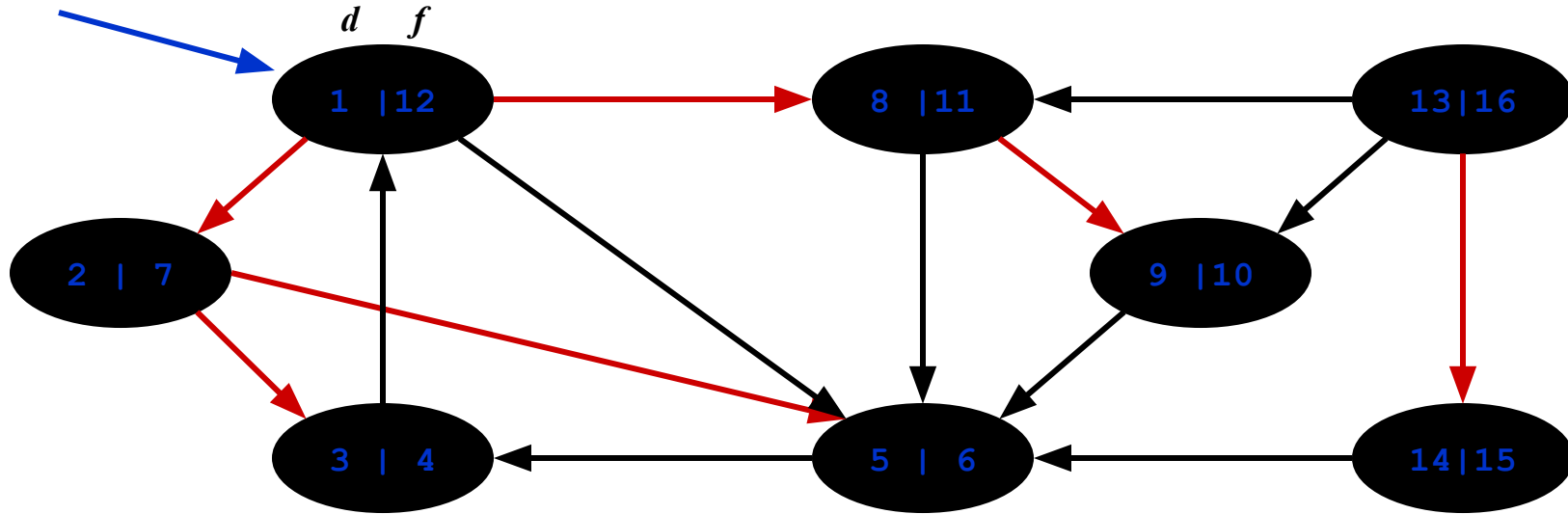


Depth First Search



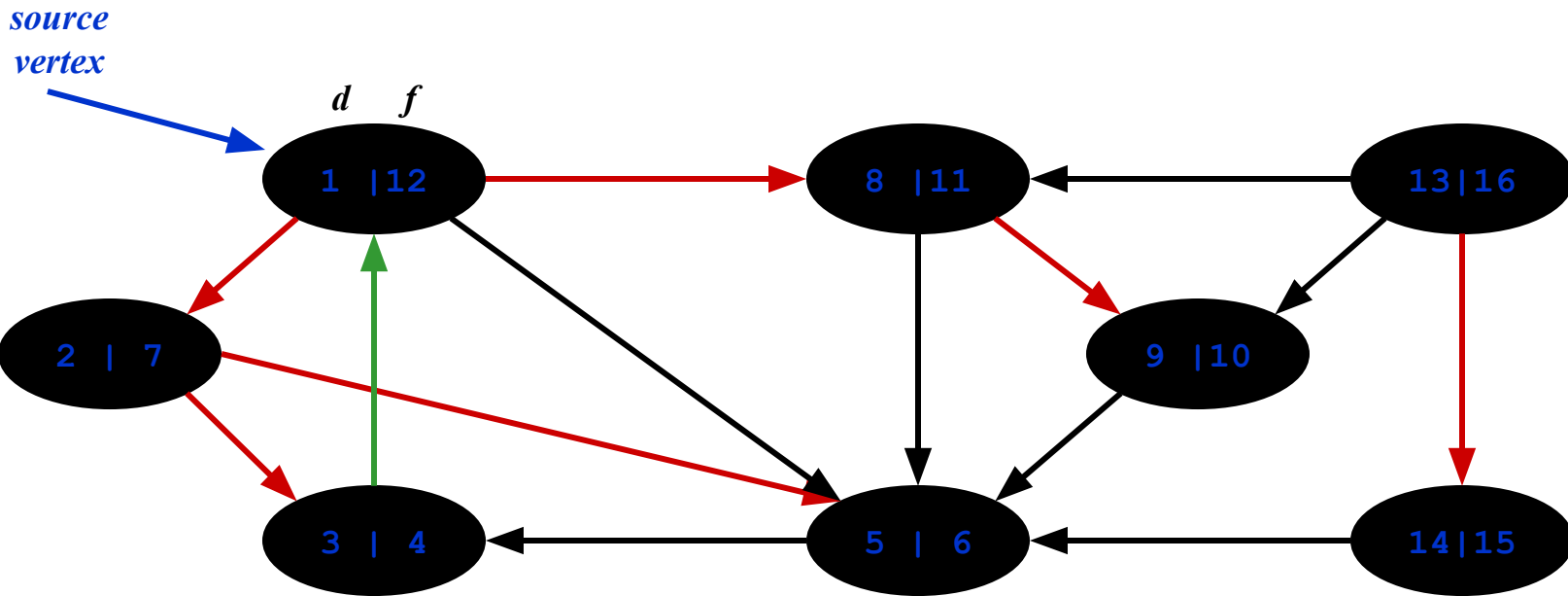
DFS Example

*source
vertex*



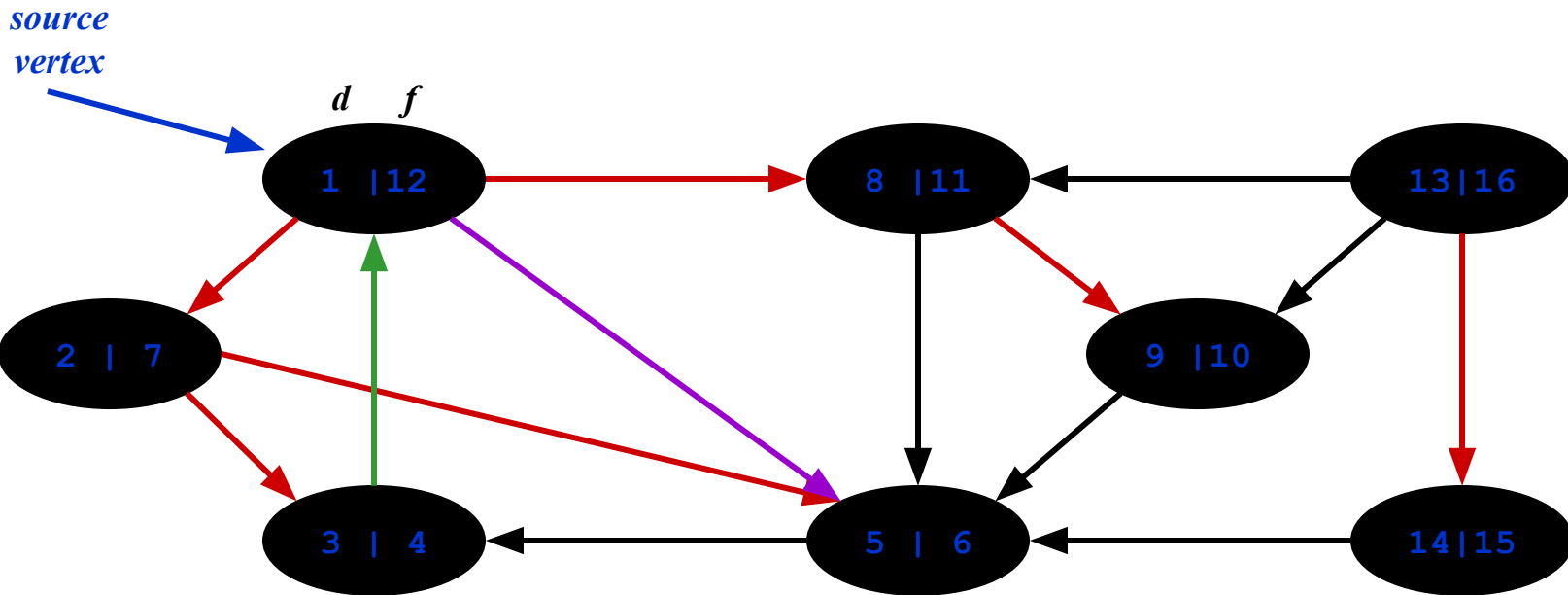
Tree edges

DFS Example



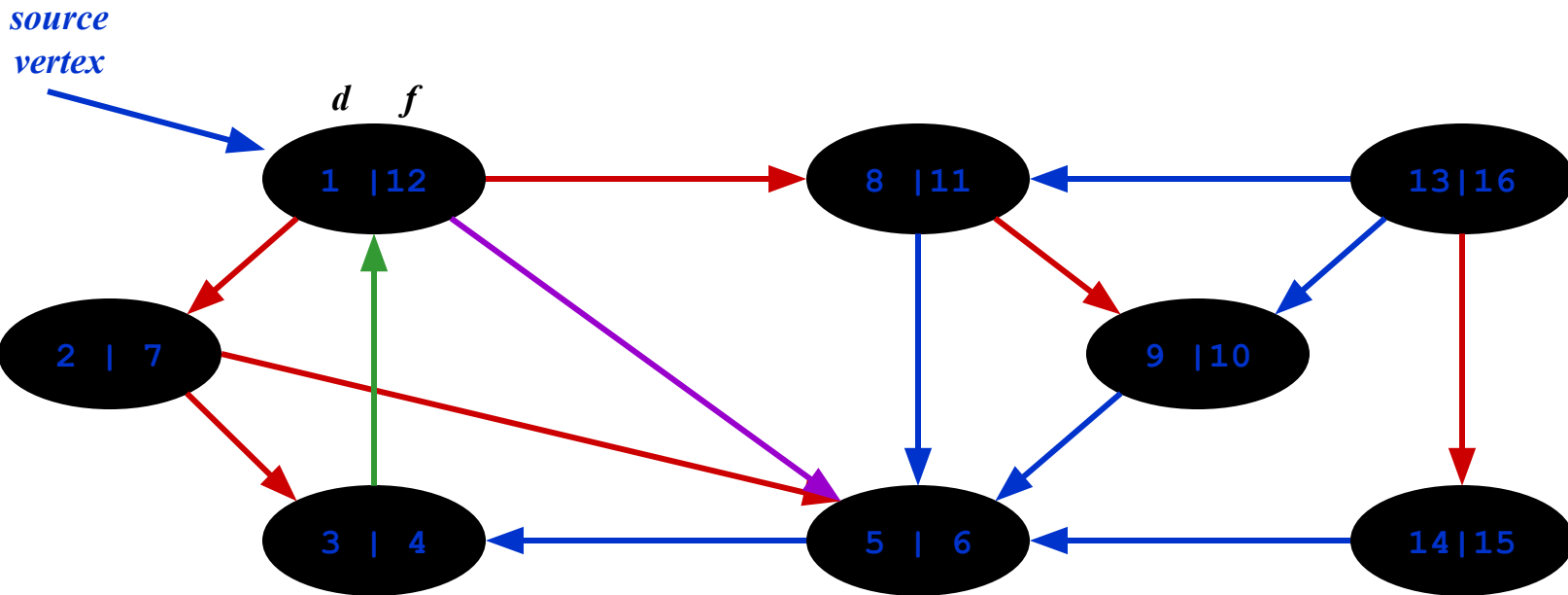
Tree edges *Back edges*

DFS Example



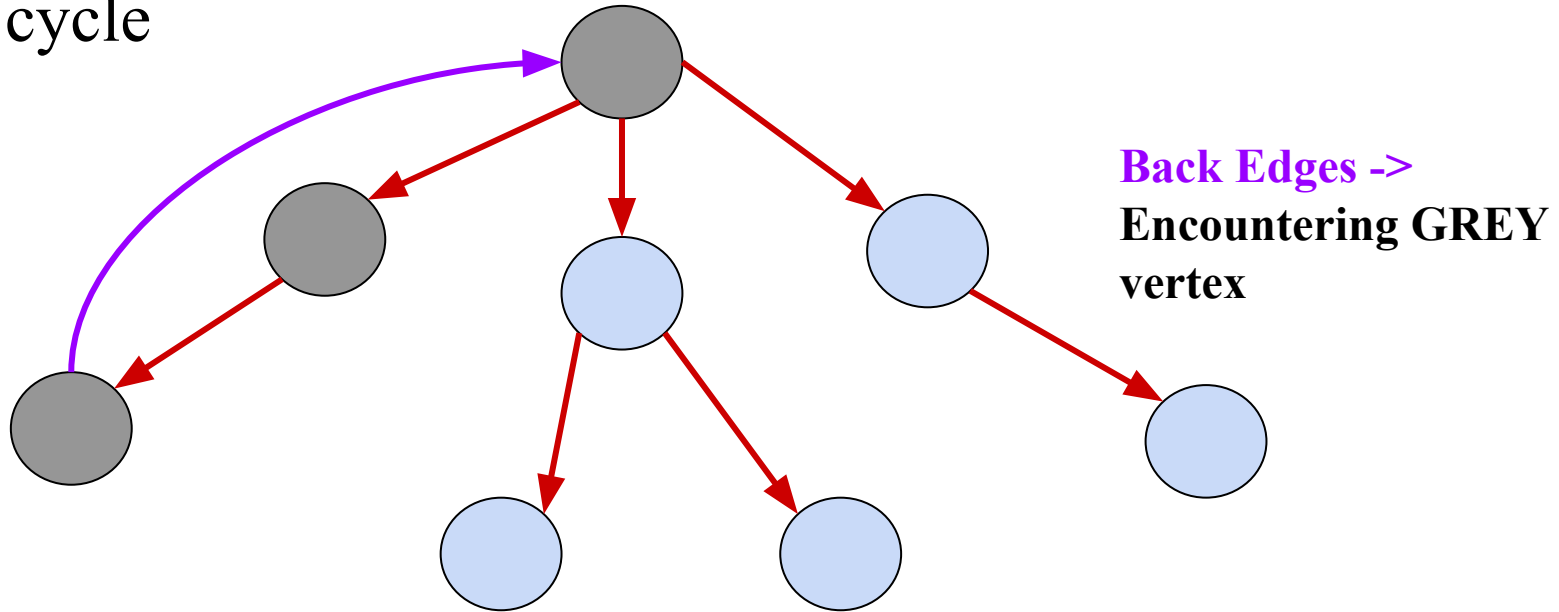
Tree edges *Back edges* *Forward edges*

DFS Example



Tree edges *Back edges* *Forward edges* *Cross edges*

- While running DFS, encountering a back edge implies a cycle



DFS And Cycles

- *How would you modify the code to detect cycles?*

```
DFS(G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit(v);
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

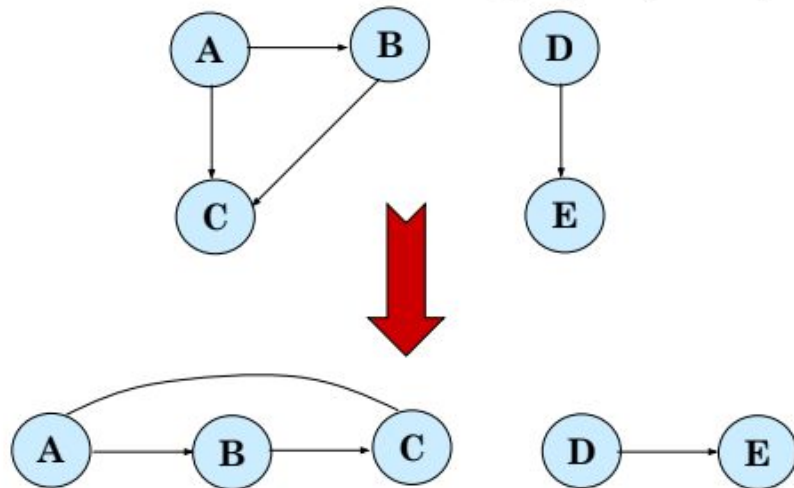
DFS And Cycles

```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
        if (v->color == GREY)
            print("cycle found!")
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

TOPOLOGICAL SORT

Want to “sort” a directed acyclic graph (DAG).



Think of original DAG as a **partial order**.

Want a **total order** that extends this partial order.

TOPOLOGICAL SORT

- ▣ Performed on a DAG.
- ▣ Linear ordering of the vertices of G such that if $(u, v) \in E$, then u appears somewhere before v .

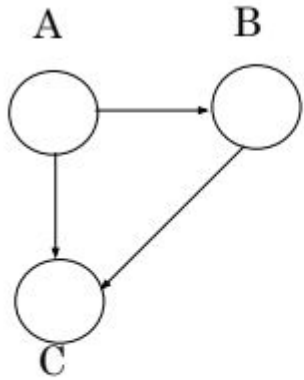
Topological-Sort (G)

1. call DFS(G) to compute finishing times $f[v]$ for all $v \in V$
2. as each vertex is finished, insert it onto the front of a linked list
3. **return** the linked list of vertices

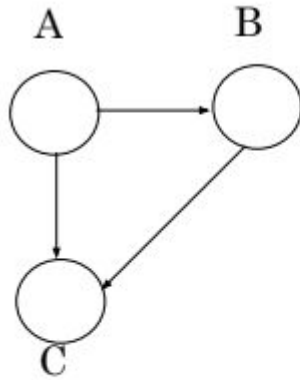
Time: $\Theta(V + E)$.

TOPOLOGICAL SORT

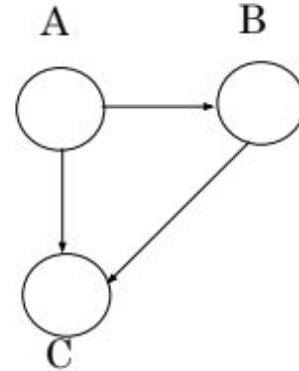
Example



Linked List:



Linked List:

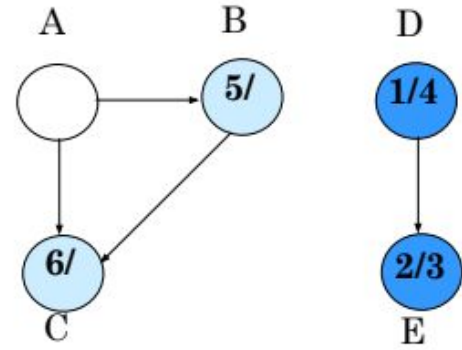
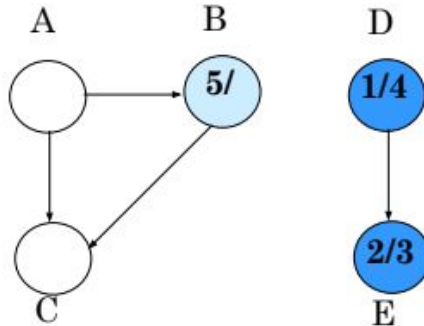
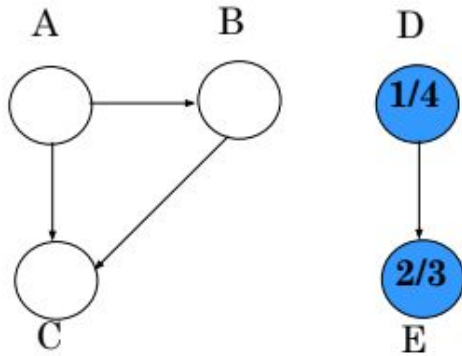


Linked List:

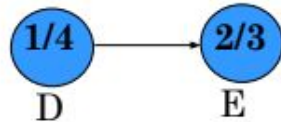


TOPOLOGICAL SORT

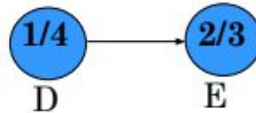
Example



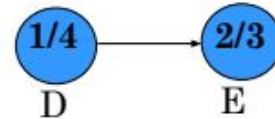
Linked List:



Linked List:

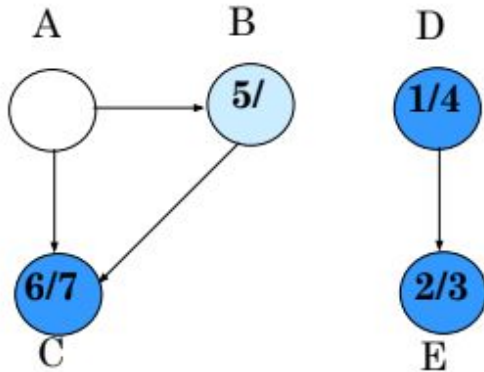


Linked List:

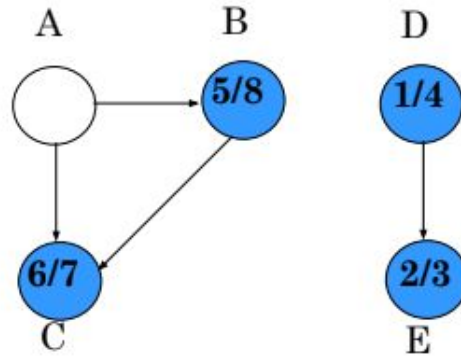
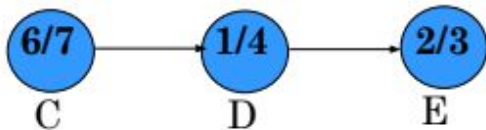


TOPOLOGICAL SORT

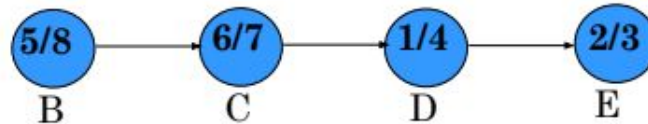
Example



Linked List:

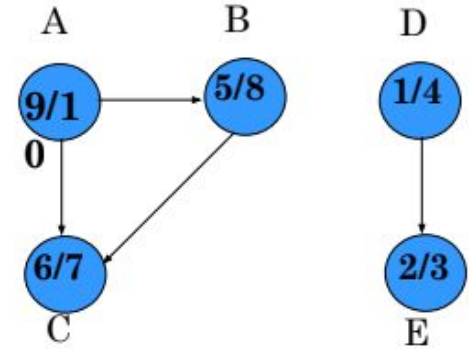
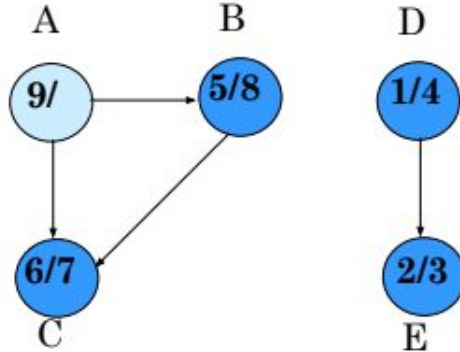


Linked List:

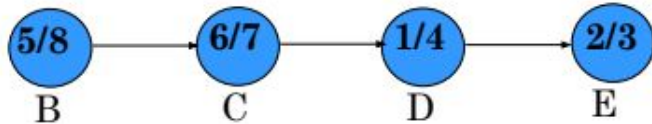


TOPOLOGICAL SORT

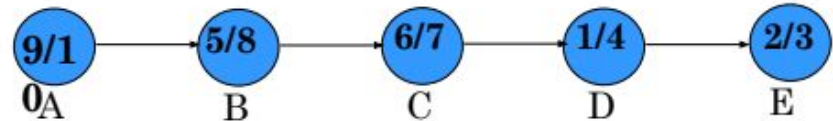
Example



Linked List:



Linked List:

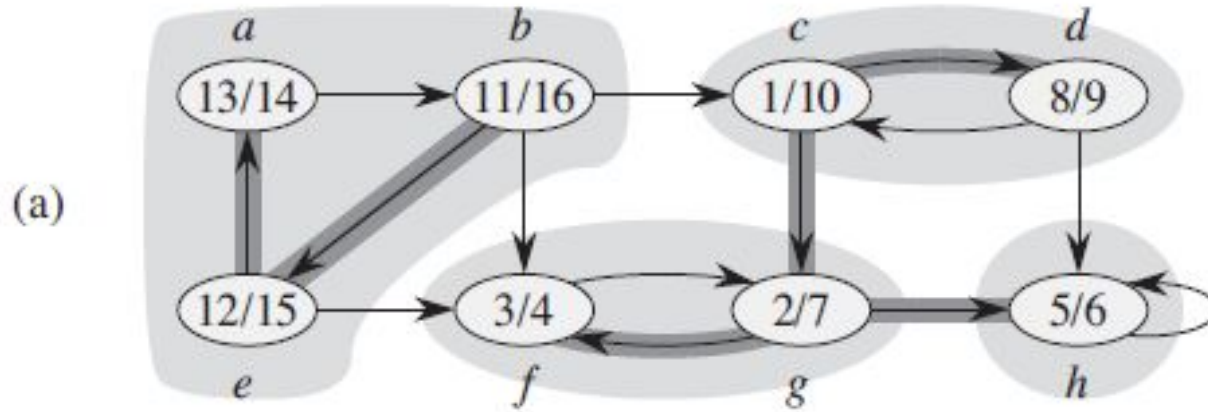


Strongly Connected Component

STRONGLY-CONNECTED-COMPONENTS(G)

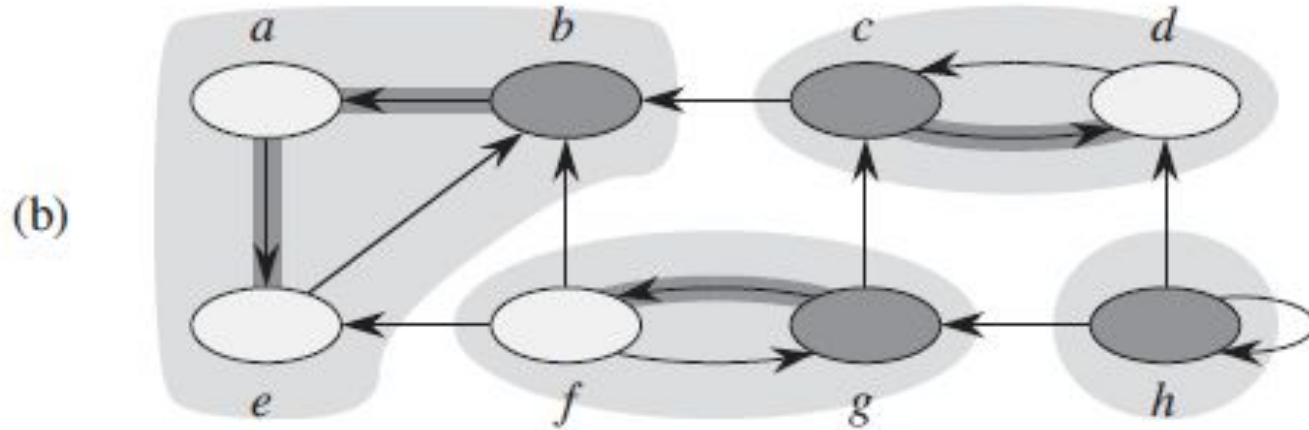
- 1 call DFS(G) to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Strongly Connected Component



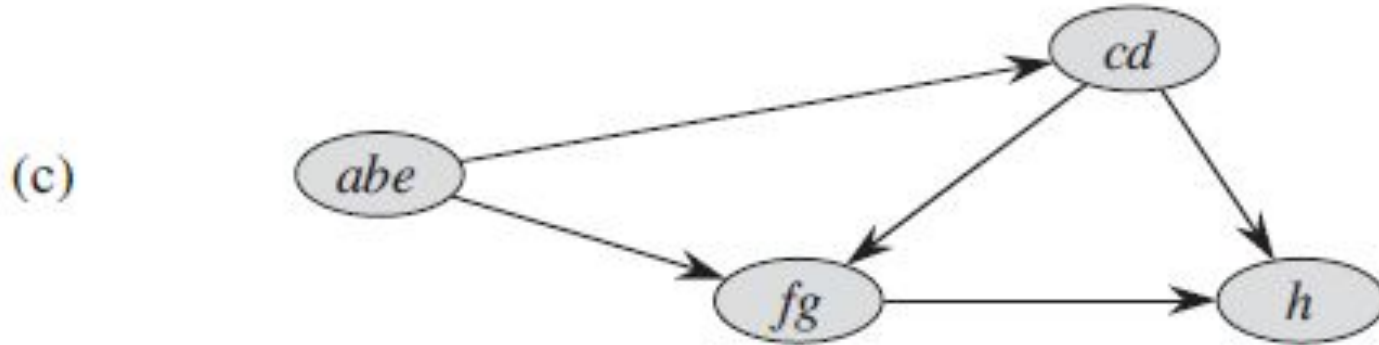
(a) A directed graph G . Each shaded region is a strongly connected component of G . Each vertex is labeled with its discovery and finishing times in a depth-first search, and tree edges are shaded.

Strongly Connected Component



(b) The graph G^T , the transpose of G , with the depth-first forest computed in line 3 of STRONGLY-CONNECTED-COMPONENTS shown and tree edges shaded. Each strongly connected component corresponds to one depth-first tree. Vertices b , c , g , and h , which are heavily shaded, are the roots of the depth-first trees produced by the depth-first search of G^T .

Strongly Connected Component



(c) The acyclic component graph G^{SCC} obtained by contracting all edges within each strongly connected component of G so that only a single vertex remains in each component.

Extra Materials

- The algorithm of finding strongly connected components using Depth First Search
 - CLRS book page 615
- Topological Sorting
 - CLRS book page 612
- See the “Problems related to BFS section”
 - [Breadth First Search or BFS for a Graph - GeeksforGeeks](#)
- For DFS, go through this link:
 - [Applications, Advantages and Disadvantages of Depth First Search \(DFS\) - GeeksforGeeks](#)
- Practice solving problems by coding